



ТЕХНОЛОГИЧНО УЧИЛИЩЕ "ЕЛЕКТРОННИ СИСТЕМИ"
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

ДИПЛОМНА РАБОТА

по професия код 481020 „Системен програмист“
специалност код 4810201 „Системно програмиране“

Тема: Boza - Game Engine

Дипломант:
Михаил Георгиев Георгиев

Дипломен ръководител:
Николай Димитров

СОФИЯ

2026



ТЕХНОЛОГИЧНО УЧИЛИЩЕ "ЕЛЕКТРОННИ СИСТЕМИ"
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

Дата на заданието: 17.11.2025 г.

Дата на предаване: 17.02.2026 г.

Утвърждавам:.....

/проф. д-р инж. П. Якимов/

ЗАДАНИЕ

за дипломна работа

ДЪРЖАВЕН ИЗПИТ ЗА ПРИДОБИВАНЕ НА ТРЕТА СТЕПЕН НА ПРОФЕСИОНАЛНА КВАЛИФИКАЦИЯ
по професия код 481020 „Системен програмист“
специалност код 4810201 „Системно програмиране“

на ученика Михаил Георгиев Георгиев от 12Б клас

1. Тема: Game Engine

2. Изисквания:

- 2.1. Система за рендериране, използваща слой за абстракция на графичния хардуер с реализация на Vulkan, поддържащ графични и изчислителни операции.
- 2.2. Система за зареждане и управление на материали и графични ресурси (текстури, семплери).
- 2.3. Основен игрови цикъл с управлението на игрови обекти чрез глобални системи, дефинирани от потребителя.
- 2.4. Система за създаване и управление на прозорци и обработка на входни събития.
- 2.5. Съвременна C++ архитектура, базирана на C++ модули и интеграция на външни зависимости чрез vcpkg.

3. Съдържание
- 3.1 Теоретична част
 - 3.2 Практическа част
 - 3.3 Приложение

Дипломант :.....

/ Михаил Георгиев /

Ръководител:.....

Николай Димитров /

ВРИД Директор.....

/ ст. пр. д-р Веселка Христова /



ТЕХНОЛОГИЧНО УЧИЛИЩЕ "ЕЛЕКТРОННИ СИСТЕМИ"
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

СТ А Н О В И Щ Е
КЪМ ДИПЛОМНА РАБОТА

Тема на дипломната работа: Game Engine

Ученик: Михаил Георгиев Георгиев

Клас: 12б

Професия: код 481020 „Системен програмист“

Специалност: код 4810201 „Системно програмиране“

Дипломен ръководител: Николай Димитров

Дипломната работа е посветена на една амбициозна и комплексна тема - разработка на игрови енджин. В процеса на работа дипломантът демонстрира добра инженерна култура, системно мислене и способност да осмисля и прилага сложни технически концепции.

Изграден е работещ прототип/заготовка на енджин с модулна организация и няколко ясно разграничени, добре развити подсистеми. Графичният слой е реализиран на принципа „Vulkan-first“ и е изграден върху абстракция за хардуерния интерфейс (RHI). Впечатление прави подходът към шейдърите - разработен е отделен инструмент, който ги компилира до SPIR-V и подготвя нужните варианти за различни платформи. Решение, което е характерно за реални енджини и показващо добро разбиране на процеса от компилация, през ресурси, до изпълнение. Дипломантът е осмислил мащаба на задачата и е ограничил обхвата до реалистичен и изпълним в рамките на дипломния проект, без компромис в инженерното качество.

По време на разработката са преодолени и нетривиални практически затруднения, включително такива, свързани с модулна организация на проекта и интеграция на външни библиотеки/инструменти (напр. конфигурации на системата за изграждане, компилационни ограничения при модерни C++ подходи). Това показва устойчивост, умение за самостоятелно проучване и довеждане на технически решения докрай.

Всичко дотук изложено дава основание ученикът да бъде допуснат до защита пред комисията за провеждане на държавен изпит за придобиване на професионална квалификация по теория и практика на специалността. Предлагам дипломната работа да бъде оценена с отличен.

Предлагам за рецензент Валери Димов Христов, бакалавър Софтуерно инженерство от Софийски университет "Св. Климент Охридски", Expert Game Programmer във фирма Геймлофт България ЕООД, valeri.dhristov@gmail.com, 0886 943 699.

17.02.2025 г.

Дипломен ръководител:.....

Николай Димитров

Технически Директор,

Геймлофт България ЕООД

УВОД

Компютърните игри се основават на софтуерни системи, които функционират в непрекъснат цикъл на работа в реално време. В рамките на всеки кадър приложението обработва вход от потребителя, обновява логиката и състоянието на обектите в сцената, изчислява техните трансформации в рамките на йерархични структури, подготвя данни за графичния процесор, подава команди за изобразяване и визуализира резултата на екрана. Едновременно с това се управляват ресурси като текстури, буфери и материали, осъществява се синхронизация между централния и графичния процесор и се поддържа връзка с операционната система. Всички тези процеси са взаимно зависими и изискват точно подредено и предвидимо изпълнение, за да се избегнат графични дефекти, проблеми с производителността и трудно проследими грешки.

В практиката често се използват утвърдени игрови двигатели, които предоставят готови инструменти и решения за широко разпространени задачи като тези. Съществуват обаче проекти, при които е необходим по-прецизен контрол върху графичния слой, по-специфичен модел на организация на данните или по-висока степен на автоматизация на задачите от ниско ниво.

Целта на настоящата дипломна работа е проектирането и реализирането на Boza - лек модулен игрови двигател на C++23 с поетапен модел на изпълнение от тип ECS, слой за абстракция на графичния хардуер (RHI) с реализация на Vulkan, система за управление на ресурси и материали и инструментална верига за обработка на шейдъри, генерираща метаданни за използване по време на изпълнение. Реализираният обхват включва управление на жизнения цикъл на приложението, измерване на времето за кадър, logging инструмент, управление на сцени и йерархии от обекти, коректно прилагане на трансформации в родителско-дъщерни зависимости, автоматизирано обвързване на материали и текстури, изпълнение на изчислителни (compute) шейдъри с възможност за синхронно или асинхронно изчакване, обработка на вход, управление на прозорец, както и примерни приложения. Поради ограниченото време за разработка проектът е насочен към изграждане на стабилна архитектурна основа, а не към постигане на пълна функционалност на завършен продуктов двигател.

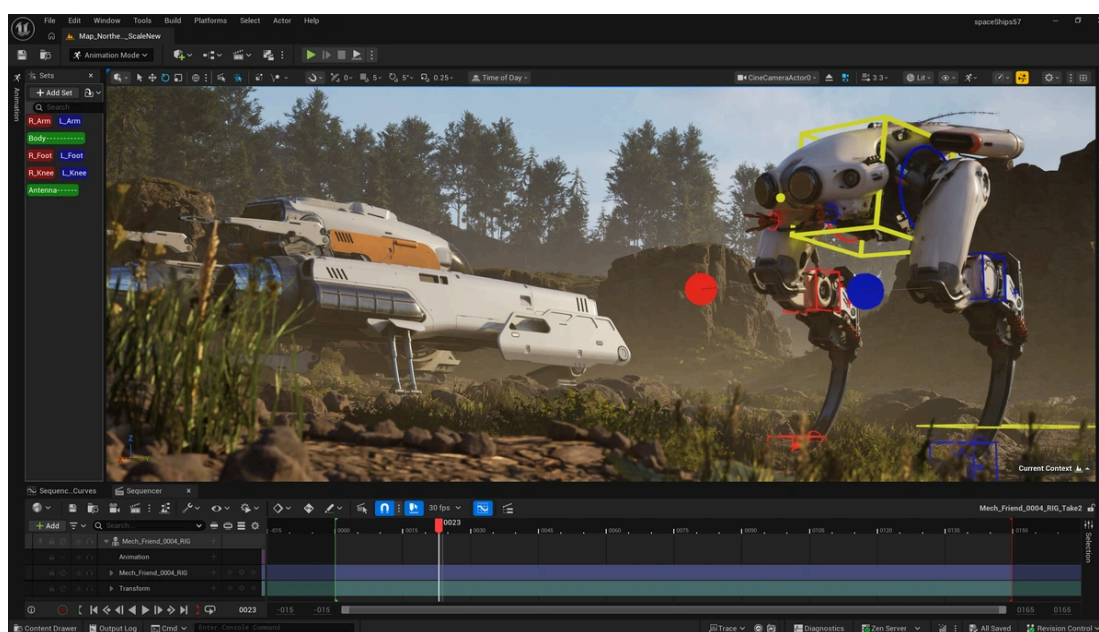
ГЛАВА 1. ПРЕГЛЕД НА СЪЩЕСТВУВАЩИ РЕШЕНИЯ И ПРАКТИКИ ПРИ РАЗРАБОТКАТА НА ИГРОВИ ДВИГАТЕЛИ

1.1 Категории игрови двигатели и типични профили на употреба

Изборът на категория игрови двигател зависи от мащаба на проекта, състава на екипа, необходимото ниво на контрол, сроковете за разработка и наличния ресурс за поддръжка. Не съществува универсално оптимално решение; най-подходящият избор се определя от конкретния контекст и приоритети на проекта.

1.1.1 Големи универсални комерсиални двигатели

Големите комерсиални двигатели са предназначени за широк спектър от производствени сценарии - проекти с високо визуално качество, екипи от различни специалисти и продължителен жизнен цикъл. Тяхна основна характеристика е дълбоката интеграция между инструментите за редактиране на сцени, обработка на ресурси, средствата за диагностика и профилиране, скриптовите слоеве и механизмите за публикуване на различни платформи. Предимството на този тип двигатели е наличието на цялостна и добре организирана работна среда. Като компромис могат да се посочат високата сложност при усвояване, значителният базов обем на проекта и ограничената гъвкавост при необходимост от специфичен контрол върху механизми на ниско ниво. Примери за такива двигатели са Unreal Engine и CryEngine.

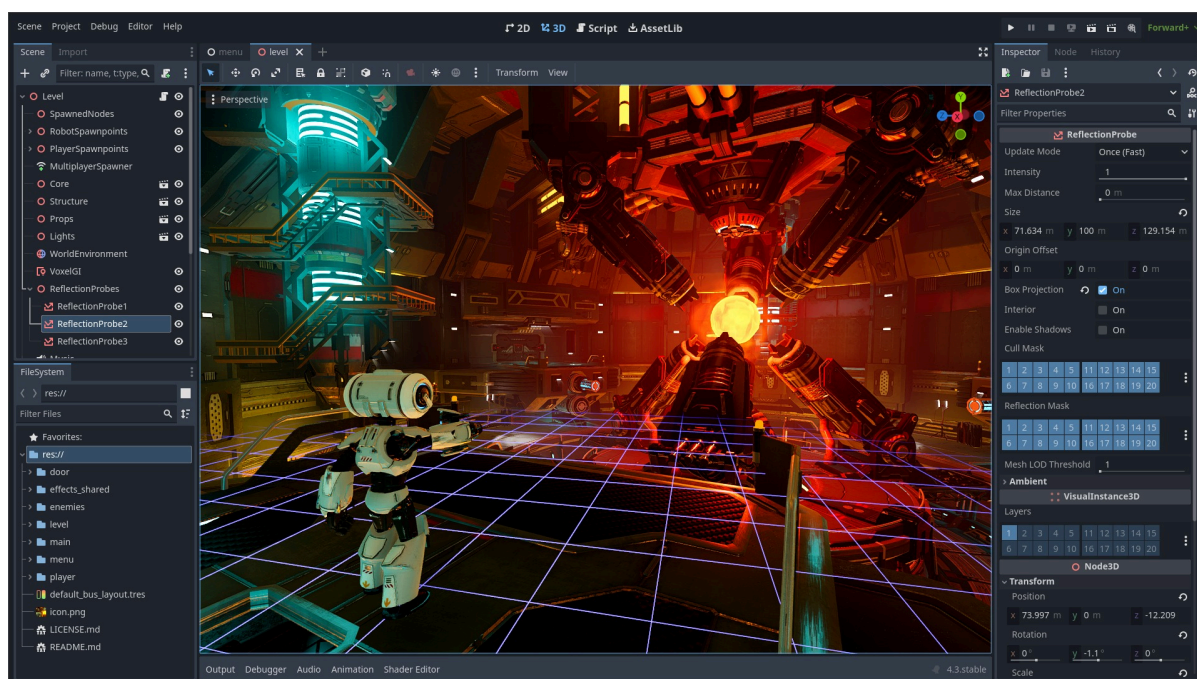


Фиг. 1.1 - Unreal Engine 5

1.1.2 Универсални двигатели за бързо прототипиране и мултиплатформена разработка

Тази категория двигатели е насочена към бърза итерация, лесен достъп до инструменти и широка екосистема от разширения и документация. Подходяща е за малки и средни проекти, при които скоростта на разработка и удобството на работа имат водещо значение. Ограничения могат да възникнат при изисквания за дълбока персонализация на архитектурата или специфични оптимизации по време на изпълнение. В подобни случаи стандартният модел на работа на двигателя може да не се окаже достатъчно гъвкав.

Типични представители на тази група са Unity и Godot.



Фиг. 1.2 - Godot

1.1.3 Вътрешнофирмени двигатели

Вътрешнофирмените двигатели се разработват специално за нуждите на конкретно студио. Те позволяват тясно съответствие с производствения процес, фокусирана функционалност и оптимизация спрямо конкретен тип проекти. Основното предизвикателство при този подход е дългосрочната поддръжка. Екипът носи пълна отговорност за развитието на инструментариума, адаптацията към нови платформи и съхраняването на натрупаното техническо знание. Пример за подобен двигател е Frostbite, разработен от DICE за студиата на Electronic Arts.

1.2 Развойни практики и технологии при разработката на игрови двигатели

1.2.1 Езици за програмиране

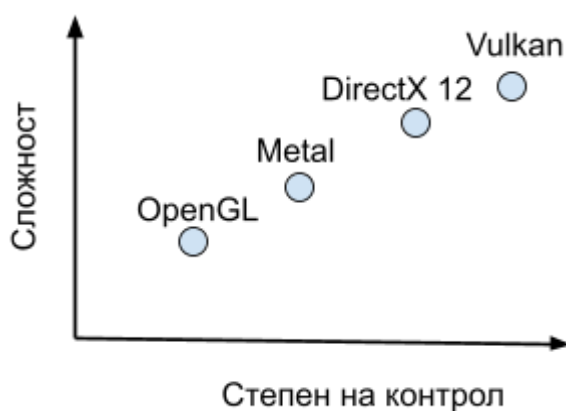
C++ остава водещ език за ядрото на игровите двигатели поради възможността за прецизен контрол върху паметта и ресурсите, висока производителност и добра съвместимост с графични и системни библиотеки.

В практиката често се използва комбиниран подход - частите на двигателя, отговарящи за работата на ниско ниво, се реализират на C или C++, докато логиката на играта и инструментите могат да използват езици от по-високо ниво или скриптові среди.

През последните години езикът Rust също се разглежда като възможна алтернатива за разработка на системен и графичен софтуер. Той предлага силен модел за гарантиране на безопасност на паметта още на ниво компилация и производителност, съпоставима с тази на C++.

1.2.2 Графични програмни интерфейси и архитектурен контрол

Графичните програмни интерфейси като OpenGL, Vulkan, DirectX 12 и Metal се различават по степента на абстракция и експлицитност. По-експлицитните интерфейси (Vulkan, DirectX 12) предоставят по-пряк контрол върху паметта и синхронизацията, но изискват по-сложна архитектура и внимателно управление на ресурсите. Интерфейсите с по-висока степен на абстракция (OpenGL) намаляват първоначалната сложност, но могат да ограничат възможностите за специфична оптимизация. Изборът между тях влияе пряко върху вътрешната структура на двигателя.



Фиг. 1.3 - Спектър между абстракция и контрол при графичните интерфейси

1.2.3 ECS и други архитектурни модели за управление на игрови обекти

Моделът Entity-Component-System (ECS) разделя идентичността на обекта (entity), данните (components) и поведението (systems). Данните се организират по типове, а системите обработват групи от компоненти, което улеснява ориентирания към данните дизайн и подобрява производителността при голям брой обекти. Този подход е особено подходящ за сцени с масови, еднотипни операции.

Алтернатива е класическият обектно-ориентиран модел, при който данните и поведението са обединени в класове и наследствени йерархии. Той е по-интуитивен, но при сложни системи може да доведе до по-трудно мащабиране и оптимизация.

Разпространен е и компонентният модел от тип „обект + компоненти“, при който обектите съдържат динамичен набор от функционални модули. Той предлага гъвкавост, но не винаги осигурява предимствата на строго ориентиран към данните подход.

В практиката често се използват хибридни архитектури, комбиниращи вътрешна ECS организация с по-удобен обектно-ориентиран публичен интерфейс. Изборът на модел влияе пряко върху производителността, разширяемостта и поддръжката на двигателя.

1.2.4 Платформен слой за прозорци и вход

Библиотеки като GLFW и SDL се използват за унифициране на създаването на прозорци, обработката на събития и управлението на входа на различни операционни системи. Това позволява ясно отделяне между платформено-зависимия код и основната логика на двигателя.

1.2.5 Build и dependency практики

Съвременните C++ проекти използват инструменти като CMake за описание на процеса на изграждане и управление на конфигурации. Инструменти за управление на външни зависимости като vcpkg или Conan улесняват интеграцията на библиотеки и осигуряват възпроизводимост на версиите и средата за разработка.

1.2.6 Конвейери за шейдъри и ресурси

Съвременните двигатели използват автоматизирани конвейери за обработка на шейдъри, които включват компилация, проверка за валидност, оптимизация и извличане на

метаданни. Тези метаданни се използват по време на изпълнение за автоматизирано обвързване на ресурси и свойства.

Подобен подход намалява вероятността от несъответствия между кода и шейдърите и повишава надеждността на системата.

1.2.7 Валидация, диагностика и профилиране

Надеждната разработка на двигател изисква системи за регистрация на събития по време на изпълнение, механизми за валидация (особено при експлицитни графични интерфейси), предвидим ред на изпълнение на етапите и инструменти за анализ на времето за кадър.

Тези механизми подпомагат ранното откриване на проблеми и оптимизирането на производителността.

1.3 Изводи от глава 1

Прегледът на съществуващите решения показва, че разработката на игрови двигател изисква съчетаване на няколко ключови аспекта: експлицитно управление на графичните ресурси, стабилна организация на изпълнението, възпроизводим процес на изграждане и добре интегрирани средства за диагностика.

ГЛАВА 2. ИЗИСКВАНИЯ КЪМ ПРОЕКТА И АРГУМЕНТИРАН ИЗБОР НА ТЕХНОЛОГИИТЕ

2.1 Формални изисквания към програмния продукт

- Система за рендериране, използваща слой за абстракция на графичния хардуер с реализация на Vulkan, поддържащ графични и изчислителни операции.
- Система за зареждане и управление на материали и графични ресурси (текстури, семплери).
- Основен игрови цикъл с управление на игрови обекти чрез глобални системи, дефинирани от потребителя.
- Система за създаване и управление на прозорци и обработка на входни събития.
- Съвременна C++ архитектура, базирана на C++ модули и интеграция на външни зависимости чрез `vcpkg`.

2.2 Функционални и нефункционални изисквания

2.2.1 Жизнен цикъл на приложението и цикъл на изпълнение

Приложението трябва да инициализира подсистемите в детерминиран ред, да предостави ясна входна точка за стартиране на потребителски код, да изпълнява стабилен цикъл на кадрите и да извърши подредено спиране с освобождаване на ресурсите.

2.2.2 ECS сцена, обект и поэтапни системи

Средата на изпълнение трябва да поддържа сцени, йерархични игрови обекти, операции за добавяне, премахване и заявка на компоненти, изпълнение на системи на определени етапи периодично или при етапите на стартиране и спиране на двигателя.

2.2.3 Управление на графични ресурси и материали

Двигателят трябва да предоставя публични абстракции за буфери, текстури, семплери, mesh-ове, материали и компоненти за рендериране, като обвързването на дескрипторните множества се управлява от метаданни, извлечени чрез рефлексия на шейдърите.

2.2.4 Изпълнение на графика и изчисления

Слоят за рендериране трябва да поддържа подаването на команди за изрисване на обекти и изпълнение на изчислителни шейдъри, като остава отделен от специфичния за имплементацията код посредством RHI интерфейсите.

2.2.5 Вход и платформена интеграция

Слоят за платформа/вход трябва да предоставя контрол върху жизнения цикъл на прозореца, периодично запитване и callback функции за клавиатурата и мишката, управление на режима на курсора и коректно поведение при затваряне на приложението.

2.2.6 Интеграция на инструменти за шейдъри

Инструменталната верига трябва да компилира шейдъри, да изпълнява SPIR-V валидация/оптимизация, да генерира метаданни и да интегрира резултатите в конвейера за изпълнение и логиката на материалите.

2.2.7 Гъвкавост и разширяемост

Архитектурата трябва да приоритизира ясни граници между модулите и заменяеми под-слоеве, така че бъдещи функционалности да могат да се интегрират без необходимост от цялостно преработване на структурата.

2.3 Аргументиран избор на технологии

2.3.1 C++23 с модули

C++23 е избран поради възможността за прецизен контрол върху паметта, ресурсите и жизнения цикъл на обектите, което е критично при разработката на игрови двигател. Неговата производителност, зрялата екосистема и директната съвместимост с графични програмни интерфейси и системни библиотеки го правят естествен избор за проект от подобен характер.

Съществена част от архитектурния подход е използването на стандартните C++ модули. В сравнение с класическите заглавни файлове, модулите осигуряват ясно дефинирани граници между интерфейс и реализация. Контролира се изрично кои символи са част от публичния интерфейс и кои остават вътрешни за модула. Това позволява

изграждане на стриктни и стабилни програмни интерфейси без риск от неволно „изтичане“ на вътрешни детайли.

За разлика от традиционния модел с включване на заглавни файлове, при който текстовото включване може да доведе до неочаквани транзитивни зависимости, модулите предотвратяват случайното разпространение на декларации и макроси. По този начин се избягват конфликти на имена, нежелани странични ефекти от макроси и трудно проследими компилационни проблеми.

Допълнително предимство е по-ясната структура на зависимости и потенциалното намаляване на времето за компилация при по-големи кодови бази. В контекста на настоящия проект модулният подход подпомага разделянето на двигателя на ясно дефинирани подсистеми с контролирана видимост и минимално пресичане на отговорности.

2.3.2 Ориентация към Windows в началния етап

Първоначалната реализация е насочена към операционната система Windows, тъй като към момента на разработката поддръжката на C++ модули е най-стабилна и развита за компилатора MSVC. Това решение е обусловено от зрелостта на инструменталната среда, а не представлява архитектурно ограничение за бъдеща поддръжка на други операционни системи.

2.3.3 Flecs като основа за ECS

Flecs е избран като реализация на ECS поради своя поетапен модел на изпълнение, вградената поддръжка на йерархии и ясно дефинираните механизми за управление на зависимости между системи. Тези характеристики позволяват естествено организиране и синхронизиране на логиката на кадъра в последователни фази, което съответства на архитектурните цели на проекта.

В сравнение с други подобни библиотеки като EnTT, Flecs предоставя по-изразена концепция за изпълнителни фази и по-богата среда за управление на отношения между обекти. Докато EnTT е минималистична и силно гъвкава библиотека, тя изисква повече допълнителна инфраструктура за реализиране на по-сложни изпълнителни модели и йерархии. В рамките на дипломната работа Flecs позволява по-бързо изграждане на стабилна основа без необходимост от разработване на собствен механизъм за планиране и координация на системите.

2.3.4 GLFW за прозорец и вход

GLFW библиотеката е избрана като лек слой за управление на прозорец и входни устройства в настолна среда. Тя предоставя точно необходимата функционалност за текущия обхват на проекта - създаване на прозорец, обработка на събития и вход от клавиатура и мишка - без допълнителна комплексност.

Алтернативи като SDL предлагат значително по-широк набор от функционалности, включително аудио подсистема, мрежови възможности и други мултимедийни компоненти. В контекста на настоящия проект тези разширения са излишни и биха увеличили сложността на интеграцията без реална необходимост.

2.3.5 CMake и vcpkg за изграждане и зависимости

CMake се използва като система за управление на компилацията с цел изграждане на възпроизводими и ясно структурирани граfi на изграждане, базирани на цели и техните зависимости. Този подход улеснява модулната организация на проекта и интеграцията със среди за разработка.

vcpkg осигурява централизирано управление и версионизиране на външните библиотеки. Чрез интеграция с CMake посредством toolchain файл зависимостите се откриват автоматично и се използват чрез стандартния механизъм `find_package` и `target_link_libraries`. Това позволява коректно и безпроблемно линкване, без ръчно задаване на пътища и компилационни настройки, като гарантира последователна и възпроизводима среда за изграждане.

2.3.6 Vulkan като първа реализация под RHI

Vulkan е избран като първи графичен бекенд поради експлицитния модел на управление на ресурсите. За разлика от OpenGL, при който голяма част от управлението на състоянието е скрито зад вътрешни механизми на драйвера, Vulkan предоставя пряк контрол върху създаването и синхронизацията на ресурси, командни буфери и изпълнителни опашки. Това позволява по-предвидимо поведение и възможности за по-добра оптимизация.

В същото време Vulkan е междуплатформен стандарт, за разлика от DirectX, който е тясно обвързан с екосистемата на Windows. Изборът на Vulkan като първоначална реализация позволява в бъдеще двигателят да бъде използван и на други операционни системи без необходимост от разработване на отделни реализации на RHI слоя за различни графични интерфейси.

2.4 Модулна организация на проекта

2.4.1 Публични модули на двигателя

Публичната част на проекта включва модулите `boza.app`, `boza.common`, `boza.core`, `boza.ecs`, `boza.gfx` и `boza.input`, както и агрегиращия модул `boza`. Тези модули формират официалния програмен интерфейс на двигателя и определят границата между вътрешната реализация и външната употреба.

2.4.2 Вътрешни модули и инструменти

Вътрешната структура включва модулите `boza.rhi`, `boza.platform` и `boza.detail`, както и специфичната реализация `boza.rhi.vulkan`. Към проекта принадлежи и външен инструмент за обработка на шейдъри (`tools/shader_processor`), който генерира междинни представяния и метаданни, използвани по време на изпълнение.

Ясното разграничение между публични и вътрешни модули подпомага архитектурната устойчивост, контрола на зависимостите и поддържането на стабилен интерфейс към потребителите на двигателя.

2.5 Управление на ресурси чрез данни и модел на метаданни за шейдъри

2.5.1 JSON конфигурационни файлове

Настройките на приложението и част от дефинициите на ресурси се съхраняват във външни JSON файлове. Този подход позволява конфигурацията да бъде променяна без необходимост от повторна компилация, като същевременно осигурява възпроизводимост на средата и ясно разграничение между код и данни.

Чрез подобни файлове се дефинират параметри на прозореца, поведение на синхронизацията, целева честота на кадрите и други глобални настройки на изпълнението.

```
{
  "window": {
    "width": 1280,
    "height": 720,
    "title": "Boza Engine Showcase",
    "fullscreen": false
  },
  "gameplay": {
    "target_fps": 0.0,
    "vsync": false,
    "physics_update_rate": 60.0
  }
}
```

Фиг. 2.1 - Примерна конфигурация на настройки на двигателя

2.5.2 Дефиниции на материали и семплери

Материалите се описват чрез отделни JSON файлове, които определят използваните шейдъри, стратегията на зареждане, обвързването на текстури и семплери, както и стойностите на uniform буферите.

Дефинициите на семплери описват параметри като метод на филтриране, режим на адресиране, ниво на детайл и анизотропна филтрация. Този модел позволява ясна декомпозиция между логика и съдържание, като материалите могат да бъдат създавани, модифицирани и разширявани независимо от компилирания код.

```
{
  "vertex_shader": "default",
  "fragment_shader": "default",
  "load_strategy": "game_load",
  "textures": {
    "albedo_map": {
      "texture": "default.png",
      "sampler": "default"
    }
  },
  "params": {
    "material": {
      "albedo_color": [0.8, 0.8, 0.8, 1.0],
      "properties": [0.0, 0.5, 0.5, 1.0]
    }
  }
}
```

Фиг. 2.2 - Примерна структура на материал, дефиниран в JSON файл

```
{
  "filter": "linear",
  "wrap_u": "repeat",
  "wrap_v": "repeat",
  "wrap_w": "repeat",
  "mipmap_mode": "linear",
  "mip_lod_bias": 0.0,
  "min_lod": 0.0,
  "max_lod": 1000.0,
  "max_anisotropy": 1.0
}
```

Фиг. 2.3 - Примерна структура на семплер, дефиниран в JSON файл

2.5.3 Метаданни от рефлексия на шейдърите

Инструментът за обработка на шейдъри генерира метаданни, които описват структурата на ресурсите, използвани от съответния шейдър. Те включват информация за дескрипторните множества и техните номера на свързване, обхвата и членовете на

push-константите, оформлението на структурираните типове и размерите на работните групи при изчислителни шейдъри.

По време на изпълнение обвързването на ресурси се извършва въз основа на тези метаданни, а не чрез твърдо заложи в кода стойности за set и binding. Това намалява риска от несъответствия между шейдър и програмен код, улеснява поддръжката и позволява по-гъвкаво развитие на графичните програми.

2.6 Изводи от глава 2

Изискванията към проекта и практическите ограничения при реализацията мотивират изборния технологичен набор: C++23 с модули, поетапен модел на изпълнение от тип ECS, слой за абстракция на графичния хардуер (RHI) с реализация чрез Vulkan, управление на ресурси чрез конфигурационни и дефиниционни файлове и обвързване на ресурси въз основа на генерирани метаданни.

ГЛАВА 3. РЕАЛИЗАЦИЯ НА ПРОЕКТА

3.1 Обзор на архитектурата на изпълнението

Средата за изпълнение е организирана като многослоен конвейер, в който отделните нива имат ясно разграничени отговорности:

- управление на жизнения цикъл на приложението и координация на стартиране и спиране;
- прогресия на ECS света, при която системите се изпълняват по предварително определени етапи;
- графична подсистема за ресурси на високо ниво, материали и поток на изобразяване;
- API-независими RHI интерфейси;
- Vulkan имплементация на тези интерфейси;
- офлайн инструмент за обработка на шейдъри, генериращ метаданни за използване по време на изпълнение.

Това разделение позволява програмният интерфейс, предназначен за потребителя на двигателя, да остане стабилен, докато детайлите на конкретната графична реализация са съхранени във вътрешни модули.

3.2 Модул **boza.app**

3.2.1 App

App е координаторът на най-високо ниво на жизнения цикъл. Той инициализира logging системата, зарежда настройките на играта, подготвя прозореца и платформената среда, свързва контекста за изобразяване с прозореца, конфигурира параметрите на игровия цикъл, задейства началните етапи на двигателя, извиква `setup` (виж *Фиг. 3.2*) и стартира главния цикъл. Публичният интерфейс предоставя инструмент за превключване към режим „цял екран“, заявка за изход, управление на състоянието на курсора и настройка на целевата честота на кадрите.

```
class App
{
public:
    bool init();
    void run() const;

    static void toggle_fullscreen();
    static void quit();
};
```



```

static inline GlobalProperty<
    &App::get_cursor_state,
    &App::set_cursor_state
> cursor_state;

static inline GlobalProperty<
    &App::get_target_fps,
    &App::set_target_fps
> target_fps;

protected:
    virtual void setup() {}
};

```

Фиг. 3.1 - Публичен интерфейс на App

```

bool App::init()
{
    app::GameSettings::load_from_file(
        AssetPaths::asset("game_settings.json"));

    window.init(...);
    game_loop.init(...);
    game_loop.run_engine_begin_stages();

    setup();
}

```

Фиг. 3.2 - Инициализация на App (опростено)

3.2.2 GameLoop

GameLoop съхранява конфигурацията на основния цикъл и превежда настройките на играта в конкретно времево поведение. Класът отговаря и за извикването на еднократните етапи за стартиране и спиране чрез SystemRegistry.

```

void GameLoop::init(const GameLoopConfig& config)
{
    Time::fixed_delta_time = 1.0f / config.physics_update_rate;

    auto& registry = SystemRegistry::instance();
    registry.initialize_all(Scene::world(), config.physics_update_rate);
}

```

Фиг. 3.3 - инициализация на GameLoop

```

void GameLoop::run() const
{
    const auto& registry = SystemRegistry::instance();
    registry.call_startup_stages();

    const auto& world = Scene::world();
    world.set_target_fps(config.target_fps);
}

```

```

world.reset_clock();

Time::init();

while (!world.should_quit())
{
    Time::update();
    const bool keep_running = world.progress(Time::delta_time());
    Scene::remove_objects_for_destruction();

    if (!keep_running) break;
}

registry.call_destroy_stages();
}

```

Фиг. 3.4 - Основна последователност на итерацията на кадър

3.2.3 GameSettings

GameSettings зарежда секциите “window” и “gameplay” от assets/game_settings.json и използва резервни стойности по подразбиране при неуспешно зареждане или невалиден формат. Това гарантира устойчиво стартиране дори при липсваща или некоректна конфигурация.

3.3 Модул boza.common

3.3.1 Property и GlobalProperty

Property<Owner, Methods...> се конфигурира изцяло по време на компилация. Списъкът "Methods..." е известен по време на компилация и се разделя на getter-и (методи без параметри) и setter-и (методи с параметри). Изборът на конкретен метод не се прави динамично при изпълнение, а се определя предварително от типовете и от начина на извикване по време на компилация.

```

export template<typename Owner, auto... Methods>
class Property final
{
    using filter = method_filter<Methods...>;
    using getter_seq = filter::getters;
    using setter_seq = filter::setters;
}

```

Фиг. 3.5 - Разделяне на getter-и и setter-и

Getter-ът е реализиран чрез deducing this (this Self&&), което позволява един-единствен get() да покрие всички варианти - за неконстантен и константен lvalue и rvalue (&, &&, const &, const &&). Типът Self се извежда автоматично според това как е извикан методът и така вътре в get() може да се провери дали обектът е const и дали е

rvalue. Ако обектът е константен, се допуска само getter, който може да бъде извикан върху const Owner&. Ако обектът е rvalue, при наличие се предпочита getter, който връща rvalue-референция (T&&), за да се позволи преместване. Във всички случаи връщаемият тип се запазва точно чрез decltype(auto), без загуба на референции или константност.

```
template<typename Self>
decltype(auto) get(this Self&& self) requires (getter_count > 0)
{
    // const & or const &&
    if constexpr (is_self_const<Self>)
        return self.call_first_getter_impl(getter_seq{});

    // Rvalue context and we have an rvalue-returning getter - use it
    else if constexpr (is_self_rvalue<Self> && has_rvalue_getter)
        return self.call_move_getter_impl(getter_seq{});

    // Regular lvalue getter
    else return self.call_first_getter_impl(getter_seq{});
}
```

Фиг. 3.6 - Избор на подходящ getter според вида на perfect-forwarded this

Setter-ът се избира чрез система за оценяване, също изцяло по време на компилация. За всеки кандидат-метод се изчислява оценка според това доколко параметрите му съвпадат с подадените аргументи: най-висок резултат получава точно съвпадение (включително по const и референтна категория), следват съвпадения по базов тип, след това допустими преобразувания (например конструиране), а несъвместимите варианти се изключват. Методът с най-висока оценка се избира и извиква директно, като аргументите се препращат с std::forward, за да се запази тяхната lvalue/rvalue категория.

```
template<typename Param, typename Arg>
constexpr int score()
{
    using P = std::remove_cvref_t<Param>;
    using A = std::remove_cvref_t<Arg>;
    // идентичен тип + ref + const
    if constexpr (std::is_same_v<Param, Arg>) return 100;
    // същият базов тип, различни ref/const
    else if constexpr (std::is_same_v<P, A>) return 80;
    // ще мине, но с конструиране/преобразуване
    else if constexpr (std::is_constructible_v<P, Arg>) return 40;
    else return -1; // не става
}

template<auto Method, typename... Args>
constexpr int method_score()
{
    using params = typename method_traits<decltype(Method)>::params;
    return pack_score<params, Args...>(score);
}
```

```

template<typename... Args>
constexpr auto find_best_setter()
{
    return argmax<setter_seq>([]<auto M>() {
        return method_score<M, Args...>();
    });
}

```

Фиг. 3.7 - Силно опростено точкуване на съвместимост за избор на най-подходящ setter

Property няма да заема място в структурата, в която е дефиниран, когато е означен с `[[no_unique_address]]` (или `[[msvc::no_unique_address]]` при MSVC). Това е възможно, защото всяка уникална комбинация от Owner и Methods... образува отделен тип Property, който има собствено статично отместване. Конфигурацията е част от типа, а не от конкретния обект, поради което не е необходимо всяка инстанция да съхранява допълнителни данни. Собственикът се възстановява чрез това статично отместване.

```

export template <typename Owner, auto... Methods>
class Property final
{
    static inline std::ptrdiff_t offset;

    Owner* owner() noexcept
    {
        return reinterpret_cast<Owner*>(
            reinterpret_cast<std::uintptr_t>(this) - offset);
    }

    const Owner* owner() const noexcept
    {
        return reinterpret_cast<const Owner*>(
            reinterpret_cast<std::uintptr_t>(this) - offset);
    }

public:
    explicit Property(const Owner* owner)
    {
        static std::ptrdiff_t temp = offset =
            reinterpret_cast<const char*>(this) -
            reinterpret_cast<const char*>(owner);
    }
};

```

Фиг. 3.8 - Статично отместване и възстановяване на Owner

`GlobalProperty<Methods...>` работи по същия начин, но вместо методи на конкретен обект използва глобални функции или статични методи. Логиката за избор на getter и setter е идентична; разликата е, че няма собственик като инстанция и не се използва отместване.

```

class Transform
{
    [msvc::no_unique_address]
    Property<
        Transform,
        &Transform::get_position,
        &Transform::set_position
    > position{ this };
};

class App
{
    static CursorState get_cursor_state();
    static void set_cursor_state(CursorState);

public:
    static inline GlobalProperty<
        &App::get_cursor_state,
        &App::set_cursor_state
    > cursor_state;
};

Transform tr;
tr.position += glm::vec3{ 1.0f, 2.0f, 3.0f };
glm::vec3 p = tr.position;

App::cursor_state = CursorState::HiddenLocked;
CursorState mode = App::cursor_state;

```

Фиг. 3.9 - Дефиниране и използване на Property и GlobalProperty

3.3.2 Flags

Flags<TEnum> е обвивка за работа с битови флагове, дефинирани чрез изброим тип. Тя гарантира типовата съвместимост и елиминира използването на обикновени цели числа като маски. Предоставя операции за комбиниране и проверка на флагове (|, &, ^, ~) и помощни методи като any, none и value.

```

Flags<TextureUsage> usage = TextureUsage::Sampled |
TextureUsage::TransferDst;
if (usage.any()) {
    // at least one bit is enabled
}

```

Фиг. 3.10 - Пример за използване на Flags

3.3.3 Random

Random е статичен помощен клас за детерминистично и недетерминистично генериране на случайни стойности. Поддържа числови диапазони, вероятностни проверки, разбъркване и избор на елементи, генериране на низове и претеглен избор на стойности от изброими типове.

```
Random::reseed(12345);
auto rarity = Random::pick_enum_weighted<Rarity>({
    { Rarity::Common, 70.0 },
    { Rarity::Rare, 25.0 },
    { Rarity::Epic, 5.0 }
});
```

Фиг. 3.11 - Пример за претеглен избор от изброим тип

3.3.4 Преекспортиране на GLM и контейнери от GTL

Модулът предоставя унифициран достъп до математическите типове на GLM и до хеш-контейнерите на GTL чрез псевдоними. Това позволява еднообразно използване на математически функции и контейнери във всички модули на двигателя. Използват се GTL контейнерите вместо стандартните, поради две причини. Първо, `std::flat_map` и `std::flat_set` биха осигурили по-висока производителност от `std::unordered_map` и `std::unordered_set` заради по-добра локалност на данните в паметта, но MSVC все още не ги имплементира. GTL предоставя еквиваленти на тези контейнери, с което запълва тази липса. Второ, GTL алтернативите на `std::unordered_map` и `std::unordered_set` са по-оптимизирани от стандартните имплементации на MSVC.

```
flat_map<std::string, MaterialDefinition> materials;
flat_set<const Texture*> valid_textures;

mt::node_map<std::string, Sampler> samplers_mt;
```

Фиг. 3.12 - Използване на псевдоними за GTL контейнери

3.4 Модул boza.core

3.4.1 Time

Time поддържа некалирани и скалирани времеви стойности, продължителност на предишния кадър, брояч на кадри и параметри за времево мащабиране и фиксирана стъпка.

```
void Time::update()
{
    const auto current_time = std::chrono::steady_clock::now();
    const std::chrono::duration<float> duration = current_time -
last_frame_time_;
    last_frame_time_ = current_time;

    unscaled_delta_time_ = duration.count();
    unscaled_time_ += unscaled_delta_time_;

    delta_time_ = unscaled_delta_time_ * time_scale;
```



```

    time_ += delta_time_;

    ++frame_count_;
}

```

Фиг. 3.13 - Вътрешна логика за актуализиране на стойностите на Time за новия кадър

3.4.2 Log

Log е статична обвивка над spdlog с удобни методи (trace, debug, info, warn, error, critical) и предаване на форматни низове. В boza.common има дефинирани правила за форматиране на GLM структурите и на Property и GlobalProperty, за автоматично извикване на getter.

```

Log::debug("Loaded {} materials", material_count);
Log::info("Position via property: {}", go.get_component<Transform>().position);
Log::critical("Identity quaternion: {}", glm::identity<glm::quat>());

```

Фиг. 3.14 - Типично използване на Log

3.4.3 assert

В Debug конфигурация assert прихваща форматирано съобщение и stacktrace, записва критично събитие и след това прекратява изпълнението. В Release версия не извършва никакви проверки.

Вместо стандартното макро от <cassert> е реализирана собствена функция, защото макросите не се експортират чрез C++ модули. Това би изисквало всяка единица код (независимо дали е част от двигателя или потребителски проект) изрично да включва <cassert>. Чрез собствена реализация се импортира заедно с модула на двигателя и остава достъпна по унифициран начин във всички части на системата.

3.5 Модул boza.ecs

3.5.1 GameObject

GameObject обвива flecs::entity и предоставя интерфейс за създаване на обекти, изграждане на йерархии, управление на компоненти и отношения, както и разпространение на активното състояние.

```

class GameObject
{
public:
    GameObject() = default;

```

```

static GameObject create(std::string_view obj_name, Scene obj_scene);
static GameObject create(std::string_view obj_name, GameObject obj_parent);
static GameObject create(std::string_view obj_name);

void destroy() const;

static GameObject find(std::string_view obj_name);
GameObject find_child(std::string_view child_name) const;

std::vector<GameObject> children() const;

template <typename C, typename... Args> C& add_component(Args&&... args);
template <typename C, typename... Args> C& ensure_component(Args&&... args);
template <typename C> decltype(auto) get_component(this auto&& self);
template <typename C> auto try_get_component(this auto&& self);
template <typename C> bool has_component() const;
template <typename C> void remove_component() const;

template <typename R, typename... Args>
R& add_relation(const GameObject& target, Args&&... args);
template <typename R, typename... Args>
R& ensure_relation(const GameObject& target, Args&&... args);

template <typename R>
decltype(auto) get_relation_data(this auto&& self, const GameObject& target);
template <typename R>
auto try_get_relation_data(this auto&& self, const GameObject& target);

template <typename R> bool has_relation(const GameObject& target) const;
template <typename R> void remove_relation(const GameObject& target) const;
template <typename R> GameObject get_relation_target() const;

template <typename R> void for_each_with_relation(auto&& callback);
void for_each_child(auto&& callback);

Property<...> active_self{ this };
Property<...> active{ this }; // само getter
Property<...> name{ this };
Property<...> parent{ this };

bool valid() const;
};

```

Фиг. 3.15 - Силно опростен интерфейс на *GameObject*

Йерархията се използва за логическо групиране и за правилна композиция на трансформации. При смяна на родител обектът и неговото поддърво се маркират за обновяване на трансформациите.

Активността се управлява на две нива: `active_self` определя локалното състояние, а `active` отчита и активността на родителите. Промяна в `active_self` автоматично се разпространява към децата. `active` отразява състоянието на `flecs` обекта - когато той не е активен, няма да бъде включван в изпълнението на системи, освен ако изрично не е посочено обратното.

`destroy()` не премахва обекта незабавно, а го маркира за изтриване, което се извършва в края на кадъра от сцената. Това гарантира безопасност при изпълнение на системи.

```
GameObject player = GameObject::create("Player");
player.ensure_component<InputCapture>();

// camera is a child of player
GameObject camera = GameObject::create("Camera", player);
player.active_self = false;

if (auto* t = player.try_get_component<MeshRenderer>()) { /* ... */ }

player.destroy();
```

Фиг. 3.16 - Примерно използване на методите на *GameObject*

Обектите се създават с `Transform` компонент по подразбиране. При използването на фабричен метод без подаден родител или сцена, родителят става автоматично `Scene::main().root()`.

3.5.2 Scene

`Scene` представлява логическа група от обекти, реализирана като коренов `GameObject`, маркиран с таг `tags::Scene`. Тя служи като организационна граница и централизира управлението на съдържание и жизнен цикъл.

```
class Scene final
{
public:
    static Scene create(std::string_view name, bool active = true);
    void destroy() const;

    GameObject& root() { return root_; }
    const GameObject& root() const { return root_; }
    static Scene get(std::string_view name);

    static inline GlobalProperty<...> main;
    static inline GlobalProperty<...> persistent;
    Property<...> name{ this };
    Property<...> active{ this };

    bool valid() const { return root_.valid(); }

private:
    static void remove_objects_for_destruction();
    static flecs::world& world();
    GameObject root_;
    static Scene main_scene_;
};
```

Фиг. 3.17 - Опростена дефиниция на *Scene*

Всяка сцена има коренов обект, под който се изгражда цялата йерархия. Свойствата `name` и `active` делегират към този корен.

`Scene::main` определя текущия контекст за стандартно създаване на обекти, а `Scene::persistent` предоставя сцена, която остава валидна през целия живот на приложението (подходяща за глобални обекти и мениджъри).

Сцената централизира достъпа до ECS света (`Scene::world()`) и отговаря за отложеното изтриване на обекти.

В архитектурен план `GameObject` дефинира единицата на съдържание, а `Scene` - границата на организация. Заедно те изграждат стабилен и ясен модел за управление на йерархия, активност и жизнен цикъл в рамките на ECS-базираната архитектура на двигателя.

3.5.3 SystemStage

`SystemStage` е шаблонна структура за дефиниране на стадий. Тя фиксира фазата (`Phase`) и компонентния договор (`With`, `Opt`, `Without`) по време на компилация и свързва този договор с конкретна функция `execute`. Целта е стадият да бъде описан декларативно чрез тип, а инфраструктурата за регистрация, извличане на компоненти и извикване да се генерира автоматично.

Съществена част от дизайна е автоматичната регистрация по време на статичната инициализация на програмата. За всеки конкретен стадий се създава единствен `SystemStageInfo`, който се добавя в `SystemRegistry` без необходимост от ръчно поддържан централен списък.

```
export template <typename Derived, Phase P, typename... Specs>
struct SystemStage
{
    using CList = ComponentList<Specs...>;
    static constexpr Phase phase = P;
    static SystemStageConfig resolve_config();
    static flecs::system create_system();

    static SystemStageInfo& init_stage_info()
    {
        static SystemStageInfo info
        {
            .phase = phase,
            .create_system = &SystemStage::create_system,
            .resolve_config = &SystemStage::resolve_config
        };

        static const bool registrar = []
        {
            SystemRegistry::instance().register_stage(info);
        };
```

```

        return true;
    }();

    return info;
}

static inline SystemStageInfo& stage_info = init_stage_info();
static SystemStageConfig config() { return SystemStageConfig{}; }
};

```

Фиг. 3.18 - Автоматична регистрация на SystemStage

```

static flecs::system create_system()
{
    auto builder = Scene::world().system();
    const auto cfg = resolve_config();
    const flecs::entity_t engine_kind = to_underlying_phase(Stage::phase);

    if constexpr (Stage::phase == Phase::None) builder.kind(flecs::OnUpdate);
    else if (engine_kind != 0) builder.kind(engine_kind);

    apply_specs_to_builder(builder, static_cast<Stage::CList*>(nullptr));

    if (cfg.multi_threaded) builder.multi_threaded();
    if (cfg.interval > 0.0f) builder.interval(cfg.interval);
    if (cfg.include_disabled) builder.with(flecs::Disabled).optional();

    flecs::system system{};

    if constexpr (is_singleton_stage<Stage>)
    {
        system = builder.run([](flecs::iter& it)
        { while (it.next()) Stage::execute(); });
    }
    else
    {
        system = builder.run([](flecs::iter& it)
        {
            while (it.next())
            {
                for (const auto row : it)
                {
                    invoke_execute<Stage, typename Stage::CList>(
                        it, row, GameObject{ it.entity(row) });
                }
            }
        });
    }

    if constexpr (is_lifecycle_phase(Stage::phase)) system.disable();
    return system;
}

```

Фиг. 3.19 - Конфигурация на flecs::system според дефиницията на SystemStage

3.5.4 SystemStageConfig

SystemStageConfig е минимален слой за runtime настройки, които не е подходящо да бъдат част от типа на стадия. Той управлява интервала на изпълнение, многонишков

режим, включването на неактивни обекти и зависимостите между стадии. По този начин типът определя „какво е стадият“, а конфигурацията определя „как да бъде планиран и подреден“.

```
static SystemStageConfig config()
{
    return {
        .multi_threaded = false,
        .include_disabled = false,
        .interval = 0.0f,
        .run_after = { DummyStage1::stage_info.system },
        .run_before = { DummyStage2::stage_info.system }
    };
}
```

Фиг. 3.20 - Пример за SystemStageConfig

3.5.5 ComponentList

ComponentList<Specs...> реализира шаблонната част на механизма. Той превръща декларативните спецификации в две неща: филтър за избор на обекти и договор за параметрите на execute. Това позволява функцията execute да бъде типово съгласувана със спецификациите без ръчно извличане на компоненти.

Ключовата идея е, че от Specs... се извежда списък от „параметърни спецификации“. В него попадат само With и Opt за непразни компоненти (празни типове не се подават като параметри). Without се използва само като филтър.

```
export template <typename T>
struct With
{
    using type = T;
    static constexpr bool required = true, optional = false, filter = false;
};

export template <typename T>
struct Opt
{
    using type = T;
    static constexpr bool required = false, optional = true, filter = false;
};

export template <typename T>
struct Without
{
    using type = T;
    static constexpr bool required = false, optional = false, filter = true;
};

template <typename S>
concept with_spec = requires { typename S::type; } && S::required;
template <typename S>
```

```

concept opt_spec = requires { typename S::type; } && S::optional;
template <typename S>
concept without_spec = requires { typename S::type; } && S::filter;
template <typename S>
concept param_spec = with_spec<S> || opt_spec<S>;

template <typename S>
using unwrap_t = S::type;

template <typename Spec>
using spec_to_tuple = std::conditional_t<
    param_spec<Spec> && !std::is_empty_v<std::remove_const_t<unwrap_t<Spec>>>,
    std::tuple<Spec>,
    std::tuple<>
>;

export template <typename... Specs> requires (
    (sizeof...(Specs) == 0) ||
    ((param_spec<Specs> || without_spec<Specs>) && ...))
struct ComponentList
{
    static constexpr std::size_t spec_count = sizeof...(Specs);

    using params =
    decltype(std::tuple_cat(std::declval<spec_to_tuple<Specs>>()...));

    static constexpr std::size_t param_count = std::tuple_size_v<params>;
    static constexpr bool is_singleton = spec_count == 0;
};

```

Фиг. 3.21 - Дефиниция на *ComponentList*

3.5.6 SystemRegistry

SystemRegistry е централен мениджър на регистрираните стадии. Той събира всички SystemStageInfo в реда на статичната регистрация, създава системите за всяка фаза и прилага зависимостите. Това гарантира единна точка за инициализация и детерминирано поддръждане.

Фазите се създават централизирано, като за всяка стойност на Phase се поддържа вътрешен идентификатор.

```

void SystemRegistry::initialize_phases(const flecs::world& world)
{
    const auto make_phase = [&](const char* name, flecs::entity_t depends_on)
    {
        auto phase = world.entity(name).add(flecs::Phase);
        if (depends_on != 0) phase.depends_on(depends_on);
        return phase;
    };

    // ...

    phase_entities[phase_index(Phase::EngineUpdate)] =
        make_phase("EngineUpdate", to_underlying_phase(Phase::PostStart));
    phase_entities[phase_index(Phase::PreUpdate)] =

```



```

        make_phase("PreUpdate", to_underlying_phase(Phase::EngineUpdate));

    // ...
}

```

Фиг. 3.22 - Инициализация и подреждане на фазите

След като всички системи са създадени, SystemRegistry прилага зависимостите от конфигурацията. run_after и run_before се свеждат до ограничения за ред на изпълнение между вече съществуващи системи.

```

void SystemRegistry::apply_dependencies(const SystemStageInfo& info) const
{
    for (auto dependency : info.config.run_after)
    {
        if (!dependency.is_valid() || dependency == info.system) continue;
        info.system.depends_on(dependency);
    }

    for (auto dependency : info.config.run_before)
    {
        if (!dependency.is_valid() || dependency == info.system) continue;
        dependency.depends_on(info.system);
    }
}

```

Фиг. 3.23 - Подреждане на стадите според конфигурацията, дефинирана от потребителя

3.5.7 Transform

Transform е компонент за пространствено състояние на обект - позиция, ротация и мащаб. Поддържа едновременно локални стойности (спрямо родителя) и световни стойности (в глобалното пространство), така че йерархичната композиция да е коректна и удобна за използване от системи като рендериране, физика и логика.

```

class Transform final
{
public:
    Property<...> position{ this };

    Property<...> rotation{ this };
    Property<...> scale{ this };
    Property<...> eulers{ this };

    Property<...> local_position{ this };
    Property<...> local_rotation{ this };
    Property<...> local_scale{ this };
    Property<...> local_eulers{ this };
}

```

```

Property<...> forward{ this };
Property<...> right{ this };
Property<...> up{ this };

private:
void evaluate_world_transform();
void mark_dirty() const;
void ensure_world_transform_up_to_date() const;

glm::vec3 local_position_{ 0.0f };
glm::quat local_rotation_{ glm::identity<glm::quat>() };
glm::vec3 local_scale_{ 1.0f };

glm::vec3 world_position_{ 0.0f };
glm::quat world_rotation_{ glm::identity<glm::quat>() };
glm::vec3 world_scale_{ 1.0f };
glm::mat4 world_matrix_{ glm::identity<glm::mat4>() };

bool dirty_{ false };

std::uint64_t world_revision_{ 1 };
std::uint64_t parent_world_revision_{ 0 };
std::uint64_t last_ensure_frame_{ 0 };
};

```

Фиг. 3.24 - Опростен интерфейс на Transform

Световната трансформация не се преизчислява при всяка промяна, а се отлага. При запис на локални/световни свойства компонентът маркира състоянието като невалидно чрез `dirty_`, а реалното изчисление се извършва при първо четене чрез `ensure_world_transform_up_to_date()`. Така множество промени в рамките на един кадър водят до най-много едно преизчисление при нужда, вместо до каскадно обновяване при всяка операция.

Валидността спрямо родителя се проверява без да се обхожда цялото дърво. При успешна оценка `world_revision_` се увеличава, а детето пази `parent_world_revision_` - ревизията на родителя към момента на последното си преизчисление. Несъответствие между тези стойности означава, че родителят е променен и световните данни на детето трябва да се обновят. Допълнително `last_ensure_frame_` кешира кадъра на последната валидация, така че многократни четения в един кадър да не водят до повторна работа.

3.5.8 TransformSystem

TransformSystem е вътрешна система, която отговаря за пакетното разпространение на промени в трансформациите по йерархията в края на фазата EngineUpdate. Тя отделя

ролите ясно: Transform държи и изчислява данните, а системата определя кога и в какъв ред да се изпълни обновяването за множество обекти.

Системата се изпълнява само за обекти с tags::TransformDirty. За да се гарантира правилна композиция, обновяването започва от корените на „мръсните“ поддървета: ако даден обект е маркиран, но и неговият родител е маркиран, обектът се пропуска като корен, защото ще бъде обработен рекурсивно при обработката на родителя. Така оценката върви строго отгоре надолу и всяко дете получава вече валидни родителски световни стойности.

По време на рекурсивното обхождане към децата се предава индикатор дали родителят е променил световната си трансформация. Ако по пътя няма промяна, клонът може да бъде пропуснат, без да се извиква evaluate_world_transform() излишно. След обработка TransformDirty се премахва веднага, за да не бъде актуализиран обектът повторно в същия кадър.

```
struct TransformSystem
{
    struct Update : PreRenderStage<Update, With<const tags::TransformDirty>>
    {
        static void execute(GameObject go)
        {
            const GameObject p = go.parent();
            if (p.has_component<tags::TransformDirty>()) return;
            update_subtree(go, true);
        }

        static void update_subtree(GameObject go, bool parent_changed)
        {
            auto& t = go.get_component<Transform>();
            const bool changed = parent_changed || t.dirty_;
            if (changed) t.evaluate_world_transform();

            t.dirty_ = false;
            go.remove_component<tags::TransformDirty>();

            go.for_each_child([&](GameObject child) {
                update_subtree(child, changed);
            });
        }
    };
};
```

Фиг. 3.25 - Дефиниция и опростена вътрешна логика на TransformSystem

TransformSystem и ensure_world_transform_up_to_date() са два съвместими механизма към една и съща гаранция - валидни световни данни. Системата обновява всички маркирани обекти веднъж на кадър, а при ранно четене (преди системата да се изпълни) Transform може да се валидира „при поискване“ и да кешира резултата за кадъра. Когато

TransformSystem достигне до такъв обект, dirty_ вече е изчистен и обикновено остава само да премахне тага, без повторно преизчисление.

3.6 Модул boza.input

3.6.1 Key, Action, KeyCombo, KeyBinding

Key е силно типизиран идентификатор за клавиши и бутони на мишката. Стойностите са съвместими с GLFW, което позволява директно преобразуване от кодовете на платформата към Key без таблици за превод. Action описва логическия тип на входа (натискане, отпускане, задържане, двоен клик, движение и скрол), т.е. как ще се интерпретира входът от системата.

Над единичните клавиши са изградени две структури: KeyCombo и KeyBinding. KeyCombo представлява „всички тези клавиши са задържани“, а KeyBinding представлява множество алтернативни комбинации, като активиране означава „поне една комбинация е активна“. Изграждат се чрез операторите & и |.

```
enum class Key : std::int32_t
{
    A = GLFW_KEY_A,
    // ...
    Up = GLFW_KEY_UP,
    // ...
    LCtrl = GLFW_KEY_LEFT_CONTROL,
    // ...
    MouseLeft = GLFW_MOUSE_BUTTON_LEFT,
};

enum class Action : std::uint8_t
{
    Press,
    Release,
    Hold,
    DoubleClick,
    MouseMove,
    MouseScroll
};

struct KeyCombo{ std::vector<Key> keys; };
struct KeyBinding{ std::vector<KeyCombo> combos; };

KeyCombo operator&(Key lhs, Key rhs);
KeyBinding operator|(KeyCombo lhs, KeyCombo rhs);
```

Фиг. 3.26 - Опростени дефиниции на Key, Action, KeyCombo, KeyBinding

3.6.2 KeyState

KeyState държи минимално състояние за клавиш в рамките на кадъра: моментно натискане (`pressed_`), задържане (`held_`) и време на последно натискане (`last_press_time`) за засичане на double click. Това позволява да се отдели „събитие в кадъра“ от „състояние между кадри“, без потребителят да има нужда от собствени таймери.

Логиката за double click е централизирана в обработката на GLFW callbacks, където за натискане се сравнява текущото време с `last_press_time`, а резултатът се записва в кадърните списъци (`keys_pressed`, `keys_double_clicked`). Важното е, че callback-ът само попълва `InputSystem::frame`, а реалното извикване на реакции се случва по-късно в `InputSystem::Update`.

```
constexpr double double_click_timeout = 0.3;

class KeyState final
{
public:
    bool is_pressed() const { return pressed_; }
    bool is_held() const { return held_; }

    void set_pressed(const bool pressed) { pressed_ = pressed; }
    void set_held(const bool held) { held_ = held; }

    double last_press_time{ 0.0 };

private:
    bool pressed_{ false };
    bool held_{ false };
};
```

Фиг. 3.27 - Дефиниция на KeyState

3.6.3 CursorState

CursorState описва режимите на курсора като входна абстракция. Реалното прилагане е в `boza.platform::window`, но типът е част от публичния интерфейс, за да може да се контролира през `GlobalProperty<...> App::cursor_state`.

3.6.4 InputCapture

InputCapture е компонент, който съхранява записи на действия, които трябва да се изпълнят при дадени входни условия. Той не чете вход самостоятелно и не прави проверки за състояние на клавиши; това е работа на InputSystem. Компонентът единствено съхранява регистрирани callback функции, групирани по действие и тип свързване.

За Press, Release, Hold и DoubleClick са налични два паралелни механизма. Първият е директно свързване към единичен Key чрез `flat_map<Key, callback>`, което е най-евтиният път за класическо управление тип „Space → Jump“. Вторият е свързване към KeyBinding чрез списък от BindingEvent, което позволява комбинации и алтернативи (напр. Ctrl+S или F5). При вход с мишката компонентът пази списъци от callback функции за преместване на курсора и скролиране, тъй като тези събития са непрекъснати и могат да имат повече от един слушател.

```
struct BindingEvent
{
    KeyBinding binding;
    std::move_only_function<void()> callback;
};

class InputCapture final
{
public:
    template <Action A> void on(Key key, auto&& callback);
    template <Action A> void on(KeyCombo combo, auto&& callback);
    template <Action A> void on(KeyBinding binding, auto&& callback);

    template <Action A> requires (A == Action::MouseMove)
    void on(auto&& callback);

    template <Action A> requires (A == Action::MouseScroll)
    void on(auto&& callback);

    void clear();

private:
    flat_map<Key, std::move_only_function<void()>> press_events_;
    flat_map<Key, std::move_only_function<void()>> release_events_;
    flat_map<Key, std::move_only_function<void()>> hold_events_;
    flat_map<Key, std::move_only_function<void()>> double_click_events_;

    std::vector<BindingEvent> press_bindings_;
    std::vector<BindingEvent> release_bindings_;
    std::vector<BindingEvent> hold_bindings_;
    std::vector<BindingEvent> double_click_bindings_;

    std::vector<std::move_only_function<void(glm::vec2)>> mouse_move_callbacks_;
    std::vector<std::move_only_function<void(glm::vec2)>> mouse_scroll_callbacks_;
};
```

Фиг. 3.28 - Опростена структура на InputCapture

3.6.5 InputSystem

InputSystem е централната система за вход. Тя изпълнява три задачи, които е важно да са на едно място: регистрация и задаване на GLFW callback функции при засечен вход, натрупване на входни данни за текущия кадър и разпределение на тези данни към всички InputCapture компоненти в правилен ред и фаза.

InputSystem поддържа структура InputFrameData, която съхранява входното състояние за текущия кадър. Тя съдържа асоциативна таблица от KeyState за всички наблюдавани клавиши, както и списъци със събития, възникнали в рамките на текущия цикъл на обновяване - натиснати, отпуснати и двойно натиснати клавиши. За мишката се съхраняват относителни изменения на позицията (mouse_delta) и скрола (scroll_delta), акумулирани от платформените callback функции.

GLFW callback функциите не извикват потребителски код директно. Те единствено актуализират InputSystem::frame. Реалната обработка се извършва в стадия Update, което гарантира детерминираност и ясно дефиниран момент на реакция в рамките на игровия цикъл.

Destroy е симетричният стадий на Begin. Той откачва callback функциите от GLFW и изчиства кадърните структури, за да няма висящи указатели или остатъчно състояние след изключване.

```
export class Input final
{
public:
    Input() = delete;

    static bool is_pressed(Key key);
    static bool is_held(Key key);
};

export struct InputSystem
{
    struct InputFrameData final
    {
        flat_map<Key, KeyState> key_states{};
        std::vector<Key> keys_pressed{};
        std::vector<Key> keys_released{};
        std::vector<Key> keys_double_clicked{};
        glm::vec2 mouse_delta{ 0.0f, 0.0f };
        glm::vec2 scroll_delta{ 0.0f, 0.0f };
    };

    static inline InputFrameData frame{};

    static inline glm::vec2 last_cursor_pos{ 0.0f, 0.0f };
    static inline bool first_cursor_move{ true };

    struct Begin final : EngineBeginStage<Begin> { static void execute(); };
    struct Input final : EngineUpdateStage<Input> { static void execute(); };
    struct Update final : EngineUpdateStage<Update, With<InputCapture>>
    {
        static SystemStageConfig config()
        { return { .run_after = { Input::stage_info.system } }; }

        static void execute(InputCapture& capture);
    };

    struct Destroy : EngineDestroyStage<Destroy> { static void execute(); };
};
```

```

private:
    static void process_press_events(
        InputCapture& ic, Key key, const flat_map<Key, KeyState>& states);
    static void process_release_events(
        InputCapture& ic, Key key, const flat_map<Key, KeyState>& states);
    static void process_double_click_events(
        InputCapture& ic, Key key, const flat_map<Key, KeyState>& states);
    static void process_hold_events(
        InputCapture& ic, const flat_map<Key, KeyState>& states);

    static void process_mouse_move(InputCapture& capture, glm::vec2 delta);
    static void process_mouse_scroll(InputCapture& capture, glm::vec2 offset);
};

```

Фиг. 3.29 - Дефиниция на *Input* и *InputSystem*

Вътрешните функции `process_*` са умишлено тясно специализирани и разделени по тип вход. Те правят две неща: първо проверяват директните регистрации по `Key`, а после, ако има, оценяват `KeyBinding` списъка и извикват всички `binding callback` функции, за които някоя комбинация е активна спрямо текущите `key_states`. Така обработката на единичните клавиши се изпълнява в $O(1)$ време, а комбинациите са с $O(n)$ комплексност, но се изпълняват само когато действително има събитие или когато се обработва `Hold`.

```

void InputSystem::Input::execute()
{
    for (auto& key_state : frame.key_states | std::views::values)
        key_state.set_pressed(false);

    clear_frame_data();

    platform::Window* window = rhi::RenderContext::window();
    window->poll_events();
    if (window->should_close()) App::quit();
}

void InputSystem::Update::execute(InputCapture& capture)
{
    for (const Key key : frame.keys_pressed)
        process_press_events(capture, key, frame.key_states);
    for (const Key key : frame.keys_released)
        process_release_events(capture, key, frame.key_states);
    for (const Key key : frame.keys_double_clicked)
        process_double_click_events(capture, key, frame.key_states);

    process_hold_events(capture, frame.key_states);

    if (glm::length2(frame.mouse_delta) > 1e-8f)
        process_mouse_move(capture, frame.mouse_delta);
    if (glm::length2(frame.scroll_delta) > 1e-8f)
        process_mouse_scroll(capture, frame.scroll_delta);
}

```

Фиг. 3.30 - Вътрешна логика на *InputSystem::Input* и *InputSystem::Update*

3.7 Модул boza.platform

3.7.1 Window

Window управлява инициализацията на GLFW, създаването на прозореца и превключването във fullscreen/windowed режим. Състояния като текущ размер, последни размери и позиция се пазят, за да може fullscreen режимът да се включва и изключва без загуба на контекст.

```
class Window final
{
public:
    static bool init();
    void init(...);
    bool create(rhi::GraphicsApi api);

    void destroy();
    static void terminate();

    void toggle_fullscreen();

    std::uint32_t width() const { return width_; }
    std::uint32_t height() const { return height_; }

    bool should_close() const;
    static void poll_events();
    void set_window_resize_callback();

    void show() const;
    void hide() const;

    CursorState cursor_state() const;
    void set_cursor_state(CursorState state);

#ifdef BOZA_VULKAN_ENABLED
    VkResult create_vulkan_surface(VkInstance instance, VkSurfaceKHR&);
#endif
private:
    uint32_t width_{};
    uint32_t height_{};

    std::string title_{};
    bool fullscreen_{};

    uint32_t last_width_{};
    uint32_t last_height_{};

    uint32_t last_pos_x_{};
    uint32_t last_pos_y_{};

    CursorState current_cursor_state_{ CursorState::Normal };

    std::atomic_bool resized_{ false };
    GLFWwindow* window_{ nullptr };
};
```

Фиг. 3.31 - Опростена дефиниция на Window

Обработката на промяна на размера е реализирана чрез framebuffer callback. Вместо директно пресъздаване на ресурси в callback-а, прозорецът само актуализира размерите и маркира атомичен флаг. Методът `has_resized()` връща промяната еднократно и нулира флага, за да може графичният слой да реагира синхронизирано в подходящ момент от кадъра.

```
void Window::set_window_resize_callback()
{
    glfwSetWindowUserPointer(window_, this);
    glfwSetFramebufferSizeCallback(window_,
    [] (GLFWwindow* window, int width, const int height)
    {
        const Window* self = glfwGetWindowUserPointer(window);
        self->width_ = static_cast<uint32_t>(width);
        self->height_ = static_cast<uint32_t>(height);
        self->resized_.store(true);
    });
}

bool Window::has_resized()
{
    if (resized_.load()) { resized_.store(false); return true; }
    return false;
}
```

Фиг. 3.32 - Сигнализация за *resize* чрез флаг

3.8 Модул `boza.gfx`

3.8.1 Архитектурна роля и граници на модула

`boza.gfx` е графичният модул на двигателя, работещ на високо ниво. Той моделира рендерирането чрез обекти на двигателя (Mesh-ове, материали, текстури, камери и рендерируеми компоненти), държи авторските данни, изпълнителния кеш и оркестрацията на кадъра, а реалното изпълнение делегира на бекенд абстракцията. Бекенд-специфичните обекти се третират като детайлна реализация зад стабилни интерфейси.

3.8.2 Общи структури за модула

`boza.gfx` дефинира малък набор от споделени структури, използвани във всички графични ресурси. `ResourceAccessMode` описва дали един ресурс е единичен (Static) или индексан по кадри (Dynamic), а `ShaderDataType` предоставя каноничен речник от типове за време на изпълнение, използван при рефлексия и валидация на свойства. Те се използват в `Material`, `ComputeDispatcher`, `Buffer` и `Texture`, така че всички пътища за обновяване да следват една и съща конвенция за типове и времена на живот.

```
enum class ResourceAccessMode : std::uint8_t
{
    Static,
    Dynamic
};

enum class ShaderDataType : std::uint8_t
{
    Unknown,
    Bool,
    Int, Uint,
    Float, Double,
    Vec2, Vec3, Vec4,
    Mat2, Mat3, Mat4,
    Sampler2D, SamplerCube
    // ...
};
```

Фиг. 3.33 - Декларация на *ResourceAccessMode* и *ShaderDataType*

3.8.3 Buffer

Buffer е обвивка за RHI буферни ресурси на ниско ниво, използвани от vertex, index, uniform, storage и staging операции. Той притежава един или повече RHI буфери в зависимост от *ResourceAccessMode*, осигурява операции за качване и обратно четене на данни с проверка на обхвата и поддържа mapping за достъп от процесора. Класът изглежда и се управлява по един и същ начин независимо дали подлежащото хранилище е единично или разпределено по кадри.

```
enum class BufferUsage : std::uint8_t
{
    Vertex, Index,
    Uniform, Storage, Staging
};

class Buffer final
{
public:
    Buffer(
        std::size_t buffer_size, BufferUsage usage,
        ResourceAccessMode buffer_access_mode = ResourceAccessMode::Static);

    void upload(const void* data, std::size_t data_size, std::size_t offset)
    const;
    void read_back(void* data, std::size_t data_size, std::size_t offset)
    const;

    template <typename T> void upload(const T& data);
    template <typename T> void upload(std::span<T> data);

    std::vector<std::uint8_t> read_back() const;

    Property<...> size{ this };
    Property<...> access_mode{ this };

    void* map() const;
```

```

void unmap() const;
void* rhi_handle() const;

private:
    std::vector<void*> rhi_buffers_;
    std::size_t size_;
    ResourceAccessMode access_mode_;
};

```

Фиг. 3.34 - Опростен интерфейс на Buffer

Изборът на вътрешен буфер се извършва спрямо текущия кадър: при динамичен буфер се използва индексът на кадъра, а при статичен - винаги индекс 0. По този начин се осигурява единен програмен интерфейс и се елиминира необходимостта външният код да управлява дескрипторните множества за всеки кадър поотделно.

```

void* Buffer::rhi_handle() const
{
    const std::uint32_t frame_index =
        rhi::RenderContext::swapchain()->current_frame();
    const std::uint32_t buffer_index =
        access_mode_ == ResourceAccessMode::Dynamic ? frame_index : 0;

    return rhi_buffers_[buffer_index];
}

```

Фиг. 3.35 - Избор на rhi::Buffer* съобразно кадъра в Buffer

Класът валидира операциите върху диапазони от памет преди изпълнение на запис или четене. Това осигурява навременна диагностика при опити за достъп извън границите, което е особено важно, когато отместванията се изчисляват въз основа на рефлексивна информация за подредбата.

3.8.4 Texture и TextureLoader

Texture представя графичен ресурс от тип изображение със сходна стратегия на жизнения цикъл като Buffer. Поддържа явно създаване чрез TextureSettings, зареждане по име, опционален достъп, индексирани по кадри, качване и обратно четене на данни, както и явни преходи на разположението в паметта при необходимост от страна на рендериращ или изчислителен процес. Интерфейсът на високо ниво остава ресурсно ориентиран и не излага примитиви за синхронизация на бекенда.

```

struct TextureSettings final
{
    TextureType          type{ TextureType::Texture2D };
    TextureFormat         format{ TextureFormat::RGBA8 };
    ResourceAccessMode    access_mode{ ResourceAccessMode::Static };
};

```

```

    std::uint32_t      width{ 1u };
    std::uint32_t      height{ 1u };
    std::uint32_t      depth{ 1u };
    Flags<TextureUsage> usage_flags{};
};

class Texture final
{
public:
    static Texture& copy(
        std::string_view src_name,
        std::string_view dst_name,
        ResourceAccessMode access_mode);

    static Texture& create(std::string_view name, TextureSettings& settings);
    static Texture& get_or_load(std::string_view name);
    static Texture& get_or_load_cubemap(std::string_view name);

    void upload(const void* data, std::size_t size) const;
    void upload_layer(void* data, std::size_t size, std::uint32_t layer);
    std::vector<std::uint8_t> read_back() const;
    void transition_layout(TextureLayout old_l, TextureLayout new_l);

    Property<...> width{ this };
    Property<...> height{ this };
    Property<...> depth{ this };
    Property<...> type{ this };
    Property<...> format{ this };

    void* rhi_handle() const;
};

```

Фиг. 3.36 - Опростена дефиниция на Texture

TextureLoader централизира управлението на жизнения цикъл на текстурите и прилага политика за резервен ресурс при грешка. При инициализация създава постоянна текстура за грешки, която се връща при неуспешно зареждане. Така се гарантира валидност на дескрипторите дори при липсваща текстура и се предотвратяват каскадни сризове в по-късните етапи на рендериране.

Специален случай е зареждането на кубична текстура. Поддържа се единично изображение с кръстовидна подредба, от което се извличат шестте лица по предварително дефинирани координати. Това опростява подготовката на изходното изображение, като при изпълнение се получава стандартна кубична текстура.

3.8.5 Sampler и SamplerLoader

Sampler описва параметрите за филтрация, адресиране (обвиване), нива на детайлност и анизотропия като именуван обект по време на изпълнение. SamplerLoader допълва тази функционалност със създаване по данни от файлови дефиниции и със sampler по подразбиране, наличен след инициализация. Така материалите се позовават на sampler-и

чрез стабилни имена, а политиката на семплиране се отделя от съхранението на текстурите.

Съвместно TextureLoader и SamplerLoader осигуряват надежден преход от файлови ресурси към изпълнение в рамките на boza.gfx, като липсващото съдържание се обработва контролирано, а стойностите по подразбиране, дефинирани чрез данни, могат да бъдат променяни за конкретен материал при необходимост.

3.8.6 Mesh

Mesh съхранява неизменяеми процесорни данни за геометрията и предварително изчислени граници за culling. Регистрира се по име и се поддържа в множество за валидност на указатели, което позволява на изпълнителните кешове да проверяват дали даден указател към Mesh остава валиден.

При създаване се изчислява единна ограждаща сфера на базата на върховете, която впоследствие RenderingSystem трансформира в световни координати за тестове спрямо зрителния обем.

```
void Mesh::recalculate_bounds()
{
    glm::vec3 min_position = vertices_.front().position;
    glm::vec3 max_position = vertices_.front().position;

    for (const Vertex& vertex : vertices_)
    {
        min_position = min(min_position, vertex.position);
        max_position = max(max_position, vertex.position);
    }

    const glm::vec3 center = (min_position + max_position) * 0.5f;

    float radius_squared = 0.0f;
    for (const Vertex& vertex : vertices_)
    {
        radius_squared = std::max(
            radius_squared, glm::length2(vertex.position - center));
    }

    bounds_.center = center;
    bounds_.radius = std::sqrt(radius_squared);
}
```

Фиг. 3.37 - Изчисляване на ограждаща сфера в Mesh

3.8.7 Material и MaterialLoader

3.8.7.1 Material

Material представлява свързващия слой между метаданните на шейдъра и данните за конкретно draw повикване. Той съхранява референции към графични конвейери, описания и дескрипторни множества, междинна памет за push константи, staging буфери за всеки uniform блок (UBO) и лек механизъм за проследяване на променени записи.

MaterialSettings описва двойката шейдъри и параметрите на състоянието на рендериране, които определят идентичността на конвейера. Структурата е хешируема, което позволява повторно използване на конвейерни обекти чрез кеширане.

```
struct MaterialSettings
{
    std::string vertex_shader;
    std::string fragment_shader;
    CompareOp    depth_compare_op{ CompareOp::Less };
    bool         depth_test_enable{ true };
    bool         depth_write_enable{ true };
    CullMode     cull_mode{ CullMode::Back };
    FrontFace    front_face{ FrontFace::CounterClockwise };
};

class Material
{
public:
    static Material& create(std::string_view name, MaterialSettings& settings);
    static Material& get(std::string_view name);
    static Material* try_get(std::string_view name);

    PropertyBinder operator[] (std::string_view name);

    void bind() const;

    void push_constants(const std::string& name, const auto& value);
    void update_property(const std::string& name, const auto& value);

    void update_texture(const std::string& name, const Texture& texture);
    void update_texture(std::string& name, Texture& texture, Sampler& s);
    void update_buffer(const std::string& name, const Buffer& buffer);

    std::optional<BindingInfo> lookup_binding(const std::string& name) const;

private:
    void* pipeline_{ nullptr };
    void* pipeline_layout_{ nullptr };
    std::vector<void*> descriptor_set_layouts_;
    std::vector<void*> descriptor_sets_;
    std::vector<std::vector<void*>> descriptor_sets_per_frame_;
    std::vector<std::uint8_t> push_constant_staging_;

    flat_map<std::uint32_t, bool> dirty_sets_;
    flat_map<std::uint32_t, std::vector<std::uint8_t>> uniform_buffer_staging_;
    flat_map<std::uint32_t, void*> uniform_buffers_;
    flat_map<std::string, Sampler*> default_samplers_;
```

```

void* descriptor_pool{ nullptr };
bool  cpu_cull_enabled{ true };
};

```

Фиг. 3.38 - Опростена дефиниция на MaterialSettings и Material

Изпълнението на Material използва бекенд абстракции по два координирани етапа: конфигуриране на конвейера при създаване чрез MaterialLoader и обвързване в Material::bind по време на кадъра.

При инициализация MaterialLoader проверява в ResourceCache за съществуваща еквивалентна конфигурация. При липса на кеширан конвейер, boza.gfx изгражда оформлението на дескрипторните множества и оформлението на конвейера чрез rhi::PipelineBuilder, създава графичен конвейер според избраните шейдъри и параметрите на растеризация и дълбочина, след което съхранява получените дескриптори в кеша за повторна употреба от материали със същите настройки.

```

const auto* cached = resource_cache->get_cached_pipeline(
    settings.vertex_shader, settings.fragment_shader, settings_hash);

if (!cached)
{
    rhi::PipelineBuilder builder(api, device, { vert_shader, frag_shader });
    builder.build_descriptor_set_layouts();
    pipeline_layout = builder.build_pipeline_layout();
    pipeline = builder.build_graphics_pipeline(
        swapchain, swapchain->depth_format(), raster_state, depth_state);

    resource_cache->cache_pipeline(
        settings.vertex_shader, settings.fragment_shader, settings_hash, {
            .pipeline = pipeline,
            .layout = pipeline_layout,
            .descriptor_set_layouts = builder.get_descriptor_set_layouts()
        });
}

```

Фиг. 3.39 - Основна логика за създаване на pipeline и кеширане в MaterialLoader

След получаване на дескрипторите за конвейера се разпределят множества от дескриптори за всеки едновременно обработван кадър. Така един обект Material може да съхранява обвързвания към статични и динамични ресурси, без да излага управлението на индексите по кадри към извикващия код. При изпълнение Material::bind избира съответния сегмент от дескрипторното множество за текущия кадър и обвързва конвейера и множествата към активния команден буфер.

```

void Material::bind() const

```



```

{
    auto* cmd = rhi::RenderContext::current_command_buffer();
    cmd->bind_graphics_pipeline(static_cast<rhi::GraphicsPipeline*>(pipeline_));

    const std::vector<void*>* active_descriptor_sets = &descriptor_sets_;

    if (!descriptor_sets_per_frame_.empty())
    {
        std::uint32_t frame_index = swapchain->current_frame();
        frame_index %= descriptor_sets_per_frame_.size();
        active_descriptor_sets = &descriptor_sets_per_frame_[frame_index];
    }

    std::vector<rhi::DescriptorSet*> sets;
    sets.reserve(active_descriptor_sets->size());
    for (auto* set : *active_descriptor_sets)
        sets.push_back(static_cast<rhi::DescriptorSet*>(set));
    cmd->bind_descriptor_sets(pipeline_layout_, sets, 0);
}

```

Фиг. 3.40 - Избор на дескрипторно множество по кадър в `Material::bind` (опростено)

Свойствата на материала се актуализират чрез символни имена, които се разрешават посредством рефлексията на шейдърите. Процесът първо извлича съответния `BindingInfo`, след което разклонява според категорията на дескриптора: `push` константите се записват в буфера `push_constant_staging_`, членовете на `uniform` блокове се отразяват в `CPU staging` буфера на съответния `UBO`, а обвързванията на текстури и буфери се преобразуват в операции за обновяване на дескрипторни множества.

В конфигурация за отстраняване на грешки `boza.gfx` валидира типовете и размерите спрямо данните от рефлексията преди извършване на записа.

```

void Material::update_property(
    std::string& name, const void* data, std::size_t size, ShaderDataType type)
{
    const auto binding_info = reflection_->lookup(name);
    if (!binding_info.has_value()) return;

    const auto& info = binding_info.value();

    #ifdef BOZA_DEBUG
    if (!rhi::validate_property_type(type, size, info.data_type, info.size))
    { ... }
    #endif

    if (info.is_push_constant)
    {
        std::memcpy(push_constant_staging_.data() + info.offset, data, size);
        return;
    }

    if (info.descriptor_type != rhi::DescriptorType::UniformBuffer) return;
}

```

```

const std::uint32_t binding_key = (info.set << 16) | info.binding;

if (!uniform_buffers_.contains(binding_key))
{
    auto parent_info = reflection_->lookup(name.substr(0, name.find('.')));
    auto buffer_size = parent_info.has_value() ? parent_info->size : 256;

    auto* buffer = create_buffer(...);

    uniform_buffers_[binding_key] = buffer;
    uniform_buffer_staging_[binding_key].resize(buffer_size, 0);

    rhi::DescriptorWrite write{
        .binding = info.binding,
        .type = rhi::DescriptorType::UniformBuffer,
        .info = rhi::UniformBuffer{ buffer, 0, buffer_size }
    };

    if (!descriptor_sets_per_frame_.empty())
    {
        for (const auto& frame_sets : descriptor_sets_per_frame_)
        {
            if (info.set < frame_sets.size())
                frame_sets[info.set]->update({ &write, 1 });
        }
        else if (info.set < descriptor_sets_.size())
            descriptor_sets_[info.set]->update({ &write, 1 });
    }

    auto& staging = uniform_buffer_staging_[binding_key];
    std::memcpy(staging.data() + info.offset, data, size);

    rhi::Buffer* buffer = uniform_buffers_[binding_key];
    buffer->upload(staging.data(), staging.size(), 0);
}

```

Фиг. 3.41 - Обновяване на свойство на шейдър в Material (push константи и uniform буфери)

PropertyBinder представлява лек помощен слой над механизма за обновяване на материали, като делегира operator= към Material::update_property, update_texture или update_buffer според типа на присвоената стойност. Архитектурният принцип е данните на материала да се управляват чрез рефлексия и именувани свойства, а не чрез твърдо дефинирани структури в кода на приложението.

3.8.7.2 MaterialLoader

MaterialLoader определя политиката за създаване на материали, обработката на дефиниции, регистъра за изпълнение и глобалните ресурсни обвързвания. Дефинициите се зареждат от JSON файлове и поддържат две стратегии: "game_load" - създаване при графична инициализация, и "on_demand" - инстанциране при първа заявка.

Loader-ът създава и управлява три uniform буфера на ниво двигател (CameraUBO, LightUBO, TimeUBO), които се обвързват автоматично към съвместими материали. Така се централизира подаването на състоянието за всеки кадър и се избягва повтарящ се код в рендериращите етапи.

Обработват се имената на шейдърите, настройките за рендериране, присвояванията на текстури и семплери и началните стойности на UBO свойства. Стойностите се преобразуват към очакваните типове по време на изпълнение и се подават към функциите за обновяване на Material. По този начин дефинирането е изцяло базирано на данни, при запазени проверки за съвместимост на типовете по време на изпълнение.

3.8.8 Компоненти за рендериране

3.8.8.1 Camera

Camera дефинира проекционното състояние и се интегрира с ECS чрез маркера `tags::PrimaryCamera`. `Property<...> is_primary` автоматично добавя или премахва този маркер, така че изборът на активна камера се извършва чрез заявки към ECS, без използване на глобални указатели. Компонентът поддържа перспективна и ортогографска проекция и изчислява проекционната матрица въз основа на текущите параметри и аспектното съотношение на прозореца.

3.8.8.2 MeshRenderer

MeshRenderer осигурява връзката между ECS същностите и кешовете за рендериране. Съхранява имената на mesh и material, указатели с отложено разрешаване, флагове за промяна при актуализация и служебни полета за отчетност, използвани от RenderingSystem. При промяна на mesh или material компонентът добавя съответния ECS маркер, което позволява на рендериращия конвейер да обработва структурните изменения в ясно дефинирани и детерминирани стадии.

```
export class MeshRenderer final
{
public:
    Property<...> mesh{ this };
    Property<... > material{ this };

    Property<...> mesh_name{ this };
    Property<...> material_name{ this };

private:
    std::string mesh_name_{};
    std::string material_name_{};

    mutable Mesh* mesh_{ nullptr };
};
```

```

mutable Material* material_{ nullptr };

mutable Mesh* cached_mesh_{ nullptr };
mutable Material* cached_material_{ nullptr };

mutable bool dirty_mesh_{ true };
mutable bool dirty_material_{ true };
mutable bool in_render_cache_{ false };
mutable bool in_unresolved_cache_{ false };
};

```

Фиг. 3.42 - Дефиниция на MeshRenderer, управляващ инвалидиране

3.8.9 ComputeDispatcher

ComputeDispatcher реализира изчислителен работен поток на високо ниво, следващ принципите на системата за материали: актуализацията на свойства се извършва чрез рефлексия, а свързването на ресурси е символично. Размерите на диспечирането могат да се задават като брой елементи или като явен брой работни групи. Създаването на compute pipeline се кешира, дескрипторните множества се разпределят еднократно за всяка инстанция, а push константите се подготвят преди изпълнение.

```

class ComputeDispatcher
{
public:
    ComputeDispatcher(const std::string& shader_name, bool& failed);

    ComputeDispatcher& set(const std::string& name, const T& value);
    ComputeDispatcher& set(const std::string& name, const Texture& texture);
    ComputeDispatcher& set(const std::string& name, const Buffer& buffer);

    ComputeDispatcher& dispatch(
        std::uint32_t width, std::uint32_t height = 1, std::uint32_t depth = 1);

    ComputeDispatcher& dispatch_groups(
        std::uint32_t x, std::uint32_t y, std::uint32_t z);

    ComputeDispatcher& wait();
};

```

Фиг. 3.43 - Опростен интерфейс на ComputeDispatcher

3.8.9.1 Настройка на Pipeline и дескрипторни множества

При конструиране ComputeDispatcher изпълнява последователност от стъпки, сходни със създаването на материал, но специализирани за единичен compute шейдър. Шейдърният модул се получава чрез ResourceCache, след което се прави опит за използване на кеширан набор за compute pipeline; нов набор се създава единствено при липса в кеша. Наборът включва дескриптор към compute pipeline, pipeline layout и описанията на дескрипторните множества.

```

const auto shader_shared = resource_cache->get_or_create_shader(
    shader_desc,
    [=] (rhi::ShaderModuleDesc& desc){ return create_shader_module(api, desc);
});

const auto* cached = resource_cache->get_cached_compute_pipeline(shader_name);
if (!cached)
{
    rhi::PipelineBuilder builder{ api, device, { shader } };
    builder.build_descriptor_set_layouts()

    pipeline_layout = builder.build_pipeline_layout();
    pipeline = builder.build_compute_pipeline();

    resource_cache->cache_compute_pipeline(
        shader_name, { .pipeline = pipeline, .layout = pipeline_layout,
            .descriptor_set_layouts = builder.get_descriptor_set_layouts() });
}

for (auto* layout : descriptor_set_layouts)
{
    auto* desc_set = descriptor_pool->allocate_descriptor_set(layout);
    descriptor_sets.push_back(desc_set);
}

```

Фиг. 3.44 - Подготовка на pipeline и дескриптори в конструктора на ComputeDispatcher

Свързването на ресурси чрез set(name, Texture) и set(name, Buffer) използва метаданните от рефлексията, за да определи параметрите на обвързване, генерира съответния запис в дескрипторното множество, актуализира го и го маркира като променено. Така публичният интерфейс остава символно ориентиран, без да се губи точността на позициите на дескрипторите по време на изпълнение.

```

ComputeDispatcher& ComputeDispatcher::set(std::string& name, Texture& texture)
{
    const auto binding_info = impl_->reflection.lookup(name);
    const auto& info = binding_info.value();
    if (info.descriptor_type != rhi::DescriptorType::StorageImage) return
*this;

    auto* desc_set = impl_->descriptor_sets[info.set];

    rhi::DescriptorWrite write
    {
        .binding = info.binding,
        .type = rhi::DescriptorType::StorageImage,
        .info = rhi::StorageImage{ .texture = texture.rhi_handle() }
    };

    desc_set->update({ &write, 1 });
    mark_set_dirty(info.set);
    return *this;
}

```

Фиг. 3.45 - Обвързване на текстура по име чрез рефлексия към binding в descriptor set.

3.8.9.2 Изпълнение на dispatch

Реалното изпълнение на изчисленията е концентрирано в `dispatch_impl`. Методът заделя еднократен команден буфер от `device.command_pool`, обвързва изчислителния конвейер и descriptor сетовете, подава подготвените push константи при наличие, извършва dispatch на работни групи и финализира командния буфер. Така предварително конфигурираното състояние се трансформира в конкретна последователност от изчислителни команди.

dispatch се изпълнява асинхронно, а синхронизацията се осъществява чрез `wait()`. Това позволява припокриване на изчисленията с друга процесорна работа и осигурява ясно дефинирана точка за изчакване, когато резултатите са необходими.

```
void ComputeDispatcher::dispatch_impl(uint32_t x, uint32_t y, uint32_t z)
{
    auto* cmd_pool = device->command_pool(
        device->queue_family_indices().compute_family);
    auto* cmd = cmd_pool->begin_single_time_commands();
    cmd->bind_compute_pipeline(impl_->pipeline);

    if (!impl_->descriptor_sets.empty())
        cmd->bind_descriptor_sets(
            impl_->pipeline->get_layout(), impl_->descriptor_sets, 0);
    if (impl_->has_push_constants && impl_->push_constant_size > 0)
        cmd->push_constants(impl_->pipeline_layout, rhi::ShaderStage::Compute,
            0, impl_->push_constant_size, impl_->push_constant_staging.data()
        );

    cmd->dispatch(x, y, z);
    cmd_pool->end_single_time_commands(cmd);

    impl_->dirty_sets.clear();
}
```

Фиг. 3.46 - Основен поток на `ComputeDispatcher::dispatch_impl`

3.8.10 RenderingSystem

3.8.10.1 Състав и подредба на стадии

RenderingSystem реализира пълния жизнен цикъл на рендериращия кадър като детерминирана последователност от ECS стадии със строги зависимости:

- EngineBegin инициализира графичната среда и ресурсните подсистеми.
- SanityCheck поддържа и валидира кешовете.
- Process* стадите инкрементално актуализират рендер кеша при промени в mesh, material или при инвалидиране.
- BeginFrame и CameraUboUpdate подготвят кадъра и данните на активната камера.

- CullEntities изчислява видимостта на обектите.
- EndFrame подава draw заявките и финализира кадъра.
- EngineDestroy освобождава ресурсите симетрично на инициализацията.

```
export struct RenderingSystem final
{
    struct EngineBegin : EngineBeginStage<EngineBegin>
    {
        static SystemStageConfig config()
        { return { .run_before = { InputSystem::Begin::stage_info.system } }; }

        static void execute();
    };

    struct SanityCheck : EngineRenderStage<SanityCheck>
    { static void execute(); };

    struct ProcessMeshChanged : EngineRenderStage<
        ProcessMeshChanged, With<MeshRenderer>, With<tags::MeshChanged>>
    {
        static SystemStageConfig config()
        { return { .run_after = { SanityCheck::stage_info.system } }; }

        static void execute(GameObject go, MeshRenderer& mr);
    };

    struct ProcessMaterialChanged : EngineRenderStage<
        ProcessMaterialChanged, With<MeshRenderer>,
        With<tags::MaterialChanged>>
    {
        static SystemStageConfig config()
        { return { .run_after = { ProcessMeshChanged::stage_info.system } }; }

        static void execute(GameObject go, MeshRenderer& mr);
    };

    struct ProcessInvalidated : EngineRenderStage<ProcessInvalidated,
        With<MeshRenderer>, With<tags::RenderCacheInvalidated>>
    {
        static SystemStageConfig config()
        {
            return {
                .include_disabled = true,
                .run_after = { ProcessMaterialChanged::stage_info.system }
            };
        }

        static void execute(GameObject go, MeshRenderer& mr);
    };

    struct BeginFrame : EngineRenderStage<BeginFrame>
    {
        static SystemStageConfig config()
        { return { .run_after = { ProcessInvalidated::stage_info.system } }; }

        static void execute();
    };
};
```

```

};

struct CameraUboUpdate : EngineRenderStage<CameraUboUpdate,
    With<const Camera>, With<const Transform>, With<tags::PrimaryCamera>>
{
    static SystemStageConfig config()
    { return { .run_after = { BeginFrame::stage_info.system } }; }

    static void execute(const Camera& cam, const Transform& transform);
};

struct CullEntities : EngineRenderStage<CullEntities>
{
    static SystemStageConfig config()
    { return { .run_after = { CameraUboUpdate::stage_info.system } }; }

    static void execute();
};

struct EndFrame : EngineRenderStage<EndFrame>
{
    static SystemStageConfig config()
    { return { .run_after = { CullEntities::stage_info.system } }; }

    static void execute();
};

struct EngineDestroy : EngineDestroyStage<EngineDestroy>
{ static void execute(); };
};

```

Фиг. 3.47 - Дефиниции на етапите на RenderingSystem

3.8.10.2 EngineBegin и EngineDestroy

EngineBegin инициализира еднократно графичния контекст и ресурсните подсистеми: избира бекенд чрез абстракционния слой, конфигурира swapchain и дескрипторната инфраструктура, инициализира TextureLoader, MaterialLoader и SamplerLoader и създава материалите, маркирани за незабавно зареждане. Така преди старта на кадровите стадии е установена напълно функционираща рендерираща среда на високо ниво.

EngineDestroy освобождава ресурсите в обратен ред: изчаква устройството да стане неактивно, затваря регистрите на loader-ите, изчиства рендер кешовете, освобождава RHI обектите и нулира статичното състояние на изпълнението. Това гарантира симетрично управление на жизнения цикъл и предотвратява застояли дескриптори при приключване.

```

bool RenderingSystem::init_graphics()
{
    platform::Window* window = rhi::RenderContext::window();

    bool found = false;

    for (const auto& api : rhi::graphics_apis_by_priority)

```



```

{
    if (api != rhi::graphics_apis_by_priority[0])
        cleanup_graphics_state(true);

    window->create(api);

    instance_.reset(create_instance(
        api, {
            .app_name = "Boza Application",
            .engine_name = "Boza",
            .app_version = { 0, 0, 1 },
            .engine_version = { ... },
            .window = window
        }));

    device_.reset(create_device(
        api, {
            .instance = instance_.get(),
            .window = window
        }));

    swapchain_.reset(create_swapchain(
        api, {
            .device = device_.get(),
            .window = window,
            .preferred_present_mode = app::GameSettings::gameplay.vsync
                ? rhi::PresentMode::Fifo
                : rhi::PresentMode::Mailbox,
            .preferred_image_count = 3,
            .max_frames_in_flight = 2,
            .enable_depth = true,
            .clear_color = { 0.1f, 0.1f, 0.15f, 1.0f },
            .clear_depth = 1.0f,
            .clear_stencil = 0
        }));

    descriptor_pool_.reset(create_descriptor_pool(
        api, {
            .device = device_.get(),
            .max_sets = 300,
            .pool_sizes = {
                { rhi::DescriptorType::UniformBuffer, 300 },
                { rhi::DescriptorType::CombinedImageSampler, 300 },
                { rhi::DescriptorType::StorageBuffer, 300 },
                { rhi::DescriptorType::StorageImage, 100 }
            }
        }));

    resource_cache_ = std::make_unique<rhi::ResourceCache>();

    rhi::RenderContext::initialize(...);

    found = true;
    break;
}

if (!found)
{
    cleanup_graphics_state(false);
    return false;
}

```

```

window->show();
return true;
}

```

Фиг. 3.48 - Избор на съвместим графичен програмен интерфейс

3.8.10.3 Топология на кеша и рендерирането

Централната структура при изпълнение е двустепенен кеш, индексирани първо по материал, а след това по mesh. Всяка листовка група съдържа рендер елементи (GameObject и флагове за видимост за текущия кадър). Паралелно се поддържа списък unresolved за обекти без валидна връзка към mesh или material, както и допълнителен кеш за групиране по pipeline с цел минимизиране на превключванията на състоянието при подаване на заявки. Кешът се обновява инкрементално чрез тагове за промяна и се валидира периодично чрез sanity стадии.

```

struct RenderElement
{
    GameObject entity{};
    bool visible{ false };
};

struct MeshBucket
{
    std::vector<RenderElement> elements{};
    std::uint32_t num_visible{ 0 };
};

struct MaterialRenderGroup
{
    flat_map<Mesh*, MeshBucket> mesh_buckets{};
    std::uint32_t num_visible{ 0 };
    bool should_try_instancing{ false };
};

static inline flat_map<Material*, MaterialRenderGroup> render_cache_{};
static inline std::vector<GameObject> unresolved_{};
static inline flat_map<rhi::GraphicsPipeline*, std::vector<Material*>>
pipeline_materials_{};

static inline flat_set<Mesh*> valid_meshes_{};
static inline flat_set<Mesh*> invalid_meshes_{};
static inline flat_set<Material*> valid_materials_{};
static inline flat_set<Material*> invalid_materials_{};

```

Фиг. 3.49 - Основна топология на кешовете за рендериране

3.8.10.4 SanityCheck

SanityCheck е първият рендериращ стадий за всеки кадър и изпълнява ролята на поддържаща точка. Той премахва временните маркери за разрушаване, нулира кешовете за

валидност, периодически извършва пълна проверка на кеша, опитва повторно разрешаване на нерешените ентитети и при необходимост презарежда групирането по pipeline и материал.

Интервалът между пълните проверки е умишлено по-голям от един кадър, за да се запази често изпълняваният път с ниска цена, като същевременно се гарантира окончателно почистване на остарели записи, възникнали при временни невалидни състояния.

```
void RenderingSystem::SanityCheck::execute()
{
    destroyed_meshes_this_frame_.clear();
    destroyed_materials_this_frame_.clear();

    clear_validity_caches();

    if (++sanity_frame_counter_ >= full_sanity_interval_)
    {
        sanity_frame_counter_ = 0;
        validate_render_cache();
        validate_unresolved_list();
        pipeline_materials_dirty_ = true;
    }

    try_resolve_unresolved();

    if (!pipeline_materials_dirty_) return;
    rebuild_pipeline_materials();
    pipeline_materials_dirty_ = false;
}
```

Фиг. 3.50 - Периодична поддръжка на кеша в SanityCheck

3.8.10.5 ProcessMeshChanged, ProcessMaterialChanged и ProcessInvalidated

Стадиите ProcessMeshChanged, ProcessMaterialChanged и ProcessInvalidated формират детерминирана верига за корекция. Всеки от тях премахва текущото членство в кеша, обновява локалната „мръсна“ отчетност на компонента и или повторно включва обекта в render_cache, или го премества в unresolved, ако зависимостите не са налични.

Стадият за инвалидиране има ключова роля, тъй като обработва обекти, засегнати от унищожаване на mesh или material, като изчиства остарелите имена и указатели преди повторното им поставяне в unresolved.

```
void RenderingSystem::ProcessMeshChanged::execute(GameObject go, MeshRenderer& mr)
{
    go.remove_component<tags::MeshChanged>();
}
```

```

if (mr.in_render_cache_) remove_from_render_cache(mr, go);
if (mr.in_unresolved_cache_) remove_from_unresolved(mr, go);

mr.dirty_mesh_ = false;

Mesh* mesh = mr.mesh;
Material* material = mr.material;

if (mesh && material) insert_into_render_cache(mr, go, mesh, material);
else insert_into_unresolved(mr, go);
}

void RenderingSystem::ProcessInvalidated::execute(GameObject go, MeshRenderer&
mr)
{
    go.remove_component<tags::RenderCacheInvalidated>();
    if (mr.in_render_cache_) remove_from_render_cache(mr, go);

    if (destroyed_meshes_this_frame_.contains(mr.mesh_))
    { mr.mesh_ = nullptr; mr.mesh_name_.clear(); }

    if (destroyed_materials_this_frame_.contains(mr.material_))
    { mr.material_ = nullptr; mr.material_name_.clear(); }

    if (!mr.in_unresolved_cache_) insert_into_unresolved(mr, go);
}

```

Фиг. 3.51 - Обработка на промяна и инвалидиране в RenderingSystem

3.8.10.6 BeginFrame и CameraUboUpdate

BeginFrame инициира рендериращ кадър: придобива командния контекст, обновява глобалния времеви буфер и отваря render pass за текущия кадър. При неуспешно придобиване на кадър или липсващ swapchain, стадият завършва без активиране на кадър.

CameraUboUpdate записва данните за гледна точка и проекция на активната камера в съответния буфер и обновява равнините на фростума, използвани за CPU culling.

```

void RenderingSystem::BeginFrame::execute()
{
    frame_active_ = false;
    swapchain_ -> begin_frame();
    rhi::RenderContext::set_current_command_buffer(
        swapchain_ -> current_command_buffer());
    gfx::MaterialLoader::instance()
        .update_time_ubo(Time::time(), Time::delta_time());

    swapchain_ -> begin_render_pass(swapchain_ -> current_image_index());
    frame_active_ = true;
}

void RenderingSystem::CameraUboUpdate::execute(const Camera& cam, const
Transform& transform)
{
    auto* camera_ubo = gfx::MaterialLoader::instance().camera_ubo();
    const float aspect_ratio = rhi::RenderContext::window() -> aspect_ratio();
}

```

```

const gfx::CameraUBO ubo_data {
    .view = transform.view_matrix(),
    .proj = cam.projection_matrix(aspect_ratio)
};

frustum_.set_from_view_projection(ubo_data.proj * ubo_data.view);

camera_ubo->upload(&ubo_data, sizeof(gfx::CameraUBO), 0);
}

```

Фиг. 3.52 - BeginFrame и CameraUboUpdate

3.8.10.7 CullEntities

CullEntities определя видимостта на всеки елемент и акумулира броя на видимите обекти по mesh и по материал. CPU кълингът е конфигурируем чрез флага `cpu_cull_enabled`, което позволява специфични случаи (напр. skybox) да го заобикалят, без да се променя общият път на подаване.

За всеки потенциално рендерируем обект се конструира ограждаща сфера в световни координати въз основа на локалните граници на мрежата и текущия мащаб на трансформацията. Сферата се тества спрямо шестте равнини на фростума, като флаговете за видимост и съответните броячи се актуализират на място и впоследствие се използват при подаването на draw заявките.

```

if (frustum_.sphere_visible(world_center, world_radius))
{
    elem.visible = true;
    ++bucket.num_visible;
    ++mat_group.num_visible;
}

```

Фиг. 3.53 - Проверка за видимост

3.8.10.8 Подаване на draw заявки и логика за инстанциране

Подаването на draw заявки е организирано йерархично: първо по pipeline, след това по материал и накрая по mesh-група. Тази подредба минимизира превключванията на pipeline състоянието и съответства на структурата на кеша. Преди рендериране на материал системата изгражда или извлича от кеша runtime информацията, получена чрез рефлексия, обвързва при необходимост резервни storage буфери, обновява глобалните обвързвания на двигателя и активира материала.

За всяка mesh-група се предпочита инстанциран път, когато броят на видимите елементи е достатъчен и материалът отговаря на изискванията за инстанциране. Те се валидират чрез рефлексия и включват съвместим layout на push-константния обхват и

коректно деклариран storage буфер. При несъответствие се използва резервният път с индивидуални draw извиквания и per-draw push константи.

```
const bool should_instance =
    render_info &&
    render_info->instancing_range_index.has_value() &&
    bucket.num_visible >= instancing_threshold_ &&
    bucket.num_visible <= max_instance_index;

if (should_instance && try_instanced_draw(cmd, material, bucket, render_info,
    gpu_mesh))
    continue;

draw_elements_individually(cmd, material, bucket, render_info, gpu_mesh);
```

Фиг. 3.54 - Условие за избор между инстанцирано и самостоятелно рисуване

Инстанцираният режим агрегира данните за всички видими елементи в един непрекъснат блок, качва ги еднократно в динамичен storage буфер, задава push-константен флаг use_instancing и извършва едно индексирано draw извикване с instance_count, равен на броя видими обекти. Резервният режим подава моделните данни поотделно и изпълнява по едно индексирано draw извикване за всеки видим елемент.

```
struct DrawData {
    mat4 model;
};

layout(set = 0, binding = 5) readonly buffer PcInstances {
    DrawData items[];
} pc_instances;

layout(push_constant) uniform PushConstants {
    DrawData payload;
    uint use_instancing;
} pc;
```

Фиг. 3.55 - Нужна структура на push-константите за позволяване на инстанциране

3.8.10.9 EndFrame

EndFrame приключва текущия render pass, подава командите за изпълнение, представя кадъра и нулира активното командно състояние в render контекста. Стадият използва флага frame_active_ като явен предпазен механизъм, който предотвратява подаване на draw заявки, ако BeginFrame е завършил неуспешно в рамките на същата итерация.

```
void RenderingSystem::EndFrame::execute()
```

```

{
    auto* cmd = rhi::RenderContext::current_command_buffer();
    submit_draws();

    const std::uint32_t image_idx = swapchain->current_image_index();

    rhi::RenderContext::set_current_command_buffer(nullptr);

    swapchain->end_render_pass(image_idx);
    swapchain->end_frame();
    frame_active_ = false;
}

```

Фиг. 3.56 - Финализиране на кадъра в EndFrame

3.9 Модул boza.rhi

Модулът boza.rhi е слой, който отделя рендериращата логика на двигателя от конкретен графичен API. Той задава общ, стабилен интерфейс за работата с GPU по време на изпълнение - създаване и управление на ресурси, запис и подаване на команди, както и необходимата синхронизация. Целта му не е да предлага пълния интерфейс на даден API, а да дефинира точно онези операции, от които boza.gfx и по-горните слоеве реално се нуждаят.

Отговорностите в модула са разделени ясно:

- Избор и инициализация на конкретната графична имплементация;
- Правила за жизнен цикъл, валидност и собственост на обектите;
- Помощни механизми за изграждане на pipeline;
- Runtime reflection за шейдъри и обвързвания;
- Кеширане и повторна употреба на ресурси;
- Свързване и поддръжка на глобалния render контекст.

Публичните интерфейси умишлено не съдържат детайли, специфични за дадена имплементация. Обекти като Device, Swapchain, CommandBuffer, DescriptorSet и Texture описват какви действия са възможни, а конкретната имплементация определя как те се изпълняват спрямо избрания API. Така boza.rhi остава ясната техническа граница между рендеринговия слой на двигателя и ниското ниво на работа с драйвери и платформа.

3.9.1 Подмодул boza.rhi.api

Подмодулът boza.rhi.api определя избора на графична имплементация по време на изпълнение. Изброимият тип GraphicsApi се формира условно при компилация според наличните възможности, а масивът graphics_apis_by_priority задава ред на приоритет за

резервно преминаване при неуспешна инициализация. Тази стратегия се използва от RenderingSystem при стартиране.

По този начин двигателят може последователно да опитва наличните имплементации в строго определен ред, докато намери работеща конфигурация, без логиката за приоритет да се дублира или разпилява в други части на системата.

```
enum class GraphicsApi
{
    BOZA_IF_OPENGL (OpenGL,)
    BOZA_IF_VULKAN (Vulkan,)
    BOZA_IF_METAL (Metal,)
    BOZA_IF_DX11 (DirectX11,)
    BOZA_IF_DX12 (DirectX12)
};

inline constexpr std::array graphics_apis_by_priority
{
    BOZA_IF_DX12 (GraphicsApi::DirectX12,)
    BOZA_IF_METAL (GraphicsApi::Metal,)
    BOZA_IF_VULKAN (GraphicsApi::Vulkan,)
    BOZA_IF_DX11 (GraphicsApi::DirectX11,)
    BOZA_IF_OPENGL (GraphicsApi::OpenGL,)
};
```

Фиг. 3.57 - Избор на графична имплементация и ред на приоритет

3.9.2 Подмодул boza.rhi.objects

3.9.2.1 Основен протокол за създаване

Всички семейства обекти в RHI следват общ модел, реализиран чрез шаблона GraphicsObject<Derived, Desc>. Базовият клас съхранява типизиран описател и дефинира задължителните операции за жизнения цикъл - init() и destroy(). Статичният помощен метод create<Concrete>() създава конкретната имплементация, извиква инициализацията и връща обекта само ако тя е успешна. Така се гарантира, че към извикващия код никога не достига частично създаден или невалиден обект.

Този подход осигурява единно управление на собствеността и унифицирана семантика на създаване и унищожаване за всички основни RHI обекти - Instance, Device, Swapchain, дескрипторни структури, ресурси и конвейери.

```
template<typename Derived, typename Desc>
class GraphicsObject
{
public:
    virtual ~GraphicsObject() = default;

    virtual bool init() = 0;
```



```

virtual void destroy() = 0;

template<typename Concrete>
static Derived* create(const Desc& desc)
{
    const auto ptr = new Concrete(desc);

    if (!ptr->init())
    {
        delete ptr;
        return nullptr;
    }

    return ptr;
}

protected:
    explicit GraphicsObject(const Desc& desc) : desc_(desc) {}
    Desc desc_;
};

```

Фиг. 3.58 - Общ модел за жизнения цикъл и създаване на RHI обекти

3.9.2.2 Instance

Instance е първичният графичен обект, който представя връзката между приложението, платформата и графичната имплементация. Той съдържа метаданни за име и версия на приложението и двигателя, както и указател към прозореца, необходим за договаряне на платформено-зависимите разширения.

```

struct InstanceDesc
{
    std::string app_name;
    std::string engine_name;

    glm::u8vec3 app_version;
    glm::u8vec3 engine_version;

    Window* window;
};

class Instance : public GraphicsObject<Instance, InstanceDesc>{ ... };

```

Фиг. 3.59 - InstanceDesc и декларация на rhi::Instance

3.9.2.3 Device

Device представя логическото устройство и централизира достъпа до опашките за команди и свързаните с тях CommandPool обекти. При инициализация устройството открива и фиксира индексите на семействата опашки (QueueFamilyIndices) и впоследствие предоставя достъп до CommandQueue и CommandPool чрез индекс на семейство.

Интерфейсът дефинира изрично четири роли - graphics, present, compute и transfer, за да осигури предвидим избор на опашка. Семействата се определят еднократно при инициализацията на устройството и след това се използват директно, без допълнителни проверки по време на кадър.

```
struct DeviceDesc
{
    Instance* instance;
    platform::Window* window;
};

class Device : public GraphicsObject<Device, DeviceDesc>
{
public:
    struct QueueFamilyIndices
    {
        uint32_t graphics_family{ std::numeric_limits<uint32_t>::max() };
        uint32_t present_family{ std::numeric_limits<uint32_t>::max() };
        uint32_t compute_family{ std::numeric_limits<uint32_t>::max() };
        uint32_t transfer_family{ std::numeric_limits<uint32_t>::max() };
    };

    CommandQueue* queue(uint32_t queue_family_index);
    CommandPool* command_pool(uint32_t queue_family_index);

    virtual void wait_idle() = 0;

protected:
    explicit Device(const DeviceDesc& desc) : GraphicsObject(desc) {}

    QueueFamilyIndices queue_family_indices_{};

    flat_map<std::uint32_t, std::unique_ptr<CommandQueue>> queues_{};
    flat_map<std::uint32_t, std::unique_ptr<CommandPool>> command_pools_{};
};
```

Фиг. 3.60 - Опростена дефиниция на *rhi::Device*

3.9.2.4 Swapchain

Swapchain определя последователността от операции за обработка на един кадър - begin_frame, acquire_next_image, begin_render_pass, end_render_pass, submit, present и end_frame. Освен това предоставя текущия индекс на кадъра/изображението и синхронизационните примитиви (командни буфери и fence) за активната рамка. Моделът е умишлено строг, тъй като рендериращите стадии разчитат на детерминирани преходи и не могат безопасно да продължат при невалиден swapchain.

```
enum class PresentMode : std::uint8_t
{
    Immediate,    // No vsync (may tear)
    Mailbox,      // Triple buffering (as fast as it can get with no tearing)
```

```

    Fifo,           // Vsync, always supported (bound to monitor refresh rate)
    FifoRelaxed // Vsync with relaxed timing (allow late frames to be
displayed immediately, may tear)
};

struct SwapchainDesc
{
    Device* device;
    Window* window;

    PresentMode    preferred_present_mode{ PresentMode::Mailbox };
    std::uint32_t preferred_image_count{ 3 };
    std::uint32_t max_frames_in_flight{ 2 };
    bool          enable_depth{ false };
    DepthFormat   depth_format{ DepthFormat::Auto };

    std::array<float, 4> clear_color{ 0.0f, 0.0f, 0.0f, 1.0f };
    float             clear_depth{ 1.0f };
    std::uint32_t     clear_stencil{ 0 };
};

class Swapchain : public GraphicsObject<Swapchain, SwapchainDesc>
{
public:
    virtual bool begin_frame() = 0;
    virtual bool end_frame() = 0;

    virtual std::uint32_t acquire_next_image() = 0;
    virtual bool present(std::uint32_t image_index) = 0;

    virtual bool begin_render_pass(std::uint32_t image_idx) = 0;
    virtual bool end_render_pass(std::uint32_t image_idx) = 0;

    virtual CommandBuffer* current_command_buffer() = 0;
    virtual Fence* current_fence() = 0;

protected:
    explicit Swapchain(const SwapchainDesc& desc) : GraphicsObject(desc) {}
    virtual bool recreate() = 0;
};

```

Фиг. 3.61 - Опростена дефиниция на *rhi::Swapchain*

3.9.2.5 Командни обекти

3.9.2.5.1 CommandPool

CommandPool заделя и рециклира командни буфери за едно семейство опашки. Той поддържа както дълготрайни (persistent) буфери, така и помощни функции за еднократни команди, използвани при качване на данни и преходи на ресурси.

```

enum class CommandPoolOption : std::uint8_t
{
    None = 0b00,
    Transient = 0b01, // Command buffers allocated from pool are short-lived
    ResetCommandBuffer = 0b10 // Allow individual command buffer reset
};

```

```

struct CommandPoolDesc
{
    Device*           device;
    Flags<CommandPoolOption> flags;
    std::uint32_t     queue_family_index;
};

class CommandPool : public GraphicsObject<CommandPool, CommandPoolDesc>
{
public:
    virtual CommandBuffer* allocate_command_buffer(bool is_primary) = 0;
    virtual void free_command_buffer(CommandBuffer* command_buffer) = 0;

    virtual CommandBuffer* begin_single_time_commands() = 0;
    virtual bool end_single_time_commands(CommandBuffer* command_buffer) = 0;

protected:
    CommandPool(const CommandPoolDesc& desc) : GraphicsObject(desc) {}
};

```

Фиг. 3.62 - Опростена дефиниция на *rhi::CommandPool*

3.9.2.5.2 CommandBuffer

CommandBuffer предоставя операции за запис на команди, обвързване на pipeline, индексирано рисуване и dispatch, обвързване на DescriptorSet, push константи и явни бариери за изображения. Интерфейсът е умишлено на ниско ниво, тъй като boza.gfx го използва директно при рендериране на всеки кадър и при изчислителни задачи.

```

class CommandBuffer
{
public:
    virtual bool begin() = 0;
    virtual bool end() = 0;

    virtual void draw(...) = 0;
    virtual void draw_indexed(...) = 0;

    virtual void dispatch(...) = 0;

    virtual void bind_graphics_pipeline(GraphicsPipeline* pipeline) = 0;
    virtual void bind_compute_pipeline(ComputePipeline* pipeline) = 0;

    virtual void bind_vertex_buffer(...) = 0;
    virtual void bind_index_buffer(...) = 0;

    virtual void bind_descriptor_set(...) = 0;

    virtual void push_constants(...) = 0;

    virtual void image_barrier(...) = 0;

protected:
    explicit CommandBuffer(const CommandBufferDesc& desc) : desc_(desc) {}
    CommandBufferDesc desc_;
};

```

3.9.2.5.3 CommandQueue

CommandQueue изпълнява записаните командни буфери и обединява подаването за показване на изображението в една абстракция. Той връща типизиран PresentResult, което позволява на горните слоеве изрично да реагират при неактуална или неподходяща повърхност.

```
struct SubmitInfo
{
    std::vector<CommandBuffer*>    command_buffers;
    std::vector<Semaphore*>        wait_semaphores;
    std::vector<std::uint32_t>     wait_stages;
    std::vector<Semaphore*>        signal_semaphores;
    Fence*                         signal_fence{ nullptr };
};

struct PresentInfo
{
    std::vector<Swapchain*>        swapchains;
    std::vector<std::uint32_t>     image_indices;
    std::vector<Semaphore*>        wait_semaphores;
};

class CommandQueue : public GraphicsObject<CommandQueue, CommandQueueDesc>
{
public:
    virtual bool submit(const SubmitInfo& submit_info) = 0;
    virtual PresentResult present(const PresentInfo& present_info) = 0;
    virtual bool wait_idle() = 0;

protected:
    CommandQueue(const CommandQueueDesc& desc) : GraphicsObject(desc) {}
};
```

Фиг. 3.64 - Интерфейс на *rhi::CommandQueue* за *submit* и *present* операции

3.9.2.6 Обекти за синхронизация

Моделът за синхронизация се основава на два обекта: Semaphore и Fence. Semaphore осигурява координация между операции на графичния процесор и поддържа както бинарен, така и времеви режим в един интерфейс. Fence служи за синхронизация между CPU и GPU и предоставя операции за изчакване, нулиране и проверка дали изпълнението е завършило.

Разграничението е съществено за работата на двигателя: семафорите управляват последователността и темпото на подаване на задачи към GPU, докато Fence указва кога процесорът може безопасно да използва отново ресурсите на даден кадър.

```

enum class SemaphoreType : std::uint8_t { Binary, Timeline };

struct SemaphoreDesc
{
    Device*      device;
    SemaphoreType type{ SemaphoreType::Binary };
    std::uint64_t value{ 0 }; // Only for timeline semaphores
};

class Semaphore : public GraphicsObject<Semaphore, SemaphoreDesc>
{
public:
    virtual bool signal(std::uint64_t value) = 0;
    virtual bool wait(std::uint64_t value, std::uint64_t timeout) = 0;
    virtual std::uint64_t counter_value() const = 0;
};

struct FenceDesc { Device* device; bool signaled = false; };

class Fence : public GraphicsObject<Fence, FenceDesc>
{
public:
    virtual bool wait(std::uint64_t timeout) = 0;
    virtual bool reset() = 0;
    virtual bool is_signaled() const = 0;
};

```

Фиг. 3.65 - Обекти за синхронизация

3.9.2.7 Дескрипторни обекти

Дескрипторите се управляват чрез три обекта: DescriptorSetLayout описва схемата на обвързванията, DescriptorPool управлява разпределянето и нулирането на множества, а DescriptorSet представлява конкретен набор от обвързани ресурси. DescriptorWrite използва типизиран variant за данни на буфери и изображения, което позволява динамични актуализации по време на изпълнение, без системите на високо ниво да работят с API-специфични структури.

```

using DescriptorInfo = std::variant<
    UniformBuffer,
    StorageBuffer,
    CombinedImageSampler,
    StorageImage>;
struct DescriptorWrite
{
    std::uint32_t binding;
    std::uint32_t array_element;
    DescriptorType type;
    DescriptorInfo info;
};

```

Фиг. 3.66 - Типизиран запис на дескриптор чрез DescriptorWrite

3.9.2.8 Ресурсни обекти

Buffer предоставя операции за map/unmap, upload, read_back и заявка за размера. Texture поддържа upload, read_back и явни преходи на разположението (layout transition). Sampler съхранява параметрите за филтриране, обвиване и сравнение.

```
enum class BufferMemoryType : std::uint8_t
{
    DeviceLocal,
    HostVisible,
    HostCoherent
};

struct BufferDesc
{
    Device*      device;
    size_t       size;
    BufferUsage   usage;
    BufferMemoryType memory_type;
};

class Buffer : public GraphicsObject<Buffer, BufferDesc>
{
public:
    virtual void* map() = 0;
    virtual void unmap() = 0;

    virtual void upload(const void* data, size_t size, size_t offset) = 0;
    virtual void read_back(void* data, size_t size, size_t offset) = 0;

protected:
    explicit Buffer(const BufferDesc& desc) : GraphicsObject(desc) {}

    mutable std::mutex resource_mutex_;
    std::atomic_bool in_use_{ false };
};

struct TextureDesc
{
    Device* device;

    TextureType      type;
    TextureFormat     format;
    TextureSampleCount sample_count{ TextureSampleCount::Count1 };
    Flags<TextureUsage> usage;

    std::uint32_t width{ 1 };
    std::uint32_t height{ 1 };
    std::uint32_t depth{ 1 };

    std::uint32_t mip_levels{ 1 };
    std::uint32_t array_layers{ 1 };
};

class Texture : public GraphicsObject<Texture, TextureDesc>
{
public:
```

```

virtual void upload(const void* data, size_t size, uint32_t layer) = 0;
virtual void read_back(void* data, size_t size, uint32_t layer) = 0;
virtual void transition_layout(TextureLayout, TextureLayout) = 0;

protected:
    explicit Texture(const TextureDesc& desc) : GraphicsObject(desc) {}

    mutable std::mutex resource_mutex_;
    std::atomic_bool in_use_{ false };
};

```

Фиг. 3.67 - Опростен интерфейс на *rhi::Buffer* и *rhi::Texture*

3.9.2.9 ShaderModule и Metadata

ShaderModule съхранява идентификатора на компилирания шейдър заедно с извлечената метаинформация, използвана от системите за pipeline и материали. Тези данни описват ресурсните обвързвания, декларациите на push константите, структурните типове и размера на работната група при compute шейдъри. По този начин рефлексията се използва като основен източник на информация по време на изпълнение, а не като помощна функция само за инструменти.

```

class ShaderModule : public GraphicsObject<ShaderModule, ShaderModuleDesc>
{
    struct PushConstantMember
    {
        std::string name;
        std::string type_name;
        ShaderDataType data_type{ ShaderDataType::Unknown };
        // ...
    };

    struct StructType
    {
        std::uint32_t size{ 0 };
        std::vector<PushConstantMember> members;
    };

    struct ShaderResource
    {
        std::uint32_t set{ std::numeric_limits<std::uint32_t>::max() };
        std::uint32_t binding{ std::numeric_limits<std::uint32_t>::max() };
        std::uint32_t location{ std::numeric_limits<std::uint32_t>::max() };
        // ...
    };

    struct PushConstant
    {
        ShaderStage stage;
        std::uint32_t offset{ 0 };
        std::uint32_t size{ 0 };
        std::vector<PushConstantMember> members;
    };

    struct MetaData
    {

```



```

        flat_map<std::string, StructType> struct_types;
        flat_map<std::string, ShaderResource> uniform_buffers;
        flat_map<std::string, ShaderResource> storage_buffers;
        flat_map<std::string, ShaderResource> stage_inputs;
        flat_map<std::string, ShaderResource> stage_outputs;
        flat_map<std::string, ShaderResource> subpass_inputs;
        flat_map<std::string, ShaderResource> sampled_images;
        flat_map<std::string, ShaderResource> storage_images;
        flat_map<std::string, PushConstant> push_constants;
        glm::uvec3 work_group_size{ 1, 1, 1 };
    };
};

```

Фиг. 3.68 - Опростена дефиниция на *rhi::ShaderModule* и вътрешни структури

3.9.2.10 Конвейерни обекти

PipelineLayout обединява оформленията на дескрипторните множества и обхватите на push константите, дефинирани в шейдърите, в единна схема на обвързване.

GraphicsPipeline и ComputePipeline използват това оформление заедно със съответните шейдърни модули.

```

struct PipelineLayoutDesc
{
    Device* device;
    std::vector<ShaderModule*> shaders;
    std::vector<DescriptorSetLayout*> set_layouts;
};

struct GraphicsPipelineDesc
{
    Device* device;

    std::vector<ShaderModule*> shaders;
    PipelineLayout* layout;

    PrimitiveTopology topology{ PrimitiveTopology::TriangleList };
    std::vector<VertexInputBinding> bindings;
    std::vector<VertexInputAttribute> attributes;

    RasterizationState rasterization;
    DepthStencilState depth_stencil;
    ColorBlendState color_blend;
    MultisampleState multisample;
};

```

Фиг. 3.69 - Описатели за създаване на *rhi::PipelineLayout* и *rhi::GraphicsPipeline*

3.9.3 Подмодул boza.rhi.factory

Фабричният подмодул преобразува избраната графична имплементация (GraphicsApi) в конкретно създаване на обекти. Всички фабрични функции делегират реалното създаване

към вътрешния статичен метод `GraphicsObject::create<Concrete>()`, който инстанцира съответния клас, извършва инициализацията и връща обекта само при успех.

По този начин се гарантира единна семантика на създаване за всички RHI обекти - към извикващия код никога не се връща частично инициализиран или невалиден обект, а управлението на жизнения цикъл остава централизирано в базовия клас.

```
#define FACTORY_FUNC(CLASS, CLASS_LOWER) \
    CLASS* create_##CLASS_LOWER(GraphicsApi api, const CLASS##Desc& desc) { \
        switch (api) { \
            BOZA_IF_VULKAN( \
                case GraphicsApi::Vulkan: return CLASS::create<vk::CLASS>(desc);) \
            // ... \
            default: return nullptr; \
        } \
    } \

FACTORY_FUNC(Instance, instance)
FACTORY_FUNC(Device, device)
FACTORY_FUNC(Swapchain, swapchain)
```

Фиг. 3.70 - Фабрични функции `create_` за избор на имплементация според `GraphicsApi`

3.9.4 PipelineBuilder

3.9.4.1 Създаване на DescriptorSetLayout

PipelineBuilder изгражда инстанции на DescriptorSetLayout директно от метаданните на шейдърите. Алгоритъмът обединява всички деклариран ресурси по (set, binding) и за всеки индекс на set създава по един DescriptorSetLayout в нарастващ ред. Видимостта по етапи се акумулира между всички участващи шейдъри, така че едно и също обвързване, използвано в повече от един етап, да бъде описано коректно в крайното оформление.

```
bool PipelineBuilder::build_descriptor_set_layouts()
{
    auto bindings_by_set = merge_descriptor_bindings();
    for (std::uint32_t set_idx = 0; set_idx <= max_set; ++set_idx)
    {
        std::vector<DescriptorSetLayoutBinding> layout_bindings;

        if (bindings_by_set.contains(set_idx))
        {
            for (const auto& [binding, type, stages, count] : bindings)
                layout_bindings.emplace_back(binding, type, stages, count);
        }
        auto* layout = create_descriptor_set_layout(
            api_, {
                .device = device_,
                .bindings = layout_bindings
            });
    }
}
```

```

        descriptor_set_layouts_.push_back(layout);
    }

    return true;
}

```

Фиг. 3.71 - Създаване на `rhi::DescriptorSetLayout` според извлечените метаданни

3.9.4.2 Създаване на `PipelineLayout`, `GraphicsPipeline` и `ComputePipeline`

След като `DescriptorSetLayout` обектите са създадени, `PipelineBuilder` изгражда `PipelineLayout`, а върху него - конкретен pipeline: графичен (`GraphicsPipeline`) или изчислителен (`ComputePipeline`).

При графичен pipeline той извежда описанието на входните върхови атрибути от vertex шейдъра и съгласува настройките за смесване на цветовете с броя на цветните прикачвания на текущата цел за рендериране. Така конфигурацията се определя еднозначно от входните данни и се намалява необходимостта от ръчно настройване в по-горните системи.

```

PipelineLayout* PipelineBuilder::build_pipeline_layout()
{
    pipeline_layout_ = create_pipeline_layout(
        api_, {
            .device = device_,
            .shaders = shaders_,
            .set_layouts = descriptor_set_layouts_
        });

    return pipeline_layout_;
}

GraphicsPipeline* PipelineBuilder::build_graphics_pipeline(...) const
{
    // ...

    auto* pipeline = create_graphics_pipeline(
        api_, {
            .device = device_,
            .shaders = shaders_,
            .layout = pipeline_layout_,
            .topology = topology,
            .bindings = bindings,
            .attributes = attributes,
            .rasterization = rasterization,
            .depth_stencil = depth_stencil,
            .color_blend = adjusted_color_blend,
            .color_attachment_formats = color_attachment_formats,
            .depth_attachment_format = depth_attachment_format,
            .stencil_attachment_format = 0
        });

    return pipeline;
}

```

3.9.5 DescriptorReflection

DescriptorReflection обединява метаданните на шейдърите в таблица за търсене по символни имена. Членовете на uniform буферите се експортират със символни псевдоними в точкова нотация (напр. `"material.albedo_color"`), storage буферите се обединяват чрез акумулиране на етапите, обхватите на push константи се проследяват с отмествания на членове, а типовете структури се обединяват за проверка на съответствието. Това е ключовата връзка между символните актуализации на свойства в *boza.gfx* и точните отмествания на дескрипторните множества по време на изпълнение.

```
void DescriptorReflection::build_from_shaders(span<ShaderModule*> shaders)
{
    for (const auto* shader : shaders)
    {
        merge_struct_types(metadata.struct_types);

        for (const auto& [name, resource] : metadata.uniform_buffers)
            add_uniform_buffer_members(name, resource);

        for (const auto& [name, resource] : metadata.storage_buffers)
            add_storage_buffer(name, resource, stage_flags);

        for (const auto& [name, resource] : metadata.sampled_images)
        {
            const BindingInfo info{ ... };
            bindings_[name] = info;
        }

        for (const auto& [name, resource] : metadata.storage_images)
        {
            const BindingInfo info{ ... };
            bindings_[name] = info;
        }

        for (const auto& [name, pc] : metadata.push_constants)
            add_push_constant_range(name, pc, stage_flags);
    }

    for (const auto& [range_name, range_info] : push_constant_ranges_)
        add_push_constant_members(range_name, range_info);
}
```

Фиг. 3.73 - Обединяване на рефлексираната информация от шейдърните етапи

3.9.6 ResourceCache

ResourceCache централизира управлението на шейдърните модули, текстурите и кешираните конфигурации на конвейери чрез асоциативни таблици, защитени с mutex. Записите за шейдъри и текстури се съхраняват като `weak_ptr`, което позволява повторна

употреба без налагане на собственост върху ресурсите. Кешовете за графични и изчислителни конвейери съдържат нисконивно представяне на обектите, организирано по стабилни ключове.

Този механизъм предотвратява повторно създаване на шейдърни модули и скъпото изграждане на конвейери при еквивалентни конфигурации, като по този начин намалява времето за инициализация и натоварването по време на изпълнение.

```
struct GraphicsPipelineKey
{
    std::string vertex_shader;
    std::string fragment_shader;
    std::size_t settings_hash{ 0 };

    bool operator==(const GraphicsPipelineKey&) const = default;

    struct Hash { ... };
};

struct CachedGraphicsPipeline
{
    GraphicsPipeline* pipeline{ nullptr };
    PipelineLayout* layout{ nullptr };
    std::vector<DescriptorSetLayout*> descriptor_set_layouts;
};

template<typename T, typename KeyType, typename MapType>
std::shared_ptr<T> get_or_create(...)
{
    std::lock_guard lock{ mutex };

    if (auto it = cache.find(key); it != cache.end())
        if (auto shared = it->second.lock()) return shared;

    T* raw_ptr = factory();

    auto shared = std::shared_ptr<T>(raw_ptr,
        [](T* ptr) { ptr->destroy(); delete ptr; });

    cache[key] = shared;
    return shared;
}
```

Фиг. 3.74 - Модел на споделен кеш за ресурси

3.9.7 RenderContext

RenderContext е глобален обект, който обединява текущото състояние на рендерирането: активния прозорец, устройството, Swapchain, ResourceCache, DescriptorPool, текущия команден буфер и избрания графичен API. Той служи като централизирана точка за достъп до тези обекти, без да се налага предаването им през множество слоеве на системата.

```

class RenderContext final
{
public:
    static void initialize(...);

    static void set_window(platform::Window* window);
    static void set_current_command_buffer(CommandBuffer* command_buffer);

    static platform::Window* window();
    static Device* device();
    static Swapchain* swapchain();
    static ResourceCache* resource_cache();
    static DescriptorPool* descriptor_pool();
    static CommandBuffer* current_command_buffer();
    static GraphicsApi api();
};

```

Фиг. 3.75 - Опростен интерфейс на RenderContext

3.9.8 BufferLayoutRules

BufferLayoutRules моделира правилата за подравняване и размер на данните в буфери и push константи съгласно стандартите Std140 и Std430. Тъй като push константите, uniform и storage буферите използват различни схеми за оформление, модулът изчислява подравняване, размер и отместване по детерминистичен начин според конкретния режим. Това позволява при запис от страна на процесора отместванията да се изчисляват коректно и да съвпадат с очакваното разположение на данните в шейдъра, без ръчно пресмятане или твърдо кодирани стойности.

```

class Std140LayoutRules final : public BufferLayoutRules
{
public:
    std::size_t get_alignment(const ShaderValueType type) const override;
    std::size_t get_size(const ShaderValueType type) const override;
    std::size_t get_array_stride(const ShaderValueType type) const override;
    std::size_t align_offset(
        const std::size_t offset, const ShaderValueType type) const override;
};

inline const BufferLayoutRules& get_layout_rules(BufferLayoutType type)
{
    static Std140LayoutRules std140;
    static Std430LayoutRules std430;

    switch (type)
    {
        case BufferLayoutType::PushConstant:
        case BufferLayoutType::Std140: return std140;
        case BufferLayoutType::Std430: return std430;
    }
}

```

3.9.9 Помощни функции за проверка на тип и размер

Помощният типов слой извършва преобразуване от GLM стойности към ShaderDataType и валидира съответствието им спрямо отразените метаданни на шейдъра.

В boza.gfx се използват при Debug конфигурация, за да се извършват проверки дали подаденият тип и размер съвпадат с очакваните според рефлексията - както за членове на uniform буферите, така и за push константи и други дескрипторни обвързвания.

Така се гарантира, че към графичния слой се подават структурно коректни данни, а грешките се откриват рано - още при актуализация на свойствата, а не едва при изпълнение на шейдъра.

```
template<typename T>
constexpr ShaderDataType get_expected_shader_type()
{
    if constexpr (std::same_as<T, float>) return ShaderDataType::Float;
    // ...
    else if constexpr (std::same_as<T, glm::mat4>)
        return ShaderDataType::Mat4;
    else return ShaderDataType::Unknown;
}

constexpr std::size_t get_type_size(const ShaderDataType type)
{
    switch (type)
    {
        case ShaderDataType::Float: return sizeof(float);
        // ...
        case ShaderDataType::Mat4: return sizeof(glm::mat4);
        default: return 0;
    }
}

constexpr std::string_view get_type_name(const ShaderDataType type)
{
    switch (type)
    {
        case ShaderDataType::Float: return "float";
        // ...
        case ShaderDataType::Mat4: return "mat4";
        // ...
        case ShaderDataType::SamplerCube: return "samplerCube";
        default: return "unknown";
    }
}

bool validate_property_type(
    const ShaderDataType provided_type,
    const std::size_t provided_size,
    const ShaderDataType actual_type,
    const std::size_t actual_size)
{

```

```

return provided_size == actual_size && (
    provided_type == actual_type ||
    provided_type == ShaderDataType::Unknown);
}

```

Фиг. 3.77 - Помощни функции за проверка на отразени типове

3.10 Модул boza.rhi.vulkan

Модулът boza.rhi.vulkan предоставя конкретна имплементация на класовете в boza.rhi.objects за графичния API Vulkan. Всеки интерфейсен обект в boza.rhi.objects се съпоставя с клас vk::* със същия жизнен цикъл.

3.10.1 vk::Instance

vk::Instance инициализира Vulkan чрез Volk и създава VkInstance с необходимите разширения и слоеве.

Volk зарежда адресите на Vulkan функциите по време на изпълнение, което позволява коректно използване на API без статично обвързване към конкретна библиотека. След създаване на инстанцията се извършва допълнително зареждане на функциите, свързани с нея.

Преди извикване на vkCreateInstance() се проверява дали всички изисквани разширения и слоеве са налични. Така при липса на зависимост инициализацията се прекратява рано и се избягват частични или невалидни състояния.

В Debug конфигурация се създава и механизъм за получаване на диагностични съобщения от валидационния слой, които се пренасочват към системата за логване.

```

bool Instance::init()
{
    if (!vk_check(volkInitialize(), "Failed to initialize Volk"))
        return false;

    if (!create_instance()) return false;

    volkLoadInstance(vk_instance_);

#ifdef BOZA_DEBUG
    if (!create_debug_messenger()) return false;
#endif

    return true;
}

```

Фиг. 3.78 - Последователност при стартиране на Vulkan инстанция

3.10.2 vk::Device

3.10.2.1 Последователност на създаване и необходими зависимости

vk::Device създава и конфигурира логическото устройство и всички свързани с него ресурси, като използва вече инициализираната vk::Instance и повърхността за изобразяване. Процесът включва избор на подходящо физическо устройство, откриване на необходимите семейства опашки (графична, за представяне и при възможност отделни за изчисления и трансфер), създаване на логическото устройство с активирани нужни разширения и възможности, и зареждане на функциите му чрез Volk. След това се създават обвивки за опашките, инициализират се обекти за управление и разпределяне на командни буфери за съответните семейства опашки и се създава алокаторът (VMA), който централизира управлението на графичната памет.

```
bool Device::init()
{
    if (!desc_.window->create_vulkan_surface(vk_instance, surface_))
        return false;

    if (!choose_physical_device() ||
        !find_queue_families() ||
        !create_logical_device())
        return false;

    volkLoadDevice(logical_device_);

    if (!get_queues()) return false;
    if (!create_command_pools()) return false;

    allocator_.reset(Allocator::create<Allocator>({ ... }));
    return allocator_ != nullptr;
}
```

Фиг. 3.79 - Поетапна инициализация на vk::Device

3.10.2.2 Избор на физическо устройство

При избора на физическо устройство се проверяват изискваните разширения и изискваните функции на Vulkan 1.3, като synchronization2 и dynamicRendering. Проверките на възможностите на семействата опашки се правят на по-късен етап, но ранните проверки на разширения и функции изключват несъвместимите устройства. Избраната стратегия предпочита най-напред годността и съвместимостта на устройството, а след това специализацията на опашките в следващите стъпки.

```
for (const auto& device : physical_devices)
{
```

```

// ...

if (!supported_vk13_features.synchronization2 ||
    !supported_vk13_features.dynamicRendering)
    continue;

physical_device_ = device;
return true;
}

```

Фиг. 3.80 - Цикъл за филтриране на подходящо устройство

3.10.2.3 Откриване на семейства опашки

При откриване на семейства опашки системата отдава приоритет на специализирани семейства за изчисления и за прехвърляне на данни, когато такива са налични. Ако липсват, се използват общи семейства, които поддържат повече от един тип операции. Този подход позволява графичните и асинхронните задачи да се разпределят по-ефективно и да се намали конкуренцията между тях, без да се усложнява логиката в по-горните слоеве на двигателя.

```

if (!found_compute_family &&
    (props.queueFlags & VK_QUEUE_COMPUTE_BIT) &&
    !(props.queueFlags & VK_QUEUE_GRAPHICS_BIT))
{
    queue_family_indices.compute_family = i;
    found_compute_family = true;
}

if (!found_transfer_family &&
    (props.queueFlags & VK_QUEUE_TRANSFER_BIT) &&
    !(props.queueFlags & VK_QUEUE_GRAPHICS_BIT) &&
    !(props.queueFlags & VK_QUEUE_COMPUTE_BIT))
{
    queue_family_indices.transfer_family = i;
    found_transfer_family = true;
}

```

Фиг. 3.81 - Предпочитан избор на специални семейства опашки

3.10.3 vk::Allocator

vk::Allocator централизира създаването и унищожаването на алокатора от VMA (Vulkan Memory Allocator) и го изгражда върху вече избраното физическо устройство, логическото устройство и Vulkan инстанцията. При инициализация класът попълва VmaVulkanFunctions с указатели към нужните Vulkan функции и ги подава на VMA, за да може библиотеката да работи без скрити зависимости и с точно този Vulkan контекст. След това се създава един общ алокатор (VmaAllocator), който се използва от всички типове

ресурси - както буфери, така и изображения, така че управлението на паметта да следва единна политика и да не се дублира логика в отделните класове.

VMA се използва вместо ръчно заделяне на памет с функциите, които Vulkan предоставя, защото автоматизира избора на подходящ тип памет, обединява множество заделяния в по-големи блокове и значително опростява управлението на буфери и изображения, като намалява риска от грешки и фрагментация.

```
const VmaAllocatorCreateInfo allocator_info
{
    .physicalDevice = physical_device,
    .device = logical_device,
    .pVulkanFunctions = &functions,
    .instance = vk_instance,
    .vulkanApiVersion = vk_api_version_1_3
};

vmaCreateAllocator(&allocator_info, &vma_allocator_);
```

Фиг. 3.82 - Настройка на VMA алокатор във `vk::Allocator`

3.10.4 `vk::Swapchain`

`vk::Swapchain` управлява подаването на кадри към прозореца и реализира цикъла „придобиване – изобразяване – представяне“. Той обвива `VkSwapchainKHR`, изображенията му и техните изгледи (`VkImageView`), опционален дълбочинен буфер, както и синхронизацията и командните буфери за няколко кадъра в обработка. Всеки „кадър в обработка“ е отделен пакет ресурси (команден буфер + fence + семафори), което позволява процесорът да подготвя следващ кадър, докато предишният все още се изпълнява на GPU или се представя на екрана.

Swapchain-ът обикновено съдържа няколко изображения (в случая три). В даден момент едно изображение може да е вече показано или в процес на представяне, друго да се рендерира, а трето да е следващото налично за придобиване. Тази ротация намалява блокирането при изчакване и стабилизира честотата на кадрите при краткотрайни забавяния.

3.10.4.1 Последователност на инициализацията

Инициализацията създава всички зависимости в последователност, която избягва частично готов swapchain. Първо се заявяват възможностите на повърхността (`query_swapchain_support`), след което се създава `VkSwapchainKHR` и се извличат изображенията му. За всяко изображение се създава `VkImageView`, а текущото му

разположение се проследява в `image_layouts_`. Ако е активирана дълбочина, се създава дълбочинно изображение и изглед към него. След това се заделя по един команден буфер за всеки кадър в обработка и се създават обектите за синхронизация (`fence` + семафори). Callback-ът за преоразмеряване на прозореца се регистрира едва накрая, само ако цялата конструкция е успешна.

```
bool Swapchain::init()
{
    frames_.resize(desc_.max_frames_in_flight);

    if (!query_swapchain_support() ||
        !create_vk_swapchain() ||
        !create_image_views() ||
        !create_depth_resources() ||
        !create_command_buffers() ||
        !create_sync_objects())
        return false;

    desc_.window->set_window_resize_callback();
    return true;
}
```

Фиг. 3.83 - Инициализация на *Swapchain*: ресурси и зависимости

3.10.4.2 Цикъл за придобиване и представяне

Началото на кадъра (`begin_frame`) придобива следващо изображение и подготвя командния буфер на текущия кадър в обработка. Придобиването е защитено: първо се изчаква `fence`-ът на текущия пакет ресурси, за да се гарантира, че GPU е приключил предишната му употреба, след което се извиква `vkAcquireNextImageKHR`. Ако прозорецът е минимизиран, връща се специална стойност за „пропускане“ на кадъра, а при `out-of-date/suboptimal` или прекомерно изчакване се подава сигнал за пресъздаване.

```
bool Swapchain::begin_frame()
{
    current_image_index_ = acquire_next_image();

    if (current_image_index_ == invalid_image_idx ||
        current_image_index_ == skip_image_idx) return false;

    frame_started_ = true;
    const auto& frame = frames_[current_frame_];
    if (!frame.cmd_buffer->reset()) return false;
    if (!frame.cmd_buffer->begin()) return false;

    return true;
}
```

Фиг. 3.84 - Начало на кадър: придобиване на изображение и подготовка на командния

буфер

present() подава командния буфер към графичната опашка, като изчаква семафора за „изображението е налично“, и сигнализира семафора за „рендерирането приключи“, след което опашката за представяне показва изображението. При out-of-date/suboptimal от представянето също се активира пресъздаване, но кадърът се счита за успешно приключен, за да не се разкъса цикълът.

```
bool Swapchain::present(uint32_t image_index)
{
    const auto& [cmd_buffer,
        in_flight_fence,
        image_available_semaphore,
        render_finished_semaphore] = frames_[current_frame_];

    if (!device->graphics_queue()->submit({
        .command_buffers = { cmd_buffer.get() },
        .wait_semaphores = { image_available_semaphore.get() },
        .wait_stages = { VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT },
        .signal_semaphores = { render_finished_semaphore.get() },
        .signal_fence = in_flight_fence.get()
    })) return false;

    const PresentResult result = device->present_queue()->present({
        .swapchains = { this },
        .image_indices = { image_index },
        .wait_semaphores = { render_finished_semaphore.get() }
    });

    if (result == PresentResult::OutOfDate ||
        result == PresentResult::Suboptimal)
    {
        should_recreate_ = true; return true;
    }

    return result == PresentResult::Success;
}
```

Фиг. 3.85 - Подаване и представяне на кадър:

submit към графична опашка и *present* към опашка за представяне

3.10.4.3 Динамични граници на рендериращия проход

Рендерирането се ограничава чрез vkCmdBeginRendering и vkCmdEndRendering, като преди и след това се правят явни преходи на разположението на изображението. Текущото разположение се кешира в image_layouts_, за да се извършват коректни преходи между кадрите.

```
cmd_buffer->pipeline_image_barrier(
    images_[image_idx],
    image_layouts_[image_idx],
    VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
    vk_pipeline_stage_2_top_of_pipe_bit,
    0,
    vk_pipeline_stage_2_color_attachment_output_bit,
```

```

    vk_access_2_color_attachment_write_bit
);

vkCmdBeginRendering(cmd_buffer->vk_command_buffer(), &rendering_info);

// ...

vkCmdEndRendering(cmd_buffer->vk_command_buffer());

cmd_buffer->pipeline_image_barrier(
    images_[image_idx],
    image_layouts_[image_idx],
    VK_IMAGE_LAYOUT_PRESENT_SRC_KHR,
    vk_pipeline_stage_2_color_attachment_output_bit,
    vk_access_2_color_attachment_write_bit,
    vk_pipeline_stage_2_bottom_of_pipe_bit,
    0
);

```

Фиг. 3.86 - Динамичен рендериращ проход и преходи на разположението на изображението

3.10.4.4 Пресъздаване на swapchain-а

Пресъздаването се активира чрез `should_recreate_` при `out-of-date/suboptimal`, при дълго блокиране в придобиването или при неподходящи размери (вкл. минимизиран прозорец). Реализацията изчаква графичната и опашката за представяне да приключат, освобождава зависимите ресурси (изгледи, дълбочина, синхронизация и командни буфери), след което създава нов `VkSwapchainKHR`, изгражда отново ресурсите и накрая унищожава стария `swapchain`.

```

bool Swapchain::recreate()
{
    if (desc_.window->is_minimized()) return true;

    should_recreate_ = false;

    desc_.device->graphics_queue()->wait_idle();
    desc_.device->present_queue()->wait_idle();

    const auto old_swapchain = vk_swapchain_;

    // destroy views + depth + sync + free cmd buffers ...
    // query support, create swapchain(old), recreate views/depth/cmd/sync

    if (old_swapchain)
        vkDestroySwapchainKHR(vk_device, old_swapchain, nullptr);

    current_frame_ = 0;
    return true;
}

```

Фиг. 3.87 - Пресъздаване на Swapchain: освобождаване на зависими ресурси и повторна инициализация (опростено)

3.10.5 Командни обекти

3.10.5.1 vk::CommandPool

vk::CommandPool обвива VkCommandPool и е обвързан с конкретно семейство опашки. При създаване той преобразува флаговете от RHI слоя към съответните флагове на Vulkan и създава pool за заделяне на командни буфери.

От него се заделят първични или вторични VkCommandBuffer обекти, които се асоциират с RHI инстанции на CommandBuffer. Той също централизира управлението на жизнения цикъл на командните буфери за дадено семейство опашки.

```
rhi::CommandBuffer* CommandPool::allocate_command_buffer(bool is_primary)
{
    const VkCommandBufferAllocateInfo alloc_info
    {
        .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO,
        .pNext = nullptr,
        .commandPool = vk_command_pool_,
        .level = is_primary ? VK_COMMAND_BUFFER_LEVEL_PRIMARY :
                             VK_COMMAND_BUFFER_LEVEL_SECONDARY,
        .commandBufferCount = 1,
    };

    VkCommandBuffer vk_cmd_buffer;
    vkAllocateCommandBuffers(vk_device, &alloc_info, &vk_cmd_buffer);
    cmd_buffer->set_vk_command_buffer(vk_cmd_buffer);
    return cmd_buffer;
}
```

Фиг. 3.88 - Заделяне на команден буфер от vk::CommandPool

3.10.5.2 vk::CommandBuffer

vk::CommandBuffer реализира конкретния запис на команди върху VkCommandBuffer. Методите begin, end и reset се превеждат директно към съответните функции на Vulkan, като флаговете за употреба се извеждат от RHI слоя.

Операциите за рисуване и изчисление се подават към vkCmdDraw, vkCmdDrawIndexed и vkCmdDispatch. Обвързването на конвейер и дескрипторни множества се реализира чрез vkCmdBindPipeline и vkCmdBindDescriptorSets.

Класът поддържа текущата точка на обвързване (графична или изчислителна), така че дескрипторните множества винаги да се свързват към правилния тип pipeline.

```
void CommandBuffer::bind_graphics_pipeline(rhi::GraphicsPipeline* pipeline)
{
    vkCmdBindPipeline(vk_command_buffer_,
```

```

        VK_PIPELINE_BIND_POINT_GRAPHICS, vk_pipeline->vk_pipeline());
    current_pipeline_bind_point_ = VK_PIPELINE_BIND_POINT_GRAPHICS;
}

void CommandBuffer::bind_compute_pipeline(rhi::ComputePipeline* pipeline)
{
    vkCmdBindPipeline(vk_command_buffer_,
        VK_PIPELINE_BIND_POINT_COMPUTE, vk_pipeline->vk_pipeline());
    current_pipeline_bind_point_ = VK_PIPELINE_BIND_POINT_COMPUTE;
}

```

Фиг. 3.89 - Проследяване на активната точка на обвързване

За синхронизация на ресурси `vk::CommandBuffer` използва явни бариери чрез `vkCmdPipelineBarrier2`, което позволява изрично описание на зависимостите и преходите на състоянията на изображения.

3.10.5.3 `vk::CommandQueue`

`vk::CommandQueue` обвива `VkQueue` и реализира подаването и представянето на команди. Структурите `SubmitInfo` и `PresentInfo` от RHI слоя се преобразуват към съответните Vulkan структури, като се изграждат масиви от командни буфери и семафори.

```

const VkSubmitInfo vk_submit_info
{
    .waitSemaphoreCount = vk_wait_semaphores.size(),
    .pWaitSemaphores = vk_wait_semaphores.data(),
    .pWaitDstStageMask = vk_wait_stages.data(),
    .commandBufferCount = vk_cmd_buffers.size(),
    .pCommandBuffers = vk_cmd_buffers.data(),
    .signalSemaphoreCount = vk_signal_semaphores.size(),
    .pSignalSemaphores = vk_signal_semaphores.data()
};

vkQueueSubmit(vk_queue_, 1, &vk_submit_info, vk_fence);

```

Фиг. 3.90 - Подаване на командни буфери към опашката

3.10.6 Vulkan реализация на обектите за синхронизация

3.10.6.1 `vk::Semaphore`

`vk::Semaphore` реализира както двоични, така и времеви семафори. При времеви семафор към структурата за създаване се добавя допълнителна информация за типа и началната стойност на брояча. Двоичният семафор използва стандартна конфигурация без разширения.

```

if (desc_.type == SemaphoreType::Timeline)

```



```

{
    VkSemaphoreCreateInfo type_info
    {
        .sType = VK_STRUCTURE_TYPE_SEMAPHORE_TYPE_CREATE_INFO,
        .semaphoreType = VK_SEMAPHORE_TYPE_TIMELINE,
        .initialValue = desc_.value
    };
    const VkSemaphoreCreateInfo create_info
    {
        .sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO,
        .pNext = &type_info
    };

    vkCreateSemaphore(vk_device, &create_info, nullptr, &vk_semaphore_);
}
else
{
    static constexpr VkSemaphoreCreateInfo create_info{ ... };
    vkCreateSemaphore(vk_device, &create_info, nullptr, &vk_semaphore_);
}

```

Фиг. 3.91 - Създаване на двоичен или времеви семафор

3.10.6.2 vk::Fence

vk::Fence реализира механизъм за синхронизация между CPU и GPU. Той обвива VkFence и предоставя операции за изчакване, нулиране и проверка на състоянието.

```

const VkFenceCreateInfo create_info
{
    .sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO,
    .flags = (desc_.signaled ? VK_FENCE_CREATE_SIGNALED_BIT : {})
};

vkCreateFence(vk_device, &create_info, nullptr, &vk_fence_);

```

Фиг. 3.92 - Създаване на Fence с начално състояние

Методът wait използва vkWaitForFences с подаден таймаут, а reset извиква vkResetFences. Методът is_signaled позволява неблокираща проверка на състоянието чрез vkGetFenceStatus.

3.10.7 Vulkan реализация на дескрипторните обекти

3.10.7.1 vk::DescriptorPool

DescriptorPool отговаря за заделянето и освобождаването на множества от дескриптори. При създаване той преобразува описанието от RHI слоя (максимален брой множества и брой дескриптори по тип) към съответните Vulkan структури и извиква vkCreateDescriptorPool.

```

const VkDescriptorPoolCreateInfo create_info
{
    .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO,
    .flags = VK_DESCRIPTOR_POOL_CREATE_FREE_DESCRIPTOR_SET_BIT,
    .maxSets = desc_.max_sets,
    .poolSizeCount = pool_sizes.size(),
    .pPoolSizes = pool_sizes.data()
};
vkCreateDescriptorPool(device, &create_info, nullptr, &vk_descriptor_pool_);

```

Фиг. 3.93 - Създаване на *VkDescriptorPool*

DescriptorPool-ът позволява освобождаване на отделни множества, което е необходимо при динамично създаване и унищожаване на материали. При нужда всички множества могат да бъдат нулирани наведнъж чрез *vkResetDescriptorPool*.

3.10.7.2 *vk::DescriptorSetLayout*

DescriptorSetLayout описва структурата на едно множество - кои обвързвания съществуват, какъв е техният тип и в кои шейдърни етапи се използват. При инициализацията се създава списък от *VkDescriptorSetLayoutBinding*, извлечен от описанието в *RHI* слой, след което се извиква *vkCreateDescriptorSetLayout*.

Този обект е неизменяем след създаването си и се използва при изграждането на конвейера и при заделяне на конкретни множества.

```

const VkDescriptorSetLayoutCreateInfo create_info
{
    .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO,
    .bindingCount = bindings.size(),
    .pBindings = bindings.data()
};
vkCreateDescriptorSetLayout(
    vk_device, &create_info, nullptr, &vk_descriptor_set_layout_)

```

Фиг. 3.94 - Създаване на описание на множество от дескриптори

3.10.7.3 *vk::DescriptorSet*

DescriptorSet представлява конкретно множество с вече обвързани ресурси. То се заделя от *DescriptorPool* въз основа на дадено *DescriptorSetLayout* чрез *vkAllocateDescriptorSets*.

```

const VkDescriptorSetAllocateInfo alloc_info
{
    .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO,
    .descriptorPool = vk_descriptor_pool_,
    .descriptorSetCount = count,

```

```

        .pSetLayouts = vk_layouts.data()
    };

    vkAllocateDescriptorSets(vk_device, &alloc_info, vk_descriptor_sets.data());

```

Фиг. 3.95 - Заделяне на множество от дескриптори

Методът `update()` приема списък от записи (`DescriptorWrite`) и ги преобразува до `VkWriteDescriptorSet`. В зависимост от типа ресурс (uniform буфер, буфер за съхранение, текстура със семплер или изображение за запис) се попълват съответните структури за буфери или изображения.

```

for (const auto& [binding, array_element, type, info] : writes)
{
    VkWriteDescriptorSet vk_write
    {
        .sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET,
        .dstSet = vk_descriptor_set_,
        .dstBinding = binding,
        .dstArrayElement = array_element,
        .descriptorCount = 1,
        .descriptorType = to_vk(type)
    };

    std::visit(
        [&<typename T>(T&& arg)
        {
            using decayed_t = std::decay_t<T>;
            if constexpr (std::same_as<decayed_t, UniformBuffer>)
            {
                buffer_infos.emplace_back(...);
                vk_write.pBufferInfo = &buffer_infos.back();
            }

            else if constexpr (std::same_as<decayed_t, StorageBuffer>)
            {
                buffer_infos.emplace_back(...);
                vk_write.pBufferInfo = &buffer_infos.back();
            }
            else if constexpr (std::same_as<decayed_t, CombinedImageSampler>)
            {
                image_infos.emplace_back(...);
                vk_write.pImageInfo = &image_infos.back();
            }
            else if constexpr (std::same_as<decayed_t, StorageImage>)
            {
                image_infos.emplace_back(...);
                vk_write.pImageInfo = &image_infos.back();
            }
        },
        info);

    vk_writes.push_back(vk_write);
}

```

```
vkUpdateDescriptorSets(  
    vk_device, vk_writes.size(), vk_writes.data(), 0, nullptr);
```

Фиг. 3.96 - Обновяване на множество от дескриптори

3.10.8 Vulkan реализация на ресурсните обекти

3.10.8.1 vk::Buffer

vk::Buffer представлява линейна област от графична памет, използвана за върхови, индексни, uniform или storage данни. При инициализацията се създава VkBuffer със съответните флагове за предназначение (vertex, index, uniform, storage и др.), чрез VMA се заделя подходящ тип памет (локална за устройството или достъпна от процесора). Буферът и заделената памет се свързват в единна структура.

```
const VkBufferCreateInfo buffer_create_info  
{  
    .sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO,  
    .size = desc_.size,  
    .usage = usage_flags,  
};  
  
VmaAllocationCreateInfo allocation_create_info{ ... };  
  
vmaCreateBuffer(allocator, &buffer_create_info, &allocation_create_info,  
    &buffer_, &allocation_, &allocation_info_);
```

Фиг. 3.97 - Създаване на VkBuffer с VMA

За достъп и пренос на данни:

- map() и unmap() позволяват директен достъп до паметта при host-видими буфери.
- upload() копира данни към буфера.
- read_back() извлича данни обратно към процесора.

При буфери, които са локални за устройството, качването се реализира чрез временен staging буфер и копиране с команда в команден буфер.

Освобождаването става чрез vmaDestroyBuffer, което едновременно премахва буфера и свързаната с него памет.

3.10.8.2 vk::Texture

vk::Texture представя изображение в графичната памет - 2D текстура, кубична текстура или друго многомерно изображение. При създаване се създава VkImage според типа, формата и предназначението, чрез VMA се заделя памет и се създава VkImageView, който определя начина, по който изображението ще се използва от шейдърите.

```

const VkImageCreateInfo image_create_info
{
    .sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO,
    .flags = is_cube ? VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT : {},
    .imageType = to_vk_image_type(desc_.type),
    .format = to_vk(desc_.format),
    .extent = { desc_.width, desc_.height, desc_.depth },
    .mipLevels = desc_.mip_levels,
    .arrayLayers = desc_.array_layers * (is_cube ? 6 : 1),
    .usage = to_vk(desc_.usage) | VK_IMAGE_USAGE_TRANSFER_DST_BIT,
};

vmaCreateImage(allocator, &image_create_info, &allocation_create_info,
               &image_, &allocation_, &allocation_info_);

const VkImageViewCreateInfo image_view_create_info{ ... };

vkCreateImageView(vk_device, &image_view_create_info, nullptr, &image_view_);

```

Фиг. 3.98 - Опростено създаване на *VkImage* и *VkImageView*

Vulkan изисква изображенията да преминават между различни състояния (например за запис, четене или представяне). `transition_layout()` извършва този преход чрез бариера в команден буфер.

Подобно на буферите, качването става чрез staging буфер и копиране с команда към изображението.

3.10.8.3 vk::Sampler

`vk::Sampler` определя начина, по който текстурите се семплират в шейдърите – филтриране, адресиране по координати, анизотропия и др. Създаването се свежда до попълване на `VkSamplerCreateInfo` според описанието от RHI слоя:

```

const VkSamplerCreateInfo sampler_create_info
{
    .sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO,
    .magFilter = to_vk(desc_.filter),
    .minFilter = to_vk(desc_.filter),
    .mipmapMode = get_mipmap_mode(desc_.mipmap_mode),
    .addressModeU = to_vk(desc_.wrap_u),
    .addressModeV = to_vk(desc_.wrap_v),
    .addressModeW = to_vk(desc_.wrap_w),
    .anisotropyEnable = anisotropy_enabled,
    .maxAnisotropy = max_anisotropy,
    .compareEnable = desc_.compare_enable,
    .compareOp = to_vk(desc_.compare_op)
};

vkCreateSampler(vk_device, &sampler_create_info, nullptr, &sampler_);

```

Фиг. 3.99 - Опростено създаване на *VkSampler*

vk::Sampler не съдържа собствена памет за данни - той представлява само описание на начина на четене от изображение. Унищожаването става чрез vkDestroySampler.

3.10.9 vk::ShaderModule

vk::ShaderModule реализира създаването и унищожаването на VkShaderModule. При инициализацията се подава SPIR-V байткод, който директно се предава на vkCreateShaderModule. Обектът съхранява получения VkShaderModule, който по-късно се използва при изграждане на графичен или изчислителен конвейер.

```
const std::vector<std::uint32_t> spv_data = read_file(spv_path);

const VkShaderModuleCreateInfo create_info
{
    .sType = VK_STRUCTURE_TYPE_SHADER_MODULE_CREATE_INFO,
    .codeSize = spv_data.size() * sizeof(uint32_t),
    .pCode = spv_data.data()
};

vkCreateShaderModule(vk_device, &create_info, nullptr, &vk_shader_module_);
```

Фиг. 3.100 - Създаване на VkShaderModule

3.10.10 Vulkan реализация на конвейерните обекти

3.10.10.1 vk::PipelineLayout

vk::PipelineLayout създава VkPipelineLayout на база подадените множества от дескриптори и диапазони за push-константи. При инициализацията се подготвят списъци от VkDescriptorSetLayout и VkPushConstantRange, след което се извиква vkCreatePipelineLayout.

```
const VkPipelineLayoutCreateInfo layout_info
{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO,
    .setLayoutCount = vk_set_layouts.size(),
    .pSetLayouts = vk_set_layouts.data(),
    .pushConstantRangeCount = push_constant_ranges.size(),
    .pPushConstantRanges = push_constant_ranges.data()
};

vkCreatePipelineLayout(
    device->logical_device(), &layout_info, nullptr, &vk_pipeline_layout_);
```

Фиг. 3.101 - Създаване на VkPipelineLayout

3.10.10.2 vk::GraphicsPipeline

vk::GraphicsPipeline създава графичен конвейер чрез vkCreateGraphicsPipelines. Подготвят се необходимите структури за етапите на шейдърите, входното описание на върховете, сглобяването на примитиви, растеризацията, мултисемплирането, тестовете за дълбочина и шаблон, както и състоянието на цветните изходи. Всички те се обединяват в една структура за създаване на конвейера.

```
const VkGraphicsPipelineCreateInfo pipeline_info
{
    .sType = VK_STRUCTURE_TYPE_GRAPHICS_PIPELINE_CREATE_INFO,
    .pNext = &rendering_info,
    .stageCount = shader_stages.size(),
    .pStages = shader_stages.data(),
    .pVertexInputState = &vertex_input_info,
    .pInputAssemblyState = &input_assembly,
    .pViewportState = &viewport_state,
    .pRasterizationState = &rasterizer,
    .pMultisampleState = &multisampling,
    .pDepthStencilState = &depth_stencil,
    .pColorBlendState = &color_blending,
    .pDynamicState = &dynamic_state,
    .layout = layout->vk_pipeline_layout(),
    .renderPass = nullptr,
    // Using dynamic rendering
    .subpass = 0,
    .basePipelineHandle = nullptr,
    .basePipelineIndex = -1
};

vkCreateGraphicsPipelines(device->logical_device(), nullptr, 1,
    &pipeline_info, nullptr, &vk_pipeline_);
```

Фиг. 3.102 - Създаване на графичен VkPipeline за dynamic rendering

3.10.10.3 vk::ComputePipeline

vk::ComputePipeline създава изчислителен конвейер чрез vkCreateComputePipelines. Необходима е единствено информация за шейдърния етап и използвания VkPipelineLayout.

```
const VkComputePipelineCreateInfo pipeline_info
{
    .sType = VK_STRUCTURE_TYPE_COMPUTE_PIPELINE_CREATE_INFO,
    .stage = shader_stage,
    .layout = layout->vk_pipeline_layout()
};

vkCreateComputePipelines(device->logical_device(), nullptr, 1,
    &pipeline_info, nullptr, &vk_pipeline_);
```

Фиг. 3.103 - Създаване на изчислителен VkPipeline

3.11 Модул boza.detail

boza.detail събира вътрешни помощни средства, които не са част от интерфейса на двигателя, но се използват във вътрешната имплементация на някои модули.

3.11.1 AssetPaths

AssetPaths централизира формирането на пътища към ресурсите. Вместо разпръснати низове из кода, всички графични обекти се изграждат спрямо една основна директория.

```
static fs::path material(const std::string& material_name)
{
    return materials_dir() / (material_name + ".mat.json");
}
```

Фиг. 3.104 - Формиране на път към материал

3.11.2 FileIO

FileIO осигурява централизирано четене и запис на файлове. Поддържа и зареждане на JSON файлове чрез библиотеката nlohmann_json посредством load_json.

```
static std::optional<json> load_json(const fs::path& path)
{
    const auto text = read_text(path);
    try { return json::parse(text); }
    catch (const json::parse_error&) { return std::nullopt; }
}
```

Фиг. 3.105 - Зареждане на JSON файл

3.11.3 ImageIO

ImageIO реализира вход и изход на изображения чрез stb_image и stb_image_write.

```
static ImageData read(const fs::path& path)
{
    int w, h, c;
    if (!stbi_info(path.string().c_str(), &w, &h, &c)) return {};
    std::uint8_t* data = stbi_load(
        path.string().c_str(), &w, &h, &c, desired_channels);

    return ImageData{ w, h, desired_channels, data, true };
}

static bool write(const fs::path& path, const ImageData& image_data)
{
    std::string ext = path.extension().string();
```



```

if (ext == ".png") return stbi_write_png(...) != 0;
if (ext == ".jpg" || ext == ".jpeg") return stbi_write_jpg(...) != 0;
if (ext == ".bmp") return stbi_write_bmp(...) != 0;
if (ext == ".tga") return stbi_write_tga(...) != 0;

return false;
}

```

Фиг. 3.106 - Четене и писане на изображение

3.12 Инструмент `shader_processor`

3.12.1 Роля на инструмента

Инструментът за обработка на шейдъри представлява отделен изпълним модул, който подготвя графичните програми за използване в двигателя. Неговата основна цел е да премести сложната логика за компилация, анализ и оптимизация извън изпълнимото приложение, така че по време на работа да се зареждат готови двоични файлове и метаданни.

3.12.2 IncludeResolver

Първата съществена стъпка е обединяването на всички изходни файлове в единен текстов поток. Това се реализира чрез рекурсивно обхождане на директивите за включване. Всеки файл се чете, анализира ред по ред и при срещане на директива за включване се извиква същата функция за съответния файл.

```

std::string IncludeResolver::resolve(const fs::path& entry)
{
    std::unordered_set<std::string> visited;
    return resolve_recursive(entry, visited);
}

std::string IncludeResolver::resolve_recursive(
    const fs::path& file,
    std::unordered_set<std::string>& visited)
{
    if (visited.contains(file)) return "";
    visited.insert(file);

    const std::string raw = FileIO::read(file);
    static const std::regex include_re(...); #include "..."

    size_t start = 0;
    while (start < raw.size())
    {
        if (std::cmatch m; ...)
        {
            fs::path inc_path = ...;
            processed_source += resolve_recursive(inc_path, visited);
        }
        else
        {
            ...
        }
    }
}

```

```

        processed_source.append(line);
        processed_source.push_back('\n');
    }
    return processed_source;
}

```

Фиг. 3.107 - Опростена логика за включване на файлове

Проследяването на visited предотвратява повторно включване на един и същи файл, както и безкрайни цикли. Така се гарантира, че крайният резултат представлява напълно разширен текст без външни зависимости. Това улеснява компилатора и прави грешките по-лесни за локализиране.

3.12.3 ShaderCompiler

ShaderCompiler отговаря за превръщането на GLSL изходен код в SPIR-V байткод посредством библиотеката shaderc. Видът на шейдъра се определя автоматично от разширението на входния файл - .vert, .frag, .comp и т.н. След компилацията резултатът се проверява за валидност чрез SPIRV-Tools, преди да бъде върнат. Нивото на оптимизация е нулево и се генерира debug info, за да може при рефлексията резултатът от тази стъпка да съдържа всичката нужна информация.

```

std::vector<std::uint32_t> ShaderCompiler::compile_to_spirv(
    const std::string& source,
    const shaderc_shader_kind kind,
    const std::string& filename)
{
    const shaderc::Compiler compiler;
    shaderc::CompileOptions options;
    spvtools::SpirvTools validator{ SPV_ENV_VULKAN_1_3 };

    options.SetGenerateDebugInfo();
    options.SetOptimizationLevel(shaderc_optimization_level_zero);

    const shaderc::SpvCompilationResult module =
        compiler.CompileGlslToSpv(source, kind, filename.c_str(), options);

    std::vector<std::uint32_t> spirv{ module.cbegin(), module.cend() };
    if (!validator.Validate(spirv.data(), spirv.size())) return {};

    return std::move(spirv);
}

```

Фиг. 3.108 - Компилиране на GLSL до SPIR-V и проверка за валидност

3.12.4 SpirvOptimizer

SpirvOptimizer приема неоптимизиран SPIR-V байткод и прилага серия от проходи за подобряване на производителността чрез SPIRV-Tools. Освен това се премахва ненужната информация като рефлексивните анотации, тъй като те вече не са нужни след приключване на рефлексията. При неуспех се връща оригиналният байткод, за да не се прекъсне обработката.

```
auto SpirvOptimizer::optimize(const std::vector<std::uint32_t>& spirv)
{
    spvtools::Optimizer optimizer{ SPV_ENV_VULKAN_1_3 };

    optimizer.RegisterPass(spvtools::CreateStripDebugInfoPass());
    optimizer.RegisterPass(spvtools::CreateStripReflectInfoPass());
    optimizer.RegisterPerformancePasses();

    std::vector<std::uint32_t> optimized_spirv;
    if (!optimizer.Run(spirv.data(), spirv.size(), &optimized_spirv))
        return spirv;

    return optimized_spirv;
}
```

Фиг. 3.109 - Оптимизиране на SPIR-V байткод

3.12.5 ShaderDecompiler

ShaderDecompiler преобразува оптимизиран SPIR-V байткод обратно към езици на високо ниво чрез библиотеката SPIRV-Cross. Поддържа три целеви формата - GLSL версия 450 за OpenGL, HLSL модел 5.0 за DirectX и MSL за Metal. Тъй като Boza няма други имплементации на RHI освен тази на Vulkan, получените HLSL и MSL шейдъри не се компилират до нужните формати, за да бъдат използвани. При бъдещо добавяне на имплементации на RHI, тази стъпка ще бъде реализирана.

```
auto ShaderDecompiler::decompile_to_glsl( std::vector<std::uint32_t>& spirv)
{
    spirv_cross::CompilerGLSL glsl_compiler(spirv);
    spirv_cross::CompilerGLSL::Options options;
    options.version = 450;
    options.es      = false;
    glsl_compiler.set_common_options(options);
    return glsl_compiler.compile();
}
```

Фиг. 3.110 - Декомпилиране до GLSL за OpenGL

3.12.6 ShaderReflector

ShaderReflector анализира неоптимизирания SPIR-V байткод чрез SPIRV-Cross и извлича структурирани метаданни в JSON формат. Рефлексията покрива всички видове ресурси - uniform буфери, буфери за съхранение, входно-изходни променливи на етапа, семплирани и съхранявани изображения, push константи, както и размерите на работната група при изчислителни шейдъри.

```
json ShaderReflector::generate_metadata(  
    const std::vector<std::uint32_t>& spirv, const std::string& shader_type)  
{  
    json metadata;  
    metadata["shader_type"] = shader_type;  
  
    json struct_types = json::object();  
    std::unordered_set<std::uint32_t> reflected_struct_ids{};  
  
    spirv_cross::Compiler c{ spirv };  
    spirv_cross::ShaderResources resources{c.get_shader_resources()};  
  
    reflect_resources(c, resources.uniform_buffers, "uniform_buffers", ...);  
    reflect_resources(c, resources.storage_buffers, "storage_buffers", ...);  
    reflect_resources(c, resources.stage_inputs, "stage_inputs", ...);  
    reflect_resources(c, resources.stage_outputs, "stage_outputs", ...);  
    reflect_resources(c, resources.sampled_images, "sampled_images", ...);  
    reflect_resources(c, resources.storage_images, "storage_images", ...);  
    reflect_push_constants(c, resources, metadata, shader_type, ...);  
  
    if (shader_type == "compute")  
        reflect_compute_work_group_size(compiler, metadata);  
  
    return metadata;  
}
```

Фиг. 3.111 - Генериране на метаданни от SPIR-V (опростено)

За всеки ресурс се записват името, типът, наборът от дескриптори и свързването му. При буфери допълнително се вписват минималният им размер, описанието на членовете им с отместванията им и дали съдържат масиви с размер, определен по време на изпълнение. За буфери за съхранение се записва и режимът на достъп - само четене, само запис или четене и запис. При изображения се записват форматът и размерите на вектора. Всички структурни типове, открити по време на обхождането, се събират в отделен речник, за да се избегне тяхното дублиране в JSON изхода.

Push константите се обработват по подобен начин, но вместо набор от дескриптори и свързване се записва отместването им спрямо шейдърния етап.

3.12.7 ShaderProcessor

ShaderProcessor е главният координатор, който свързва всички останали класове в единен конвейер за обработка на един шейдърен файл. Приема път до входния файл, изходна директория и конфигурация, и изпълнява стъпките последователно.

```
bool ShaderProcessor::process(...)
{
    const std::string full_source = IncludeResolver::resolve(path);
    auto kind = ShaderCompiler::get_shader_kind(path.extension().string());

    auto spirv_unoptimized = ShaderCompiler::compile_to_spirv(
        full_source, kind, path.filename().string());

    const json metadata = ShaderReflector::generate_metadata(
        spirv_unoptimized, ShaderCompiler::get_shader_kind_string(kind));

    FileIO::write(metadata_path, metadata.dump(2));

    auto spirv_optimized = SpirvOptimizer::optimize(spirv_unoptimized);

    FileIO::write(spv_path, spirv_optimized);

    if (config.enable_opengl) { ... }
    if (config.enable_directx) { ... }
    if (config.enable_metal) { ... }

    return true;
}
```

Фиг. 3.112 - Пълният конвейер за обработка на шейдър (опростен)

3.13 CMake конфигурация

3.13.1 Главен CMakeLists.txt

Коренният конфигурационен файл задава основните параметри на проекта. Стандартът е C++23 с активиран `import std`, реализиран чрез експерименталния флаг `CMAKE_EXPERIMENTAL_CXX_IMPORT_STD`, чиято стойност зависи от конкретната версия на CMake поради разликите в идентификаторите на различните версии.

```
cmake_minimum_required(VERSION 3.30...4.1 FATAL_ERROR)

set(CMAKE_CXX_STANDARD 23)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_MODULE_STD 1)

set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD "...")

# ...
```

Фиг. 3.113 - Настройка на C++23 с поддръжка за import std

След декларацията на проекта се зареждат всички вътрешни CMake модули и се добавят поддиректориите - инструментът за шейдъри, директорията с шейдъри, двигателят и примерните приложения.

3.13.2 BozaOptions

Централизира всички превключватели - графични бекенди (Vulkan, OpenGL, DirectX 11/12, Metal) и диагностични инструменти. По подразбиране е активиран само Vulkan. При конфигурация автоматично се деактивират бекенди, несъвместими с платформата - DirectX при не-Windows, Metal при не-Apple. Всеки активен бекенд генерира съответна дефиниция на препроцесора чрез генераторни изрази.

```
add_compile_definitions(  
    $$<CONFIG:Debug>:BOZA_DEBUG  
    $$<CONFIG:Release>:BOZA_RELEASE  
  
    $$<BOOL:${BOZA_ENABLE_OPENGL}>:BOZA_OPENGL_ENABLED  
    $$<BOOL:${BOZA_ENABLE_VULKAN}>:BOZA_VULKAN_ENABLED  
    # ...  
)
```

Фиг. 3.114 - Условни дефиниции според активираните бекенди

Ако е активиран mimalloc, се създава обектна библиотека mimalloc_runtime с единствен файл, включващ <mimalloc-new-delete.h>, което пренасочва глобалните оператори new и delete за всяка цел, свързана с нея.

3.13.3 BozaCompilerWarnings

Функцията boza_enable_warnings() прилага строги предупреждения, третиращи като грешки, към конкретна цел. Наборът се различава според компилатора.

```
function(boza_enable_warnings target)  
    if (MSVC)  
        target_compile_options(${target} PRIVATE /W4 /WX /permissive- ...)  
    else ()  
        target_compile_options(${target} PRIVATE -Wall -Wextra -Werror ...)  
    endif ()  
endfunction()
```

Фиг. 3.115 - Прилагане на предупреждения към target

3.13.4 BozaUtils

Съдържа три помощни функции:

- `boza_setup_module_target()` регистрира `.ixx` и `.cppm` файлове към цел чрез `FILE_SET CXX_MODULES`.
- `boza_generate_impl()` записва подаден низ като `.cpp` файл в директорията за изграждане - използва се за активиране на еднофайлови библиотеки като STB и VMA.
- `boza_snake_to_pascal()` преобразува имена от стил `my_app` към `MyApp` и се ползва при автоматичното генериране на `main.cpp`.

```
function(boza_generate_impl target header_content out_file)
  set(impl_file "${CMAKE_CURRENT_BINARY_DIR}/${out_file}")
  file(WRITE "${impl_file}" "${header_content}")
  target_sources(${target} PRIVATE "${impl_file}")
endfunction()
```

Фиг. 3.116 - Генериране на имплементационен файл (обикновено за библиотеки)

3.13.5 BozaHeaderUnits

Реализира система за предварително компилиране на заглавни файлове на външни библиотеки като C++ header units, тъй като те не могат директно да се импортират като модули. Поддържа се само MSVC. Трите публични функции образуват единен работен процес - `add_header_unit_library()` създава именувана колекция, `target_add_header_unit()` компилира конкретен заглавен файл до `.ifc` и `.obj` чрез `/exportHeader`, а `target_link_header_units()` свързва цялата колекция с дадена цел чрез `/headerUnit`.

```
# Компилиране
add_custom_command(
  OUTPUT "${out_file}" "${obj_file}"
  COMMAND ${CMAKE_CXX_COMPILER}
  /nologo /c /EHsc /std:c++latest ${runtime_flag}
  "/external:I${header_include_dir}" /external:W0
  ${compile_flags}
  /ifcOutput "${out_file}"
  "/Fo${obj_file}"
  /exportHeader "${header_abs_path}"
  DEPENDS "${header_abs_path}"
  VERBATIM
)

# Свързване с target
target_compile_options(${target} PRIVATE
  "/headerUnit${header_path}=${ifc_path}")
target_sources(${target} PRIVATE "${obj_file}")
```

Фиг. 3.117 - Компилиране и свързване на header unit при MSVC

3.13.6 BozaHeaderUnitPatches

Някои заглавни файлове от `vcpkg` съдържат `static` декларации, несъвместими с компилирането им като `header units`. `boza_enable_vcpkg_header_fixes()` генерира CMake скрипт, изпълняван като `custom command`, който директно редактира засегнатите файлове на `flecs` и `gtl` - премахва `static` от определени декларации. Корекцията се отбелязва чрез `stamp` файл и не се повтаря при последващи конфигурации.

3.13.7 BozaAssets и BozaExecutable

`boza_link_assets()` създава връзка от директорията с `asset`-и в изходния код към директорията за изпълнение - `junction` при Windows и символна връзка при Linux/macOS.

`boza_add_executable()` автоматизира създаването на приложения върху двигателя. Открива модулните и изходните файлове, генерира `main.cpp` с инстанциране на класа на приложението (чието име се извежда от името на целта), свързва двигателя, `header units`, `mimalloc` (ако е активиран) и добавя зависимост към `compile_shaders`.

```
function(boza_add_executable target)
    boza_snake_to_pascal(app_class "${target}")
    boza_generate_impl(${target}
        "import ${target};
        int main()
        {
            ${app_class} app;
            if (!app.init()) return -1;
            app.run();
        }"
        "main.cpp")
    target_link_libraries(${target} PRIVATE Boza::Engine)
    target_link_header_units(${target} boza_engine_header_units)
    add_dependencies(${target} compile_shaders)
    boza_link_assets(${target})
endfunction()
```

Фиг. 3.118 - Конфигурация на приложения на двигателя (опростено)

3.13.8 CMakeLists.txt на Boza

Двигателят се изгражда като статична библиотека `boza_engine`. Задължителните зависимости са `flecs`, `spdlog`, `glfw` и `glm`. При активиран Vulkan бекенд допълнително се зареждат `VMA` и `volk`, като за тях се генерира обединен заглавен файл `vk_all`.

```
file(WRITE ${CMAKE_CURRENT_BINARY_DIR}/include/vk_all [[
    #include <vk_mem_alloc.h>
```



```

    #include <volk.h>
  })

  set(VK_DEFS
    VK_NO_PROTOTYPES
    VMA_STATIC_VULKAN_FUNCTIONS=0
    VMA_DYNAMIC_VULKAN_FUNCTIONS=1
  )

  target_add_header_unit(boza_engine_header_units
    ${CMAKE_CURRENT_BINARY_DIR}/include vk_all
    COMPILE_DEFINITIONS ${VK_DEFS}
    INCLUDE_DIRECTORIES ${include_dir}
  )

```

Фиг. 3.119 - Обединен Vulkan header unit

3.13.9 CMakeLists.txt на shader_processor

Инструментът се декларира като изпълним файл с псевдоним Tools::ShaderProcessor, под който се използва от останалата конфигурация. Зависимостите му са shaderc, SPIRV-Tools, SPIRV-Tools-opt, nlohmann_json и всички компоненти на spirv-cross. Съответните заглавни файлове се регистрират като header units по вече описания начин.

```

add_executable(shader_processor)
add_executable(Tools::ShaderProcessor ALIAS shader_processor)

target_link_libraries(shader_processor PRIVATE
  unofficial::shaderc::shaderc
  SPIRV-Tools-static
  SPIRV-Tools-opt
  nlohmann_json::nlohmann_json

  spirv-cross-core spirv-cross-glsl ...)

```

Фиг. 3.120 - Деклариране на shader_processor с псевдоним

3.13.10 CMakeLists.txt на шейдърите

Открива автоматично всички шейдърни файлове и за всеки генерира add_custom_command, извикващ Tools::ShaderProcessor. Аргументите за изключване на бекенди се добавят динамично. Очакваните изходни файлове се декларират изрично, за да може CMake да проследява зависимостите и да прекомпилира само при реална промяна. Целта compile_shaders агрегира всичко.

```

foreach (shader_path ${SHADER_FILES})
  add_custom_command(
    OUTPUT ${outputs}

```

```

        COMMAND ${CMAKE_COMMAND} -E make_directory ${out_dir}
        COMMAND $<TARGET_FILE:Tools::ShaderProcessor>
            ${shader_path}
            --out ${out_dir}
            ${format_args}
        DEPENDS Tools::ShaderProcessor ${shader_path}
        COMMENT "Compiling shader ${shader_name}"
        VERBATIM
    )
endforeach ()

add_custom_target(compile_shaders ALL DEPENDS ${shader_outputs})

```

Фиг. 3.121 - Автоматично компилиране на шейдъри по време на изграждане

3.14 Примерни приложения

Двете примерни приложения наследяват App, имплементират setup() и конфигурират всичко - сцени, обекти, компоненти, материали и вход, изцяло чрез публичния интерфейс на двигателя.

И двете приложения имат идентични системи CameraSystem и FPSLogger. CameraSystem улавя вход от мишката и клавиатурата и актуализира позицията и ротацията на обекти с CameraController. FPSLogger извежда всяка секунда диагностични съобщения за броя актуализации, направени през нея.

3.14.1 MaterialShowcase

Приложението демонстрира едновременно няколко механизма: генериране на текстура с изчислителен шейдър, програмно създаване на материали, управление на множество сцени и интерактивна смяна на меш и материал чрез клавиатура.

3.14.1.1 Генериране на текстура с compute шейдър

Преди всичко друго се генерира 256×256 RGBA8 текстура на GPU. Тексурата се създава с флагове Storage | Sampled | TransferDst - Storage е необходим, за да може изчислителният шейдър да записва в нея, а Sampled - за последващо четене от fragment шейдъра. Изпращането към GPU се извършва чрез ComputeDispatcher: той получава името на шейдъра, обвързва ресурсите със .set(), изпраща командата за изпълнение и блокира до завършване с .wait(). Накрая тексурата се прехвърля в оформление ShaderReadOnly. При грешка тя се унищожава и изпълнението продължава без нея.

```

static void generate_compute_texture()
{
    constexpr std::uint32_t tex_size = 256;
    const Texture& compute_texture_ = Texture::create(

```

```

    "compute_output",
    {
        .type = TextureType::Texture2D,
        .format = TextureFormat::RGBA8,
        .access_mode = ResourceAccessMode::Static,
        .width = tex_size,
        .height = tex_size,
        .usage_flags = TextureUsage::Sampled |
                     TextureUsage::Storage | TextureUsage::TransferDst,
    });

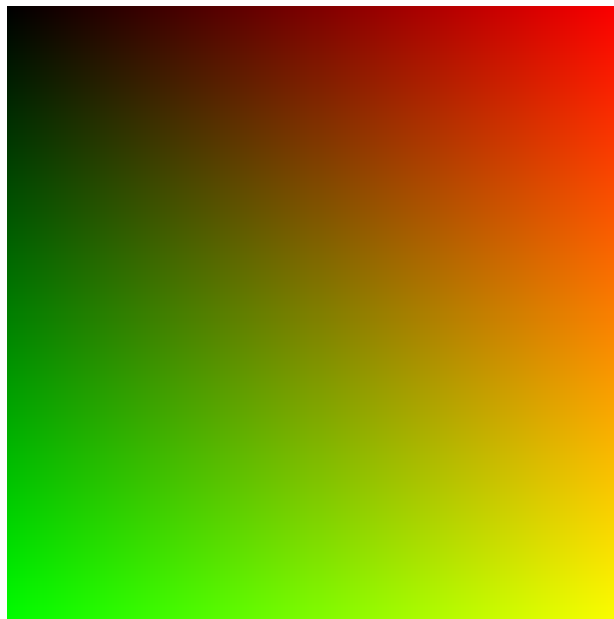
    bool failed = false;

    ComputeDispatcher dispatcher{ "uv", failed };
    dispatcher
        .set("outImage", compute_texture_)
        .dispatch(tex_size, tex_size, 1)
        .wait();

    compute_texture_.transition_layout(TextureLayout::General,
                                       TextureLayout::ShaderReadOnly);
}

```

Фиг. 3.122 - Генериране на UV текстура с ComputeDispatcher



Фиг. 3.123 - UV текстура, генерирана с ComputeDispatcher

3.14.1.2 Програмно създаване на материали

Два материала се дефинират изцяло в код чрез `Material::create()` - "custom_compute" и "pulsing". Останалите материали се зареждат автоматично от .mat.json файлове. Използва се операторът `[]` за задаване на стойности за uniform буферите и combined image-sampler обектът.

```

static void create_materials()
{
    {
        Material& mat = Material::create("custom_compute", {
            .vertex_shader = "default",
            .fragment_shader = "default"
        });

        mat["albedo_map"] = {
            Texture::get("compute_output"),
            Sampler::get("default")
        };
        mat["material.albedo_color"] = glm::vec4{ 1.0f };
        mat["material.properties"] = glm::vec4{ 0.0f, 0.8f, 0.0f, 0.0f };
    }

    {
        Material& mat = Material::create("pulsing", {
            .vertex_shader = "default",
            .fragment_shader = "default"
        });

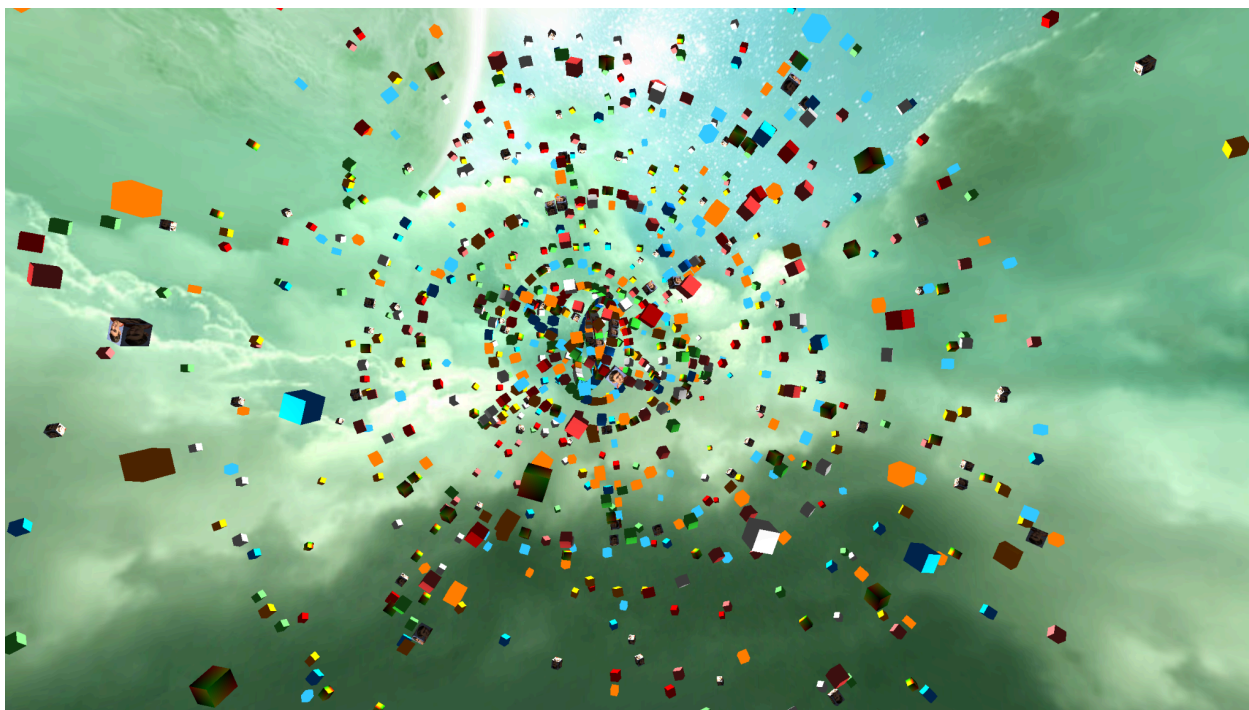
        mat["albedo_map"] = {
            Texture::get_or_load("default.png"),
            Sampler::get("default")
        };
        mat["material.albedo_color"] = glm::vec4{ 1.0f, 0.0f, 0.0f, 1.0f };
        mat["material.properties"] = glm::vec4{ 0.0f, 0.8f, 0.2f, 0.0f };
    }
}

```

Фиг. 3.124 - Създаване на материали по време на изпълнение на програмата

3.14.1.3 Управление на сцени

Приложението има две дефинирани от потребителя сцени - SceneA и SceneB. В постоянната сцена (Scene::persistent) се съдържат камерата и skybox обекта. SceneA е активна по подразбиране и съдържа йерархия от въртящи се кубове. SceneB се създава неактивна и съдържа единичния обект за демонстрация. При натискане на Enter активността на двете сцени се разменя - обектите не се унищожават, а просто спират да се обработват и изобразяват.



Фиг. 3.125 - SceneA

В SceneA родителски обект с компонент Rotator и MeshRenderer има 32×32 дъщерни обекта с компонент Orbiter, дефиниращ ос, радиус и начална фаза. Материалът на всеки куб се избира произволно от пул от 10 материала чрез `Random::pick()`.

Обектът в SceneB носи InputCapture, обвързващ клавишите M и N към функции, които сменят материала и mesh-a на обекта.

3.14.1.4 Системи

- RotatorSystem е проста системна фаза от тип UpdateStage, която обхожда всички обекти с Transform и Rotator и прилага завъртане на всеки кадър върху локалната ориентация на обекта, умножено по изминалото време.
- OrbiterSystem има две фази. При Start изчислява началната локална позиция на обекта върху орбитата и го ориентира с поглед към родителя. При Update напредва ъгълът на обиколка, позицията се преизчислява аналитично - без натрупване на грешки, и ориентацията се актуализира така, че обектът винаги да гледа към родителя си.
- ColorPulserSystem е система без заявка за обекти - тя работи глобално и пряко манипулира материала "pulsing". На всяка секунда системата избира произволен нов целеви цвят, а предишният target цвят става начален. В рамките на секундата

цветовете се преливат линейно и резултатът се записва директно в `material.albedo_color` на материала.

```
struct ColorPulserSystem : UpdateStage<ColorPulserSystem>
{
    static inline float cooldown{ 0.0f };
    static inline float speed{ 1.0f };
    static inline glm::vec3 color_a{ 0.0f };
    static inline glm::vec3 color_b{ 1.0f };
    static inline Material* material{ nullptr };

    static void execute()
    {
        if (!material)
        {
            material = Material::try_get("pulsing");
            if (!material) return;
        }

        cooldown += Time::delta_time() * speed;
        if (cooldown > 1.0f)
        {
            cooldown = 0.0f;
            color_a = color_b;
            color_b = glm::vec3{
                Random::number<float>(),
                Random::number<float>(),
                Random::number<float>()
            };
        }

        const glm::vec3 color = mix(color_a, color_b, cooldown);
        (*material)["material.albedo_color"] = glm::vec4{ color, 1.0f };
    }
};
```

Фиг. 3.126 - ColorPulserSystem

3.14.2 InstancingExample

Приложението демонстрира генериране на терен и тревна покривка чрез изчислителни шейдъри и автоматично GPU инстанциране на голям брой обекти.

3.14.2.1 Генериране на терен

Теренът се изгражда изцяло на видеокартата чрез два последователни изчислителни dispatch-a. Първият (`terrain_gen`) изчислява позициите на върховете посредством FBM шум и записва временни нормали, индекси и изображение с карта на височините. Вторият (`terrain_normals`) прочита позициите на съседните върхове и преизчислява точните нормали чрез кръстосано произведение. Картата на височините след това се прочита обратно в CPU

паметта чрез `Texture::read_back()` - за позициониране на тревните стръкове - и веднага се унищожава.

```
export bool create_terrain_mesh()
{
    const Buffer vertices
    {
        vertex_count * sizeof(Vertex),
        BufferUsage::Storage
    };

    const Buffer indices
    {
        index_count * sizeof(std::uint32_t),
        BufferUsage::Storage
    };

    const Texture& height_map = Texture::create("height_map",
    {
        .type          = TextureType::Texture2D,
        .format         = TextureFormat::R8,
        .access_mode    = ResourceAccessMode::Static,
        .width          = verts_per_axis,
        .height         = verts_per_axis,
        .usage_flags    = TextureUsage::Sampled |
                        TextureUsage::Storage |
                        TextureUsage::TransferDst |
                        TextureUsage::TransferSrc
    });

    static std::uint32_t seed = Random::number(
        std::numeric_limits<std::uint32_t>::max());

    bool failed;
    ComputeDispatcher{ "terrain_gen", failed }
        .set("vertices", vertices)
        .set("indices", indices)
        .set("height_map", height_map)
        .set("pc.width", verts_per_axis)
        .set("pc.height", verts_per_axis)
        .set("pc.seed", seed)
        .set("pc.offset", glm::vec2{ 0.0f, 0.0f })
        .set("pc.spacing", terrain_extent / terrain_cells)
        .set("pc.num_layers", 7u)
        .set("pc.base_freq", 0.015f)
        .set("pc.lacunarity", 2.0f)
        .set("pc.persistence", 0.45f)
        .set("pc.height_scale", height_scale)
        .set("pc.domain_warp", 6.0f)
        .dispatch(verts_per_axis, verts_per_axis)
        .wait();

    ComputeDispatcher{ "terrain_normals", failed }
        .set("vertices", vertices)
        .set("pc.width", verts_per_axis)
        .set("pc.height", verts_per_axis)
        .dispatch(verts_per_axis, verts_per_axis)
        .wait();

    std::vector<Vertex> vertices_data(vertex_count);
```

```

std::vector<std::uint32_t> indices_data(index_count);
vertices.read_back(vertices_data.data(), vertices.size());
indices.read_back(indices_data.data(), indices.size());
Mesh::create("terrain", vertices_data, indices_data);

return true;
}

```

Фиг. 3.127 - Генериране на height map и Mesh за терена

3.14.2.2 Поставяне на тревата

Позициите на тревните стръкове се определят като се семплира между четирите най-близки пиксели на текстурата според произволни позиции. Това, че всеки стрък трева е с един и същ Mesh и Material, позволява инстанциране.

```

float sample_terrain_height(
    std::span<std::uint8_t> terrain_height_map, float x, float z)
{
    constexpr float half_extent = terrain_extent * 0.5f;
    constexpr float spacing      = terrain_extent /
static_cast<float>(terrain_cells);

    float gx = (x + half_extent) / spacing;
    float gz = (z + half_extent) / spacing;

    gx = glm::clamp(gx, 0.0f, static_cast<float>(terrain_cells));
    gz = glm::clamp(gz, 0.0f, static_cast<float>(terrain_cells));

    const std::uint32_t x0 = glm::floor(gx);
    const std::uint32_t z0 = glm::floor(gz);
    const std::uint32_t x1 = glm::min(x0 + 1u, terrain_cells);
    const std::uint32_t z1 = glm::min(z0 + 1u, terrain_cells);

    const float tx = gx - x0;
    const float tz = gz - z0;

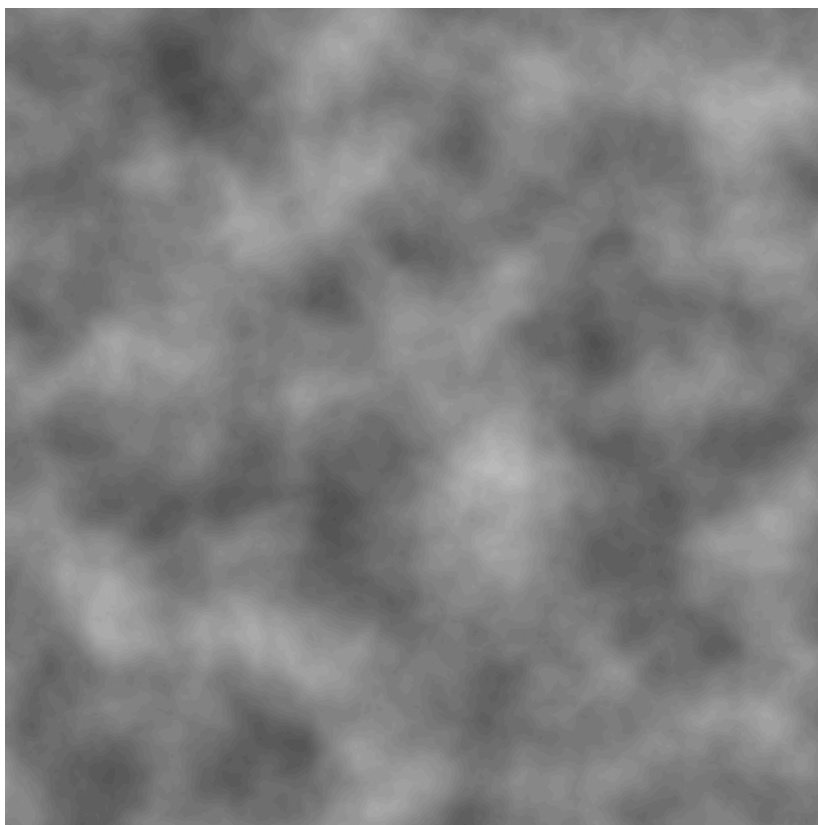
    auto idx = [&](const std::uint32_t xi, const std::uint32_t zi)
    { return zi * verts_per_axis + xi; };
    auto fix_scaling = [](const float v)
    { return height_scale * (v / 127.5f - 1.0f); };

    const float h00 = fix_scaling(terrain_height_map[idx(x0, z0)]);
    const float h10 = fix_scaling(terrain_height_map[idx(x1, z0)]);
    const float h01 = fix_scaling(terrain_height_map[idx(x0, z1)]);
    const float h11 = fix_scaling(terrain_height_map[idx(x1, z1)]);

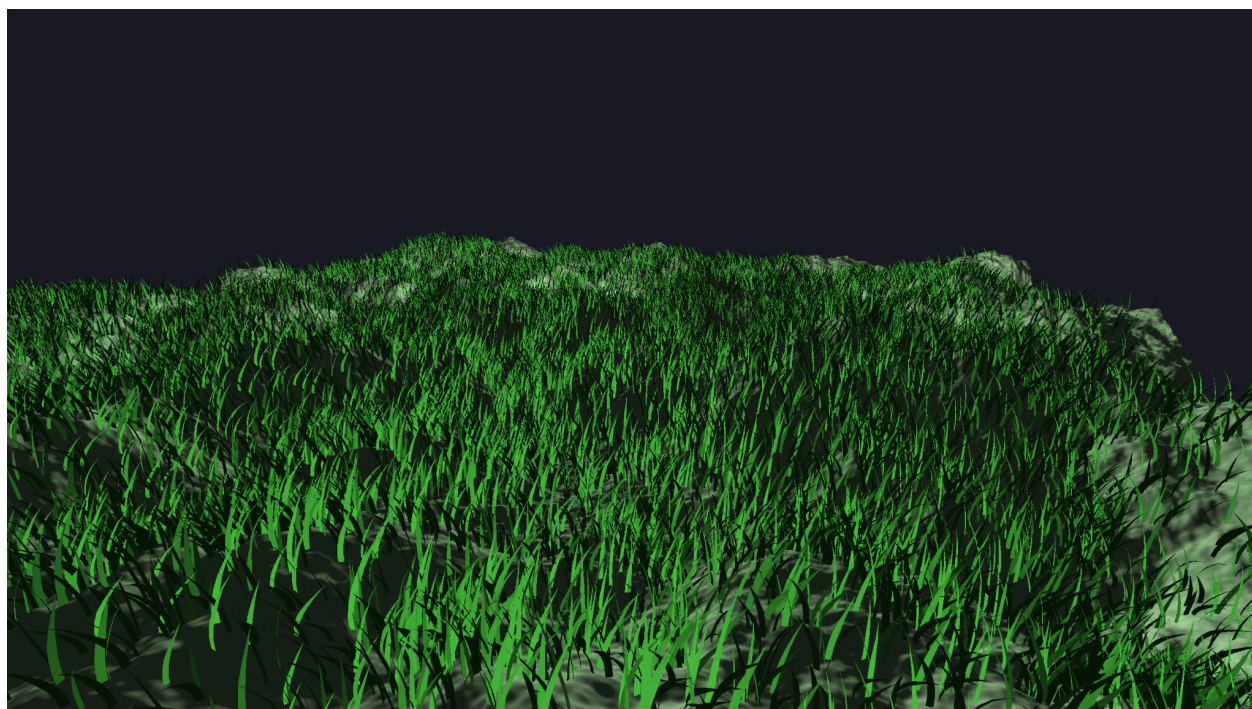
    const float hx0 = glm::mix(h00, h10, tx);
    const float hx1 = glm::mix(h01, h11, tx);
    return glm::mix(hx0, hx1, tz);
}

```

Фиг. 3.128 - Семплиране в текстурата за избор на височина за тревата



Фиг. 3.129 - Heightmap на терена



Фиг. 3.130 - InstancingExample

ГЛАВА 4. РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ

4.1 Компилиране и стартиране

Boza използва CMake като система за изграждане и изисква MSVC (Visual Studio 2022 или по-нова версия) поради зрялата поддръжка на C++ модули в тази среда. Преди компилиране трябва да са инсталирани:

- Visual Studio 2022 с компонент „Desktop development with C++“
- CMake 3.30 или по-нова версия
- vcpkg (интегриран в средата чрез VCPKG_ROOT)
- Vulkan SDK, ако ще се изгражда с Debug конфигурация

4.1.1 Компилиране и стартиране от терминал

Проектът разполага с предварително конфигурирани CMake Presets за Debug и Release, което опростява процеса до три команди. Препоръчително е да се използва Developer PowerShell или Developer Command Prompt, предоставен от Visual Studio, тъй като те автоматично настройват необходимите пътища за MSVC инструментите.

```
cmake --preset Debug
cmake --build --preset Debug
cd .\build\Debug\bin
.\my_game.exe
```

Фиг. 4.1 - Конфигуриране, компилиране и стартиране с Debug конфигурация.

Командите са аналогични за Release (замяна на „Debug“ с „Release“)

Командата `cmake --preset Debug` генерира build директорията и конфигурира всички зависимости чрез `vcpkg`. `cmake --build --preset Debug` компилира двигателя, инструмента за шейдъри и всички регистрирани примерни програми. При успешно изграждане изпълнимите файлове се намират в `build/Debug/bin`.

4.1.2 Компилиране и стартиране през VisualStudio 2022

Visual Studio поддържа директно отваряне на CMake проекти без предварително генериране на solution файл. Стъпките са:

1. File → Open → CMake... и избор на кореновия CMakeLists.txt.
2. Visual Studio автоматично засича preset-ите и предлага избор на конфигурация (Debug / Release) от падащото меню в лентата с инструменти.

3. Изчакване на завършването на генерирането на проекта - статусът е видим в прозореца „Output“.
4. Build → Build All (или Ctrl+Shift+B).
5. Избор на желаната цел за изпълнение от падащото меню вдясно от бутона за стартиране и натискане на „Run“.

4.2 Създаване на програма

Boza скрива boilerplate кода за инициализация. Потребителят не пише ръчно функцията `main()`, нито директно извиква `App::init()` и `App::run()` - тази логика се генерира автоматично от CMake помощната функция `boza_add_executable`. Необходимо е единствено да се дефинира клас, наследяващ `App`, и да се регистрира целта в CMake.

4.2.1 Създаване на цел в CMake

Всяка програма живее в собствена поддиректория (например `examples/my_game`). В нея се поставя файл `CMakeLists.txt` с едно извикване на `boza_add_executable`:

```
boza_add_executable(my_game)
```

Фиг. 4.2 - Минимален CMake файл за нова програма

След това в кореновия `CMakeLists.txt` на проекта се добавя реда:

```
add_subdirectory(examples/my_game)
```

Фиг. 4.3 - Добавяне на проекта към главния `CMakeLists.txt`

4.2.2 Дефиниране на модул на програмата

Всяка програма трябва да дефинира C++ модул, чието наименование съвпада с целта в CMake и експортира аналогичен клас с името в `PascalCase`. Класът наследява `App` и може да реализира виртуалния метод `setup()`. Това е единствената входна точка - тя се извиква веднъж при стартиране на двигателя, преди първия кадър и всички стадии на системите.

```
export module my_game;
import boza;

export class MyGame : public boza::App
{
    void setup() override {}
};
```

Фиг. 4.4 - Минимална програма, наследяваща `App`

setup() е предназначен за създаване и конфигуриране на сцени и конфигуриране на параметрите на приложението. Всичко, което трябва да се случи преди главния цикъл, се поставя тук.

4.3 Използване на Boza

4.3.1 boza.app

boza.app отговаря за жизнения цикъл и глобалните настройки на приложението. Чрез него се контролират параметри като целевия брой кадри в секунда, режимът на прозореца и опцията да се прекрати програмата.

```
void MyGame::setup()
{
    App::target_fps = 0.0f; // без ограничение

    auto& input = Scene::persistent().root().add_component<InputCapture>();
    input.on<Action::Press>(Key::F11, &App::toggle_fullscreen);
    input.on<Action::Press>(Key::Esc, &App::quit);
}
```

Фиг. 4.5 - Типична употреба на boza.app в setup()

4.3.2 boza.common

boza.common предоставя Property и GlobalProperty за дефиниране на полета с getter/setter логика.

Property<Owner, Methods...> позволява дадено поле да изглежда като обикновена член-променлива, но при четене и писане автоматично да минава през зададени getter и setter методи. Полето се използва с обичайния синтаксис за присвояване и четене - без ръчно извикване на методи.

GlobalProperty<Methods...> е аналогично, но работи със статични методи или свободни функции, без конкретен обект-собственик. Може да се използва за глобални настройки на приложението.

```
class Health
{
    float value_{ 100.0f };
    GameObject game_object_;

    float get_value() const { return value_; }
    void set_value(float new_val)
    {
        if ((value_ = new_val) <= 0.0f) game_object_.destroy();
    }
}
```

```
public:
    [msvc::no_unique_address]
    Property<Health,
        &Health::get_value,
        &Health::set_value
    > value{ this };
};
```

Фиг. 4.6 - Компонент Health с Property, чийто setter автоматично унищожава обекта при неположителна стойност

Употребата изглежда като работа с обикновено поле:

```
struct DamageSystem : UpdateStage<DamageSystem, With<Health>>
{
    static void execute(Health& hp)
    {
        hp.value -= 10.0f * Time::delta_time();
        Log::info("Health: {}", hp.value);
    }
};
```

Фиг. 4.7 - Пример за използване на Health

GlobalProperty се използва по същия начин, но без обект. Например App::cursor_state е такова поле - четенето и писането му минават през вътрешни функции, но от гледна точка на потребителя изглежда като статична променлива:

```
App::cursor_state = CursorState::HiddenLocked;
if (App::cursor_state == CursorState::HiddenLocked) { /* ... */ }
```

Фиг. 4.8 - Използване на App::cursor_state

4.3.3 boza.core

boza.core предоставя три основни помощни инструмента:

- Time за достъп до времето на кадъра
- Log за структурирано регистриране на съобщения
- assert за утвърждение на условие по време на изпълнение; при провал хвърля грешка и извежда stacktrace.

Time::delta_time() връща времето в секунди, изминало от предишния кадър. Използва се за независими от честотата на кадрите изчисления - движение, анимации, таймери и т.н.

Log поддържа форматираны съобщения по нива (trace, debug, info, warn, error, critical) и форматира по стандарта на std::format.

```

struct FPSLoggerSystem : UpdateStage<FPSLoggerSystem>
{
    static void execute()
    {
        static float      time_passed{ 0.0f };
        static std::uint32_t frame_count{ 0 };
        constexpr float    log_interval{ 1.0f };
        ++frame_count;
        time_passed += Time::delta_time();
        if (time_passed >= log_interval)
        {
            Log::info("{:.2f} FPS", frame_count / time_passed);
            time_passed = 0.0f;
            frame_count = 0;
        }
    }
};

```

Фиг. 4.9 - Типична употреба на Time и Log, предоставени от boza.core

4.3.4 boza.ecs

4.3.4.1 Scene

Scene е логическа граница, която групира обекти. Има два специални глобални обекта:

- Scene::main - текущата работна сцена; обекти, създадени без явен родител, автоматично се поставят в нея.
- Scene::persistent - сцена, която живее за целия жизнен цикъл на приложението.

Подходяща за мениджъри, глобален вход и подобни обекти.

```

Scene::main = Scene::create("Gameplay");

Scene scene = Scene::get("OldScene");
scene.active = false;
scene.destroy();

```

Фиг. 4.10 - Създаване и управление на сцени.

4.3.4.2 GameObject

GameObject е именуван обект в йерархията. Всеки обект автоматично получава компонент Transform. Обектите могат да се създават самостоятелно, като деца на друг обект или директно в дадена сцена:

```

GameObject player = GameObject::create("Player");
GameObject weapon = GameObject::create("Weapon", player);
GameObject ui = GameObject::create("HUD", Scene::persistent());

```

Фиг. 4.11 - Трите варианта за създаване на GameObject

Компонентите се управляват чрез шаблонни методи:

```
// Добавяне
auto& cam = player.add_component<Camera>();

// Добавяне само ако не съществува, иначе връща съществуващия
auto& ic = player.ensure_component<InputCapture>();

// Четене - хвърля assert ако компонентът липсва
auto& t = player.get_component<Transform>();

// Безопасно четене - връща nullptr ако компонентът липсва
if (auto* c = player.try_get_component<Camera>())
    c->is_primary = true;

// Проверка за наличие
if (player.has_component<Camera>()) { /* ... */ }

// Премахване
player.remove_component<Camera>();
```

Фиг. 4.12 - Управление на компоненти

Активността се управлява на две нива. `active_self` е локалното включено/изключено състояние на конкретния обект. `active` е само за четене и отчита и активността спрямо йерархичното дърво - ако родителят е деактивиран, `active` е `false` дори когато `active_self` е `true`. Деактивираните обекти не се включват в изпълнението на системи, освен ако не е посочено изрично.

```
GameObject boss = GameObject::find("Boss");
GameObject hand = player.find_child("RightHand");
auto children = player.children(); // std::vector<GameObject>
```

Фиг. 4.13 - Търсене на обекти по име

4.3.4.3 Transform

Transform е компонентът за пространственото състояние на обект. Има два комплекта свойства: локални (спрямо родителя) и световни (в глобалното пространство). Може да се чете и пише и двата комплекта директно:

```
auto& t = go.get_component<Transform>();

// Локални стойности - спрямо родителя
t.local_position = glm::vec3{ 0.0f, 1.5f, 0.0f };
t.local_rotation = glm::angleAxis(glm::radians(45.f), glm::vec3{ 0, 1, 0 });
t.local_scale = glm::vec3{ 2.0f };

// Световни стойности - в глобалното пространство
glm::vec3 world_pos = t.position;
t.position = glm::vec3{ 10.0f, 0.0f, 5.0f };
```

```
// Ъгли на Ойлер - алтернатива на кватерниона
t.local_eulers = glm::vec3{ 0.0f, 90.0f, 0.0f }; // pitch, yaw, roll в
градуси

// Посочни вектори (само четене)
glm::vec3 fwd = t.forward; // посоката напред в световното пространство
glm::vec3 rgt = t.right;
glm::vec3 up = t.up;
```

Фиг. 4.14 - Работа с Transform

4.3.4.4 Системи

Системите съдържат логиката, която се изпълнява всеки кадър. Дефинират се като структури, наследяващи SystemStage (или псевдонимите като StartStage, UpdateStage, PhysicsStage, за да не се подава ръчно фаза). Първият шаблонен параметър е самата система, следват незадължителни спецификатори за филтриране на обекти: With<T> (задължителен компонент), Opt<T> (незадължителен) и Without<T> (изключване).

Параметрите на execute трябва да съответстват на спецификаторите - With<T> дава T&, Opt<T> дава T* (nullptr ако компонентът липсва), Without<T> не дава параметър. Системата без спецификатори се изпълнява веднъж на кадър без обхождане на обекти. Празните типове, които служат само за филтриране, също не дават параметър.

Редът на изпълнение между системи и други настройки се контролира чрез статичния метод config().

```
// Система без обекти - изпълнява се веднъж на кадър
struct TimerSystem : UpdateStage<TimerSystem>
{
    static void execute()
    {
        Log::info("Frame: {}", Time::frame_count());
    }
};

struct Health { float value{ 100.0f }; };
struct Shield { float value{ 0.0f }; };
struct DyingTag {};

// does something that must be done before DamageSystem
struct DummySystem: UpdateStage<DummySystem> { ... };

struct DamageSystem : UpdateStage<
    DamageSystem,
    With<Health>,
    Opt<const Shield>,
    Without<DyingTag>>
{
    static SystemStageConfig config()
    {
        return {
            .multi_threaded = false,
```



```

        .include_disabled = false,
        .run_after = { DummySystem::stage_info.system }
    };
}

static void execute(GameObject, Health& hp, const Shield* shield)
{
    const float absorb = shield ? shield->value : 0.0f;
    hp.value -= (5.0f - absorb) * Time::delta_time();
}
};

```

Фиг. 4.15 - Пример за система със и без ComponentList

4.3.4.5 Отношения

Освен компоненти, обектите поддържат типизирани отношения към други обекти. Отношението свързва два обекта и може да носи данни. Полезно за изразяване на семантика като "А преследва В" или "А е собственик на В" без да се правят глобални структури.

```

struct Chasing { float speed{ 5.0f }; };

// Добавяне на отношение с данни
enemy.add_relation<Chasing>(player).speed = 8.0f;

// Проверка
if (enemy.has_relation<Chasing>(player)) { /* ... */ }

// Четене на данните
auto& data = enemy.get_relation_data<Chasing>(player);

// Намиране на целевия обект
GameObject target = enemy.get_relation_target<Chasing>();

// Обхождане на всички обекти, с които enemy има Chasing отношение
enemy.for_each_with_relation<Chasing>([](GameObject other) { /* ... */ });

// Премахване
enemy.remove_relation<Chasing>(player);

```

Фиг. 4.16 - Отношения между обекти с данни

4.3.5 boza.input

boza.input предоставя два механизма за обработка на вход: реактивен чрез InputCapture и директен чрез статичните методи на Input.

InputCapture е компонент, който се прикрепя към даден GameObject. Той позволява регистриране на callback функции за конкретни действия. Действията са дефинирани чрез Action:

- Press, Release, Hold, DoubleClick - клавишни действия; изискват задаване на клавиш или комбинация.

- `MouseMove`, `MouseScroll` - движение на мишката; получават `glm::vec2` делта като параметър на callback функцията.

Клавишните callback функции се подават с клавиш или комбинация. Комбинациите се изграждат с `&` (едновременно) и `|` (алтернатива):

`MouseMove` се извиква всеки кадър, в който мишката се е преместила. Callback-ът получава делтата на движение като `glm::vec2`. Типично се проверява `App::cursor_state`, за да се игнорира движението, когато курсорът не е заключен.

`MouseScroll` работи аналогично - callback функцията получава `glm::vec2`, където `delta.x` е хоризонтален скрол, а `delta.y` - вертикален.

`Input::is_held` връща `true` ако клавишът е натиснат по принцип, `Input::is_pressed` връща `true` ако е бил натиснат този кадър.

```
auto& ic = player.ensure_component<InputCapture>();

ic.on<Action::Press>(Key::Space, [] { Log::info("Jump"); });
ic.on<Action::DoubleClick>(Key::MouseLeft,
    [] { Log::info("Double click with mouse"); });
ic.on<Action::Press>(
    Key::LCtrl & Key::S | Key::RCtrl & Key::S,
    [] { Log::info("Save"); }
);

ic.on<Action::MouseMove>([](const glm::vec2 delta)
{
    if (App::cursor_state != CursorState::HiddenLocked) return;

    camera_yaw += delta.x * sensitivity;
    camera_pitch += delta.y * sensitivity;
});

ic.on<Action::MouseScroll>([](const glm::vec2 delta)
{
    if (App::cursor_state != CursorState::HiddenLocked) return;
    fov = std::clamp(fov - delta.y * zoom_speed, min_fov, max_fov);
});

if (Input::is_held(Key::A)) { /* A се задържа */ }
if (Input::is_pressed(Key::B)) { /* B е натиснат точно този кадър */ }
```

Фиг. 4.17 - Вход чрез callback функции или директна проверка за състояние на клавиш

4.3.6 boza.gfx

`boza.gfx` предоставя всичко, необходимо за визуализация: буфери, текстури, семплери, mesh-ове, материали, компоненти за рендериране и изпълнение на compute шейдъри. Всички ресурси (освен буферите) се идентифицират по име и се управляват от двигателя - не е необходимо ръчно унищожаване.

4.3.6.1 Buffer

Buffer е GPU буфер с определено предназначение и режим на достъп. Използва се за съхранение на данни, достъпни за шейдъри - геометрия, uniform стойности, storage данни и т.н.

BufferUsage определя за какво ще се използва буферът, по принцип Uniform и Storage са двете използвани опции за крайния потребител.

ResourceAccessMode контролира вътрешното управление:

- Static - единичен буфер; подходящ за данни, записвани веднъж или рядко.
- Dynamic - буферът се репликира по кадри; подходящ за данни, обновявани всеки кадър.

```
Buffer values_buf{
    1024 * sizeof(float),
    BufferUsage::Storage,
    ResourceAccessMode::Static
};

// Качване на данни от CPU
std::vector<float> data(1024, 1.0f);
values_buf.upload(data);

// Качване с отместване (offset в байтове)
values_buf.upload(data.data(), 512 * sizeof(float), 256 * sizeof(float));

// Четене обратно от GPU към CPU (след compute операция)
auto raw = values_buf.read_back(); // std::vector<std::uint8_t>

// Четене на конкретен диапазон
std::vector<float> result(512);
values_buf.read_back(
    result.data(), 512 * sizeof(float), 128 * sizeof(float));
```

Фиг. 4.18 - Създаване и използване на Buffer

4.3.6.2 Texture

Texture е GPU текстура. Може да се създаде програмно с явни настройки или да се зареди от файл.

```
Texture& albedo = Texture::get_or_load("stone.png");
Texture& sky = Texture::get_or_load_cubemap("skybox.png");
```

Фиг. 4.19 - Зареждане на текстури от файл

При програмно създаване се задават тип, формат, размер и флагове за употреба. Флаговете трябва да покриват всички начини, по които текстурата ще се използва:

```
Texture& render_tex = Texture::create(
    "my_texture", {
        .type           = TextureType::Texture2D,
        .format          = TextureFormat::RGBA8,
        .access_mode     = ResourceAccessMode::Static,
        .width           = 512,
        .height          = 512,
        .usage_flags     = TextureUsage::Storage | // запис от compute шейдър
        TextureUsage::Sampled | // четене от fragment шейдър
        TextureUsage::TransferSrc | // четене обратно на CPU
        TextureUsage::TransferDst // качване от CPU
    });
```

Фиг. 4.20 - Програмно създаване на текстура с явни настройки

```
Texture& copy_tex = Texture::copy("original", "copy",
    ResourceAccessMode::Static);

std::vector<std::uint8_t> pixels = render_tex.read_back();
```

Фиг. 4.21 - Копиране и четене обратно

4.3.6.3 Sampler

Sampler описва как шейдърът семплира текстура при четене - режим на адресиране, филтриране, анизотропия. Семплерите могат да се дефинират в JSON файлове или да се създават програмно и се достъпват по име.

```
{
    "filter": "linear",
    "wrap_u": "repeat",
    "wrap_v": "repeat",
    "wrap_w": "repeat",
    "mipmap_mode": "linear",
    "mip_lod_bias": 0.0,
    "min_lod": 0.0,
    "max_lod": 1000.0,
    "max_anisotropy": 1.0
}
```

Фиг. 4.22 - Дефиниране на семплер като assets/samplers/custom_sampler.smpl.json

```
auto& s = Sampler::create(
    "custom_sampler",
    SamplerFilter::Linear,
    SamplerWrap::Repeat,
    SamplerWrap::Repeat,
    SamplerWrap::Repeat,
    SamplerFilter::Linear,
    0.0f, 0.0f, 1000.0f, 1.0f
);
```

Фиг. 4.23 - Аналогично дефиниране на семплер програмно

4.3.6.4 Mesh

Mesh съхранява геометрия (върхове и индекси за подредбата им) и се регистрира по име. За момента се създава само програмно с подаване на масиви от върхове и индекси за подредба.

```
std::vector<Vertex> verts = { /* ... */ };
std::vector<std::uint32_t> idxs = { /* ... */ };

Mesh::create("my_mesh", verts, idxs);
```

Фиг. 4.24 - Създаване на Mesh

4.3.6.5 Camera

Camera е компонент, добавян към GameObject. Поддържа перспективна и ортогографска проекция. Само един обект в сцената трябва да има `is_primary = true` - той се използва за рендериране.

```
GameObject cam_obj = GameObject::create("Camera");
auto& cam = cam_obj.add_component<Camera>();
cam.is_primary = true;
cam.fov = 60.0f;
cam.near_plane = 0.1f;
cam.far_plane = 1000.0f;
```

Фиг. 4.25 - Добавяне и конфигуриране на камера

4.3.6.6 MeshRenderer

MeshRenderer е компонент, свързващ обект с Mesh и материал. Могат да се задават имена на Mesh и Material и двигателят ги разрешава автоматично, когато очакваните по имена Mesh и Material бъдат създадени (в случай, че не са по време на присвояването).

```
auto& mr = go.add_component<MeshRenderer>();
mr.mesh_name = "my_mesh";
mr.material_name = "green_stone";

// Алтернативно - директно присвояване
mr.mesh = Mesh::get("my_mesh");
mr.material = Material::get("green_stone");
```

Фиг. 4.26 - Добавяне на MeshRenderer към обект

4.3.6.7 Material

Material свързва двойка шейдъри с параметри. Параметрите се задават по имена, съответстващи точно на декларациите в шейдърите.

За следния интерфейс:

```
// vertex shader
#version 450

layout(set = 0, binding = 0)
uniform CameraUBO { mat4 view; mat4 proj; } cameraUBO;

struct DrawData { mat4 model; vec4 something; };
layout(push_constant) uniform PushConstants { DrawData data; } pc;

// fragment shader
#version 450

layout(location = 0) out vec4 outColor;

layout(set = 0, binding = 1) uniform sampler2D albedo_map;
layout(set = 0, binding = 2)
uniform MaterialUBO { vec4 albedo_color; vec4 properties; } material;
```

Фиг. 4.27 - Примерен интерфейс на vertex и fragment шейдър

Материалът може да се създаде програмно:

```
Material& mat = Material::create("green_stone", {
    .vertex_shader = "my_lit",
    .fragment_shader = "my_lit"
});

mat["albedo_map"] = {
    Texture::get_or_load("stone.png"),
    Sampler::get("default")
};

mat["material.albedo_color"] = glm::vec4{ 0.2f, 0.8f, 0.3f, 1.0f };
mat["material.properties"]   = glm::vec4{ 0.0f, 0.7f, 0.2f, 0.0f };
mat["pc.something"]          = glm::vec4{ 2.0f, 0.2f, 0.56f, 0.5f };
```

Фиг. 4.28 - Създаване на материал според интерфейса на шейдърите

Или чрез JSON дефиниция като assets/materials/green_stone.mat.json:

```
{
  "vertex_shader": "my_lit",
  "fragment_shader": "my_lit",
  "load_strategy": "on_demand",
  "textures": {
    "albedo_map": { "texture": "stone.png", "sampler": "default" }
  },
  "params": {
    "material": {
      "albedo_color": [0.2, 0.8, 0.3, 1.0],
      "properties": [0.0, 0.7, 0.2, 0.0]
    },
    "pc": { "color_override": [0.2, 0.8, 0.2, 1.0] }
  }
}
```

Фиг. 4.29 - Аналогична дефиниция на материал

При дефиниране на материал в JSON файл `load_strategy` може да е `"on_demand"` (зареждане при първа употреба) или `"game_load"` (зареждане при инициализация).

Материалите се достъпват по име след създаване:

```
Material& mat = Material::get("green_stone");  
Material* matp = Material::try_get("green_stone"); // nullptr ако го няма
```

Фиг. 4.30 - Достъп до вече създаден материал

`MaterialSettings` поддържа и допълнителни параметри на рендериращото състояние:

```
Material& mat = Material::create(  
    "transparent", {  
        .vertex_shader      = "my_lit",  
        .fragment_shader    = "my_lit",  
        .depth_test_enable  = true,  
        .depth_write_enable = false,           // без запис в depth buffer  
        .cull_mode          = CullMode::None // двустранно рендериране  
    });
```

Фиг. 4.31 - `MaterialSettings` с нестандартни параметри за рендериране

Аналогично може да се настрои по този начин и в JSON файла:

```
{  
  "vertex_shader": "my_lit",  
  "fragment_shader": "my_lit",  
  "load_strategy": "on_demand",  
  "settings": {  
    "depth_test": true,  
    "depth_write": false,  
    "cull_mode": "none"  
  },  
  "textures": {  
    "albedo_map": {  
      "texture": "stone.png",  
      "sampler": "default"  
    }  
  },  
  "params": {  
    "material": {  
      "albedo_color": [0.2, 0.8, 0.3, 1.0],  
      "properties": [0.0, 0.7, 0.2, 0.0]  
    },  
    "pc": {  
      "color_override": [0.2, 0.8, 0.2, 1.0]  
    }  
  }  
}
```

Фиг. 4.32 - Дефиниция на материал в JSON файл с нестандартни настройки

4.3.6.8 ComputeDispatcher

ComputeDispatcher изпълнява compute шейдър. Ресурсите и push константите се свързват по имена, идентични с декларациите в шейдъра.

Следният пример показва типичен compute шейдър с изходна текстура, storage буфер и push константи:

```
#version 450

layout(local_size_x = 16, local_size_y = 16) in;

layout(binding = 0, rgba8) writeonly uniform image2D outImage;

layout(set = 0, binding = 1, std430) buffer Values
{
    float data[];
} values;

layout(push_constant) uniform ComputePC
{
    uint count;
    float scale;
} pc;
```

Фиг. 4.33 - Интерфейс на compute шейдър

Изпълнението от C++:

```
constexpr std::uint32_t count = 1024;
Buffer values{
    count * sizeof(float),
    BufferUsage::Storage,
    ResourceAccessMode::Static
};

Texture& out_tex = Texture::create("compute_out", {
    .type = TextureType::Texture2D,
    .format = TextureFormat::RGBA8,
    .access_mode = ResourceAccessMode::Static,
    .width = 256, .height = 256,
    .usage_flags = TextureUsage::Storage | TextureUsage::Sampled |
                  TextureUsage::TransferDst | TextureUsage::TransferSrc
});

bool failed = false;
ComputeDispatcher dispatcher{ "my_compute", failed };
dispatcher
    .set("out_image", out_tex)
    .set("values", values)
    .set("pc.count", count)
    .set("pc.scale", 0.5f)
    .dispatch(256, 256, 1) // двигателят разделя на работни групи автоматично
    .wait();              // изчаква GPU преди четене на резултат

if (!failed) values.read_back(cpu_values.data(), count * sizeof(float), 0);
```

Фиг. 4.34 - ComputeDispatcher с текстура, буфер и push константи

`dispatch(width, height, depth)` приема размера на данните в елементи - двигателят сам разделя на работни групи според `local_size` на шейдъра. При нужда от явен контрол върху броя работни групи се използва `dispatch_groups(x, y, z)`.

`dispatch` и `dispatch_groups` са асинхронни и `wait()` блокира до края на GPU операцията. Ако резултатът не е необходим веднага, `wait()` може да се извика чак когато резултатът от шейдъра е нужен.

ЗАКЛЮЧЕНИЕ

В рамките на дипломната работа е проектиран и реализиран Boza - лек игрови двигател на C++23, организиран около ясно разграничени модули с контролирана видимост и минимално пресичане на отговорности. Архитектурата обхваща управление на жизнения цикъл на приложението, поетапен ECS модел на изпълнение чрез flecs, RHI слой за абстракция на графичния хардуер с реализация на Vulkan, система за управление на ресурси и материали, управлявана от данни чрез външни JSON дефиниции, и офлайн инструментална верига за обработка на шейдъри, генерираща SPIR-V байткод и метаданни за използване по време на изпълнение.

Ключово архитектурно решение е строгото разграничение между публичния интерфейс на двигателя и вътрешните му реализационни детайли. Модулите boza.app, boza.gfx, boza.ecs и boza.input образуват стабилна повърхност за потребителя на двигателя, докато boza.platform, boza.rhi, boza.rhi.vulkan и boza.detail остават изцяло вътрешни и недостъпни за потребителя.

Системата от метаданни, извлечени чрез рефлексия на шейдърите, позволява автоматизирано обвързване на ресурси по време на изпълнение, без ръчно дефиниране на дескрипторни множества в потребителския код. В съчетание с материалния модел на база JSON дефиниции това осигурява ясна граница между код и съдържание.

Поради ограниченото време за разработка проектът е насочен към изграждане на стабилна архитектурна основа, а не към пълна функционалност на завършен продукт двигател. Реализацията е ориентирана към Windows и MSVC, тъй като поддръжката на C++ модули е най-зряла именно за тази среда, но архитектурата не поставя ограничения пред бъдеща поддръжка на допълнителни платформи и графични бекенди.

Естествена посока за бъдещо развитие на двигателя обхваща няколко области:

- С узряването на поддръжката на C++ модули при Clang и GCC, компилирането на проекта може да бъде разширено и за тях, което ще премахне текущата обвързаност с MSVC и Windows.
- Архитектурата на RHI слоя позволява добавяне на реализации за OpenGL, DirectX и Metal без промяна на публичния интерфейс.
- Преминаване към конфигуруем rendering pipeline за GPU-driven рендериране - вместо сегашния модел на CPU culling и директни draw извиквания, процесорът ще предоставя само необходимите данни, а видеокартата сама ще извършва culling и ще

генерира командите за изобразяване през потребителски дефинирани етапи на конвейера.

- Интегриране на библиотека за физика като Jolt или Bullet за базова симулация на твърди тела и засичане на сблъсъци като самостоятелна система в ECS модела.
- Сериализация и десериализация на сцени и prefab система, която би позволила повторно използване на йерархии от обекти с конфигурируеми параметри.
- Редактор на сцени и UI система

ИЗТОЧНИЦИ

1. C++ справочник: <https://en.cppreference.com/>
2. C++23 модули – стандарт P1103R3:
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1103r3.pdf>
3. Vulkan спецификация: <https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html>
4. Vulkan наръчник: <https://vkguide.dev/>
5. Vulkan Tutorial: <https://vulkan-tutorial.com/>
6. LearnOpenGL: <https://learnopengl.com/>
7. Gregory, J. Game Engine Architecture (3rd ed.). CRC Press, 2018.
8. CMake документация: <https://cmake.org/documentation/>
9. CMake GitHub хранилище: <https://github.com/Kitware/CMake>
10. vcpkg документация: <https://learn.microsoft.com/en-us/vcpkg/>
11. vcpkg пакетен индекс: <https://vcpkg.link/>
12. Flecs документация: <https://www.flecs.dev/flecs/>
13. Flecs GitHub хранилище: <https://github.com/SanderMertens/flecs>
14. EnTT GitHub хранилище: <https://github.com/skypjack/entt>
15. EnTT документация: <https://skypjack.github.io/entt/>
16. Vulkan Memory Allocator GitHub хранилище:
<https://github.com/GPUOpen-LibrariesAndSDKs/VulkanMemoryAllocator>
17. Volk GitHub хранилище: <https://github.com/zeux/volk>
18. SPIRV-Cross GitHub хранилище: <https://github.com/KhronosGroup/SPIRV-Cross>
19. SPIRV-Tools GitHub хранилище: <https://github.com/KhronosGroup/SPIRV-Tools>
20. shaderc GitHub хранилище: <https://github.com/google/shaderc>
21. GLFW документация: <https://www.glfw.org/documentation.html>
22. GLM GitHub хранилище: <https://github.com/g-truc/glm>
23. stb GitHub хранилище: <https://github.com/nothings/stb>
24. nlohmann/json GitHub хранилище: <https://github.com/nlohmann/json>
25. GTL GitHub хранилище: <https://github.com/greg7mdp/gtl>
26. spdlog GitHub хранилище: <https://github.com/gabime/spdlog>

СЪДЪРЖАНИЕ

УВОД.....	2
ГЛАВА 1. ПРЕГЛЕД НА СЪЩЕСТВУВАЩИ РЕШЕНИЯ И ПРАКТИКИ ПРИ РАЗРАБОТКАТА НА ИГРОВИ ДВИГАТЕЛИ.....	3
1.1 Категории игрови двигатели и типични профили на употреба.....	3
1.1.1 Големи универсални комерсиални двигатели.....	3
1.1.2 Универсални двигатели за бързо прототипиране и мултиплатформена разработка.....	4
1.1.3 Вътрешнофирмени двигатели.....	4
1.2 Развойни практики и технологии при разработката на игрови двигатели.....	5
1.2.1 Езици за програмиране.....	5
1.2.2 Графични програмни интерфейси и архитектурен контрол.....	5
1.2.3 ECS и други архитектурни модели за управление на игрови обекти.....	6
1.2.4 Платформен слой за прозорци и вход.....	6
1.2.5 Build и dependency практики.....	6
1.2.6 Конвейери за шейдъри и ресурси.....	6
1.2.7 Валидация, диагностика и профилиране.....	7
1.3 Изводи от глава 1.....	7
ГЛАВА 2. ИЗИСКВАНИЯ КЪМ ПРОЕКТА И АРГУМЕНТИРАН ИЗБОР НА ТЕХНОЛОГИИТЕ.....	8
2.1 Формални изисквания към програмния продукт.....	8
2.2 Функционални и нефункционални изисквания.....	8
2.2.1 Жизнен цикъл на приложението и цикъл на изпълнение.....	8
2.2.2 ECS сцена, обект и поетапни системи.....	8
2.2.3 Управление на графични ресурси и материали.....	8
2.2.4 Изпълнение на графика и изчисления.....	9
2.2.5 Вход и платформена интеграция.....	9
2.2.6 Интеграция на инструменти за шейдъри.....	9
2.2.7 Гъвкавост и разширяемост.....	9
2.3 Аргументиран избор на технологии.....	9
2.3.1 C++23 с модули.....	9
2.3.2 Ориентация към Windows в началния етап.....	10
2.3.3 Flems като основа за ECS.....	10
2.3.4 GLFW за прозорец и вход.....	11
2.3.5 CMake и vcpkg за изграждане и зависимости.....	11
2.3.6 Vulkan като първа реализация под RHI.....	11
2.4 Модулна организация на проекта.....	12
2.4.1 Публични модули на двигателя.....	12
2.4.2 Вътрешни модули и инструменти.....	12
2.5 Управление на ресурси чрез данни и модел на метаданни за шейдъри.....	12

2.5.1 JSON конфигурационни файлове.....	12
2.5.2 Дефиниции на материали и семплери.....	13
2.5.3 Метаданни от рефлексия на шейдърите.....	13
2.6 Изводи от глава 2.....	14
ГЛАВА 3. РЕАЛИЗАЦИЯ НА ПРОЕКТА.....	15
3.1 Обзор на архитектурата на изпълнението.....	15
3.2 Модул boza.app.....	15
3.2.1 App.....	15
3.2.2 GameLoop.....	16
3.2.3 GameSettings.....	17
3.3 Модул boza.common.....	17
3.3.1 Property и GlobalProperty.....	17
3.3.2 Flags.....	20
3.3.3 Random.....	20
3.3.4 Преекспортиране на GLM и контейнери от GTL.....	21
3.4 Модул boza.core.....	21
3.4.1 Time.....	21
3.4.2 Log.....	22
3.4.3 assert.....	22
3.5 Модул boza.ecs.....	22
3.5.1 GameObject.....	22
3.5.2 Scene.....	24
3.5.3 SystemStage.....	25
3.5.4 SystemStageConfig.....	26
3.5.5 ComponentList.....	27
3.5.6 SystemRegistry.....	28
3.5.7 Transform.....	29
3.5.8 TransformSystem.....	30
3.6 Модул boza.input.....	32
3.6.1 Key, Action, KeyCombo, KeyBinding.....	32
3.6.2 KeyState.....	33
3.6.3 CursorState.....	33
3.6.4 InputCapture.....	33
3.6.5 InputSystem.....	34
3.7 Модул boza.platform.....	37
3.7.1 Window.....	37
3.8 Модул boza.gfx.....	38
3.8.1 Архитектурна роля и граници на модула.....	38
3.8.2 Общи структури за модула.....	38
3.8.3 Buffer.....	39
3.8.4 Texture и TextureLoader.....	40
3.8.5 Sampler и SamplerLoader.....	41
3.8.6 Mesh.....	42

3.8.7 Material и MaterialLoader.....	43
3.8.7.1 Material.....	43
3.8.7.2 MaterialLoader.....	46
3.8.8 Компоненти за рендериране.....	47
3.8.8.1 Camera.....	47
3.8.8.2 MeshRenderer.....	47
3.8.9 ComputeDispatcher.....	48
3.8.9.1 Настройка на Pipeline и дескрипторни множества.....	48
3.8.9.2 Изпълнение на dispatch.....	50
3.8.10 RenderingSystem.....	50
3.8.10.1 Състав и подредба на стадии.....	50
3.8.10.2 EngineBegin и EngineDestroy.....	52
3.8.10.3 Топология на кеша и рендерирането.....	54
3.8.10.4 SanityCheck.....	55
3.8.10.5 ProcessMeshChanged, ProcessMaterialChanged и ProcessInvalidated.....	55
3.8.10.6 BeginFrame и CameraUboUpdate.....	56
3.8.10.7 CullEntities.....	57
3.8.10.8 Подаване на draw заявки и логика за инстанциране.....	57
3.8.10.9 EndFrame.....	58
3.9 Модул boza.rhi.....	59
3.9.1 Подмодул boza.rhi.api.....	60
3.9.2 Подмодул boza.rhi.objects.....	60
3.9.2.1 Основен протокол за създаване.....	60
3.9.2.2 Instance.....	61
3.9.2.3 Device.....	62
3.9.2.4 Swapchain.....	62
3.9.2.5 Командни обекти.....	63
3.9.2.5.1 CommandPool.....	63
3.9.2.5.2 CommandBuffer.....	64
3.9.2.5.3 CommandQueue.....	65
3.9.2.6 Обекти за синхронизация.....	65
3.9.2.7 Дескрипторни обекти.....	66
3.9.2.8 Ресурсни обекти.....	67
3.9.2.9 ShaderModule и Metadata.....	68
3.9.2.10 Конвейерни обекти.....	69
3.9.3 Подмодул boza.rhi.factory.....	70
3.9.4 PipelineBuilder.....	70
3.9.4.1 Създаване на DescriptorSetLayout.....	70
3.9.4.2 Създаване на PipelineLayout, GraphicsPipeline и ComputePipeline.....	71
3.9.5 DescriptorReflection.....	72
3.9.6 ResourceCache.....	73
3.9.7 RenderContext.....	74
3.9.8 BufferLayoutRules.....	74
3.9.9 Помощни функции за проверка на тип и размер.....	75

3.10 Модул boza.rhi.vulkan.....	76
3.10.1 vk::Instance.....	76
3.10.2 vk::Device.....	77
3.10.2.1 Последователност на създаване и необходими зависимости.....	77
3.10.2.2 Избор на физическо устройство.....	77
3.10.2.3 Откриване на семейства опашки.....	78
3.10.3 vk::Allocator.....	78
3.10.4 vk::Swapchain.....	79
3.10.4.1 Последователност на инициализацията.....	80
3.10.4.2 Цикъл за придобиване и представяне.....	80
3.10.4.3 Динамични граници на рендериращия проход.....	81
3.10.4.4 Пресъздаване на swapchain-а.....	82
3.10.5 Командни обекти.....	83
3.10.5.1 vk::CommandPool.....	83
3.10.5.2 vk::CommandBuffer.....	83
3.10.5.3 vk::CommandQueue.....	84
3.10.6 Vulkan реализация на обектите за синхронизация.....	85
3.10.6.1 vk::Semaphore.....	85
3.10.6.2 vk::Fence.....	85
3.10.7 Vulkan реализация на дескрипторните обекти.....	86
3.10.7.1 vk::DescriptorPool.....	86
3.10.7.2 vk::DescriptorSetLayout.....	86
3.10.7.3 vk::DescriptorSet.....	87
3.10.8 Vulkan реализация на ресурсните обекти.....	88
3.10.8.1 vk::Buffer.....	88
3.10.8.2 vk::Texture.....	89
3.10.8.3 vk::Sampler.....	89
3.10.9 vk::ShaderModule.....	90
3.10.10 Vulkan реализация на конвейерните обекти.....	90
3.10.10.1 vk::PipelineLayout.....	90
3.10.10.2 vk::GraphicsPipeline.....	91
3.10.10.3 vk::ComputePipeline.....	91
3.11 Модул boza.detail.....	92
3.11.1 AssetPaths.....	92
3.11.2 FileIO.....	92
3.11.3 ImageIO.....	92
3.12 Инструмент shader_processor.....	93
3.12.1 Роля на инструмента.....	93
3.12.2 IncludeResolver.....	93
3.12.3 ShaderCompiler.....	94
3.12.4 SpirvOptimizer.....	95
3.12.5 ShaderDecompiler.....	95
3.12.6 ShaderReflector.....	96
3.12.7 ShaderProcessor.....	97
3.13 CMake конфигурация.....	97

3.13.1 Главен CMakeLists.txt.....	97
3.13.2 BozaOptions.....	98
3.13.3 BozaCompilerWarnings.....	98
3.13.4 BozaUtils.....	99
3.13.5 BozaHeaderUnits.....	99
3.13.6 BozaHeaderUnitPatches.....	100
3.13.7 BozaAssets и BozaExecutable.....	100
3.13.8 CMakeLists.txt на Boza.....	101
3.13.9 CMakeLists.txt на shader_processor.....	101
3.13.10 CMakeLists.txt на шейдърите.....	101
3.14 Примерни приложения.....	102
3.14.1 MaterialShowcase.....	102
3.14.1.1 Генериране на текстура с compute шейдър.....	102
3.14.1.2 Програмно създаване на материали.....	104
3.14.1.3 Управление на сцени.....	104
3.14.1.4 Системи.....	105
3.14.2 InstancingExample.....	106
3.14.2.1 Генериране на терен.....	106
3.14.2.2 Поставяне на тревата.....	108
ГЛАВА 4. РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ.....	110
4.1 Компилиране и стартиране.....	110
4.1.1 Компилиране и стартиране от терминал.....	110
4.1.2 Компилиране и стартиране през VisualStudio 2022.....	110
4.2 Създаване на програма.....	111
4.2.1 Създаване на цел в CMake.....	111
4.2.2 Дефиниране на модул на програмата.....	111
4.3 Използване на Boza.....	112
4.3.1 boza.app.....	112
4.3.2 boza.common.....	112
4.3.3 boza.core.....	113
4.3.4 boza.ecs.....	114
4.3.4.1 Scene.....	114
4.3.4.2 GameObject.....	114
4.3.4.3 Transform.....	115
4.3.4.4 Системи.....	116
4.3.4.5 Отношения.....	117
4.3.5 boza.input.....	117
4.3.6 boza.gfx.....	118
4.3.6.1 Buffer.....	119
4.3.6.2 Texture.....	119
4.3.6.3 Sampler.....	120
4.3.6.4 Mesh.....	121
4.3.6.5 Camera.....	121
4.3.6.6 MeshRenderer.....	121
4.3.6.7 Material.....	121
4.3.6.8 ComputeDispatcher.....	124

ЗАКЛЮЧЕНИЕ.....	126
ИЗТОЧНИЦИ.....	128
СЪДЪРЖАНИЕ.....	129